

LAPORAN PROJECT DOCKER MODUL 1 - 6

Dosen Pengampu: Dr Ferry Astika Saputra ST, M.Sc



Disusun oleh:

Anindya Zalfa Inayyah 3124500004

Elfandi Rais Mahdavikia 3124500006

Muhammad Dito Akmal Putra Dimas 3124500017

2 D3 Teknik Informatika A

**PROGRAM STUDI D3 TEKNIK INFORMATIKA
POLITEKNIK ELEKTRONIKA NEGERI SURABAYA
2025/2026**

Modul 1

Docker dan Instalasi



A. TUJUAN PEMBELAJARAN

Setelah praktikum ini, mahasiswa mampu:

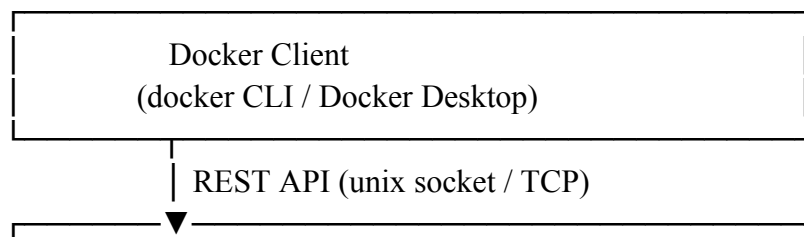
1. Menjelaskan perbedaan mendasar antara virtualisasi tradisional (VM) dan containerization
2. Memahami arsitektur Docker: Docker Engine, Docker Daemon, Docker CLI, dan Docker Registry
3. Menginstal Docker Engine di Ubuntu 22.04 menggunakan repository resmi
4. Menginstal Docker Desktop di Windows 10/11 dengan WSL2 backend
5. Menjalankan container pertama (hello-world, nginx, ubuntu) dan memahami lifecycle container
6. Menggunakan perintah dasar Docker: run, ps, stop, rm, images, pull, exec, logs
7. Memahami konsep Docker Image, Layer, dan Docker Hub sebagai registry publik
8. Menulis Dockerfile sederhana dan melakukan build image custom

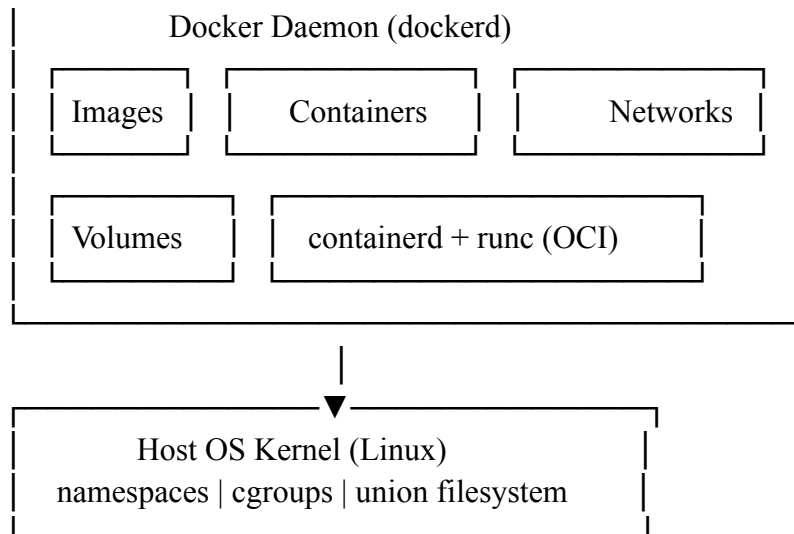
B. DASAR TEORI

1. Virtualisasi vs Containerization

Aspek	Virtual Machine (VM)	Container (Docker)
Isolasi	Full OS per instance	Shared kernel, isolated userspace
Ukuran	GB (include guest OS)	MB (hanya app + dependencies)
Startup	Menit	Detik
Overhead	Tinggi (hypervisor + guest OS)	Rendah (langsung di host kernel)
Portabilitas	Image besar, format bervariasi	Image ringan, standar OCI
Use Case	Multi-OS, strong isolation	Microservices, CI/CD, scaling

2. Arsitektur Docker



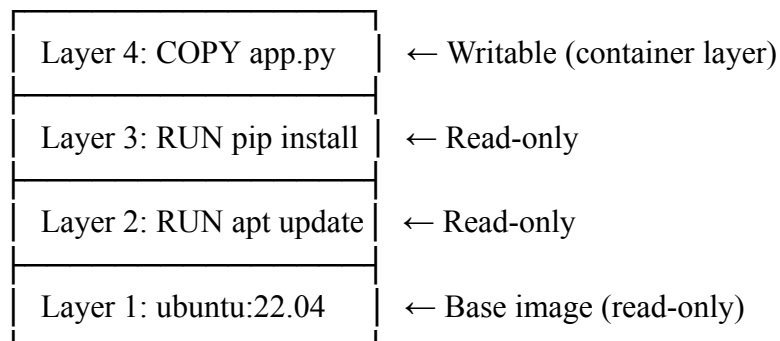


Komponen utama:

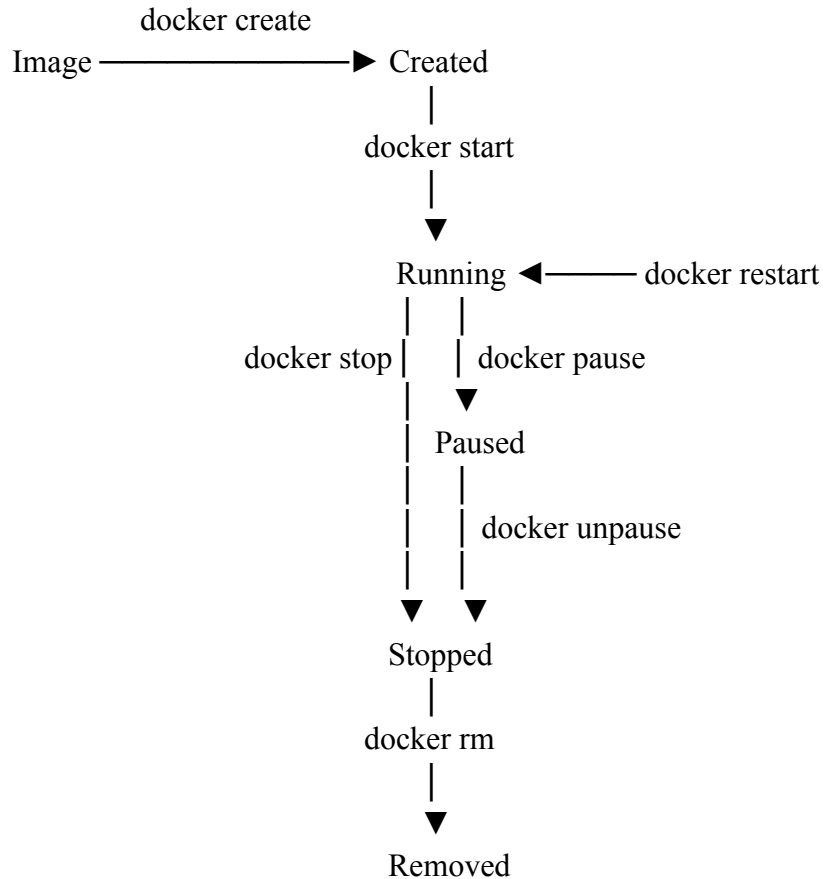
- Docker Client (docker): CLI yang mengirim perintah ke Docker Daemon melalui REST API
- Docker Daemon (dockerd): Background service yang mengelola image, container, network, dan volume
- containerd: Container runtime yang mengelola lifecycle container
- runc: Low-level runtime yang membuat dan menjalankan container sesuai spesifikasi OCI
- Docker Registry: Tempat penyimpanan image (Docker Hub, GitHub Container Registry, private registry)

3. Konsep Image dan Layer

Docker Image dibangun secara berlapis (layered filesystem). Setiap instruksi di Dockerfile menghasilkan satu layer. Layer bersifat read-only dan di-share antar image yang memiliki base sama, menghemat disk dan bandwidth.



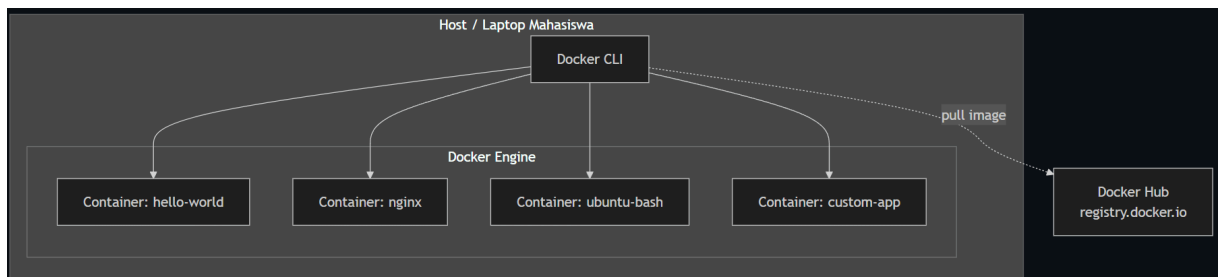
4. Container Lifecycle



5. Docker di Windows: WSL2 Backend

Docker Desktop di Windows menggunakan WSL2 (Windows Subsystem for Linux 2) sebagai backend. WSL2 menjalankan kernel Linux asli di dalam lightweight VM yang dikelola Windows, sehingga Docker container berjalan secara native di kernel Linux tanpa overhead emulasi.

C. TOPOLOGI LAB



D. PRAKTIKUM

Langkah 0: Persiapan Environment

1. Pastikan VM/Host sudah terkoneksi internet untuk download image dari Docker Hub.

```

elfandi@elfandi-Latitude-7414:~$ ping -c 3 google.com
PING google.com (2001:4860:4802:32::78) 56 data bytes
64 bytes from any-in-2001-4860-4802-32--78.1e100.net (2001:4860:4802:32::78): icmp_seq=1 ttl=114 time=26.3 ms
64 bytes from any-in-2001-4860-4802-32--78.1e100.net (2001:4860:4802:32::78): icmp_seq=2 ttl=114 time=25.2 ms
64 bytes from any-in-2001-4860-4802-32--78.1e100.net (2001:4860:4802:32::78): icmp_seq=3 ttl=114 time=28.4 ms

--- google.com ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2002ms
rtt min/avg/max/mdev = 25.227/26.653/28.406/1.318 ms
elfandi@elfandi-Latitude-7414:~$ # Update package list
sudo apt update && sudo apt upgrade -y
[sudo] password for elfandi:
Hit:1 http://id.archive.ubuntu.com/ubuntu noble InRelease
Get:2 http://id.archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Hit:3 https://dl.google.com/linux/chrome-stable/deb stable InRelease
Hit:4 https://brave-browser-apt-release.s3.brave.com stable InRelease
Get:5 http://id.archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:6 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:7 http://id.archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [1,969 kB]

```

Sebelum memulai menginstall docker, pastikan sudah terhubung ke internet dan melakukan sudo apt update untuk mengupdate package list.

Langkah 1: Instalasi Docker Engine di Ubuntu 22.04

1. Hapus versi lama (jika ada)

```

elfandi@elfandi-Latitude-7414:~$ sudo apt remove -y docker docker-engine docker.io containerd runc 2>/dev/null
[sudo] password for elfandi:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Package 'docker' is not installed, so not removed
elfandi@elfandi-Latitude-7414:~$

```

Langkah ini digunakan untuk menghapus instalasi Docker lama yang mungkin masih ada di sistem agar tidak terjadi konflik dengan versi terbaru. Perintah ini akan membersihkan paket seperti docker, containerd, dan runc jika sebelumnya sudah terinstal. Dengan begitu, sistem menjadi bersih sebelum instalasi Docker Engine yang baru dilakukan.

2. Instal dependensi

```

Package docker is not installed, so not removed
elfandi@elfandi-Latitude-7414:~$ sudo apt install -y \
    ca-certificates \
    curl \
    gnupg \
    lsb-release
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
ca-certificates is already the newest version (20240203).
ca-certificates set to manually installed.
curl is already the newest version (8.5.0-2ubuntu10.9).
gnupg is already the newest version (2.4.4-2ubuntu17.4).
gnupg set to manually installed.
lsb-release is already the newest version (12.0-2).
lsb-release set to manually installed.
The following packages were automatically installed and are no longer required:
  libgl1-amber-dri libglapi-mesa python3-netifaces
Use 'sudo apt autoremove' to remove them.
0 upgraded, 0 newly installed, 0 to remove and 2 not upgraded.

```

Pada tahap ini dilakukan instalasi paket-paket pendukung yang dibutuhkan Docker agar dapat berjalan dengan baik di Ubuntu. Paket seperti curl, ca-certificates, gnupg, dan lsb-release diperlukan untuk proses download repository, validasi keamanan, dan pengelolaan sistem. Dependensi ini wajib diinstal sebelum menambahkan repository Docker.

3. Tambahkan Docker GPG key dan repository

```

elfandi@elfandi-Latitude-7414:~$ sudo install -m 0755 -d /etc/apt/keyrings
elfandi@elfandi-Latitude-7414:~$ curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \
    sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg

sudo chmod a+r /etc/apt/keyrings/docker.gpg
elfandi@elfandi-Latitude-7414:~$ echo \
"deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] \
https://download.docker.com/linux/ubuntu \
$(lsb_release -cs) stable" | \
sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

```

Langkah ini bertujuan untuk menambahkan kunci keamanan (GPG key) dan repository resmi Docker ke dalam sistem Ubuntu. GPG key digunakan untuk memastikan paket yang diunduh berasal dari sumber resmi dan tidak dimodifikasi. Setelah itu, repository Docker ditambahkan agar sistem bisa mengunduh Docker Engine langsung dari server resmi Docker.

4. Install Docker Engine

```
elfandi@elfandi-Latitude-7414:~$ sudo apt update

sudo apt install -y \
    docker-ce \
    docker-ce-cli \
    containerd.io \
    docker-buildx-plugin \
    docker-compose-plugin
Hit:1 http://id.archive.ubuntu.com/ubuntu noble InRelease
Hit:2 http://id.archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:3 http://id.archive.ubuntu.com/ubuntu noble-backports InRelease
Hit:4 https://brave-browser-apt-release.s3.brave.com stable InRelease
Hit:5 https://dl.google.com/linux/chrome-stable/deb stable InRelease
```

Pada tahap ini dilakukan instalasi utama Docker Engine beserta komponen pendukungnya seperti docker-ce, docker-cli, containerd, dan plugin tambahan. Setelah repository ditambahkan, sistem melakukan update dan menginstal paket Docker dari sumber resmi. Tahap ini menghasilkan Docker yang siap digunakan di sistem.

5. Konfigurasi user non-root

```
elfandi@elfandi-Latitude-7414:~$ sudo usermod -aG docker $USER
elfandi@elfandi-Latitude-7414:~$ newgrp docker
```

Langkah ini digunakan untuk menambahkan user ke dalam grup Docker agar bisa menjalankan perintah Docker tanpa menggunakan sudo. Setelah user dimasukkan ke grup docker, perubahan harus diaktifkan menggunakan newgrp atau login ulang. Hal ini bertujuan agar penggunaan Docker lebih mudah dan tidak selalu membutuhkan hak akses root.

6. Verifikasi instalasi

```
elfandi@elfandi-Latitude-7414:~$ docker version
Client: Docker Engine - Community
 Version:           29.4.3
 API version:       1.54
 Go version:        go1.26.2
 Git commit:        055a478
 Built:             Wed May  6 17:07:36 2026
 OS/Arch:           linux/amd64
 Context:           default

Server: Docker Engine - Community
 Engine:
  Version:           29.4.3
  API version:       1.54 (minimum version 1.40)
  Go version:        go1.26.2
  Git commit:        56be731
  Built:             Wed May  6 17:07:36 2026
  OS/Arch:           linux/amd64
  Experimental:      false
 containerd:
  Version:           v2.2.3
  GitCommit:        77c84241c7cbdd9b4eca2591793e3d4f4317c590
 runc:
  Version:           1.3.5
  GitCommit:        v1.3.5-0-g488fc13e
 docker-init:
  Version:           0.19.0
  GitCommit:        de40ad0
```

```
elfandi@elfandi-Latitude-7414:~$ docker info
Client: Docker Engine - Community
Version: 29.4.3
Context: default
Debug Mode: false
Plugins:
  buildx: Docker Buildx (Docker Inc.)
    Version: v0.33.0
    Path: /usr/libexec/docker/cli-plugins/docker-buildx
  compose: Docker Compose (Docker Inc.)
    Version: v5.1.3
    Path: /usr/libexec/docker/cli-plugins/docker-compose

Server:
Containers: 0
  Running: 0
  Paused: 0
  Stopped: 0
Images: 0
Server Version: 29.4.3
Storage Driver: overlayfs
  driver-type: io.containerd.snapshotter.v1
Logging Driver: json-file
Cgroup Driver: systemd
Cgroup Version: 2
Plugins:
  Volume: local
  Network: bridge host ipvlan macvlan null overlay
  Log: awslogs fluentd gcplogs gelf journald json-file local splunk syslog
CDI spec directories:
  /etc/cdi
  /var/run/cdi
Swarm: inactive
Runtimes: runc io.containerd.runc.v2
Default Runtime: runc
Init Binary: docker-init
containerd version: 77c84241c7cbdd9b4eca2591793e3d4f4317c590
runc version: v1.3.5-0-g488fc13e
init version: de40ad0
Security Options:
  apparmor
  seccomp
   Profile: builtin
  cgroupns
Kernel Version: 6.17.0-22-generic
Operating System: Ubuntu 24.04.4 LTS
```

```

elfandi@elfandi-Latitude-7414:~$ sudo systemctl status docker
● docker.service - Docker Application Container Engine
   Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; preset: enabled)
   Active: active (running) since Sun 2026-05-10 11:03:20 WIB; 1min 29s ago
 TriggeredBy: ● docker.socket
    Docs: https://docs.docker.com
   Main PID: 30817 (dockerd)
     Tasks: 10
    Memory: 25.3M (peak: 28.3M)
       CPU: 460ms
    CGroup: /system.slice/docker.service
            └─30817 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock

May 10 11:03:20 elfandi-Latitude-7414 dockerd[30817]: time="2026-05-10T11:03:20.006208112Z"
May 10 11:03:20 elfandi-Latitude-7414 dockerd[30817]: time="2026-05-10T11:03:20.035686941Z"
May 10 11:03:20 elfandi-Latitude-7414 dockerd[30817]: time="2026-05-10T11:03:20.041074345Z"
May 10 11:03:20 elfandi-Latitude-7414 dockerd[30817]: time="2026-05-10T11:03:20.388528311Z"
May 10 11:03:20 elfandi-Latitude-7414 dockerd[30817]: time="2026-05-10T11:03:20.401069466Z"
May 10 11:03:20 elfandi-Latitude-7414 dockerd[30817]: time="2026-05-10T11:03:20.401195506Z"
May 10 11:03:20 elfandi-Latitude-7414 dockerd[30817]: time="2026-05-10T11:03:20.415407352Z"
May 10 11:03:20 elfandi-Latitude-7414 dockerd[30817]: time="2026-05-10T11:03:20.426556173Z"
May 10 11:03:20 elfandi-Latitude-7414 dockerd[30817]: time="2026-05-10T11:03:20.426652825Z"
May 10 11:03:20 elfandi-Latitude-7414 systemd[1]: Started docker.service - Docker Application Container Engine
elfandi@elfandi-Latitude-7414:~$ docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
4f55086f7dd0: Pull complete
d5e71e642bf5: Download complete
Digest: sha256:f9078146db2e05e794366b1bfe584a14ea6317f44027d10ef7dad65279026885
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

```

Tahap ini dilakukan untuk memastikan Docker sudah terinstal dengan benar dan dapat berjalan dengan baik. Perintah seperti `docker version`, `docker info`, dan `systemctl status docker` digunakan untuk mengecek kondisi instalasi dan service Docker. Selain itu, percobaan menjalankan container `hello-world` dilakukan untuk memastikan Docker Engine berfungsi normal.

Langkah 2: Instalasi Docker Desktop di Windows 10/11

1. Aktifkan WSL2

Buka PowerShell sebagai Administrator:

```

PS C:\Users\ASUS> wsl --install
The requested operation requires elevation.
Downloading: Windows Subsystem for Linux 2.6.3
Installing: Windows Subsystem for Linux 2.6.3
Windows Subsystem for Linux 2.6.3 has been installed.
Installing Windows optional component: VirtualMachinePlatform

Deployment Image Servicing and Management tool
Version: 10.0.26100.5074

Image Version: 10.0.26200.8246

Enabling feature(s)
[=====100.0%=====]
The operation completed successfully.
The requested operation is successful. Changes will not be effective until the system is rebooted.
The requested operation is successful. Changes will not be effective until the system is rebooted.
PS C:\Users\ASUS>

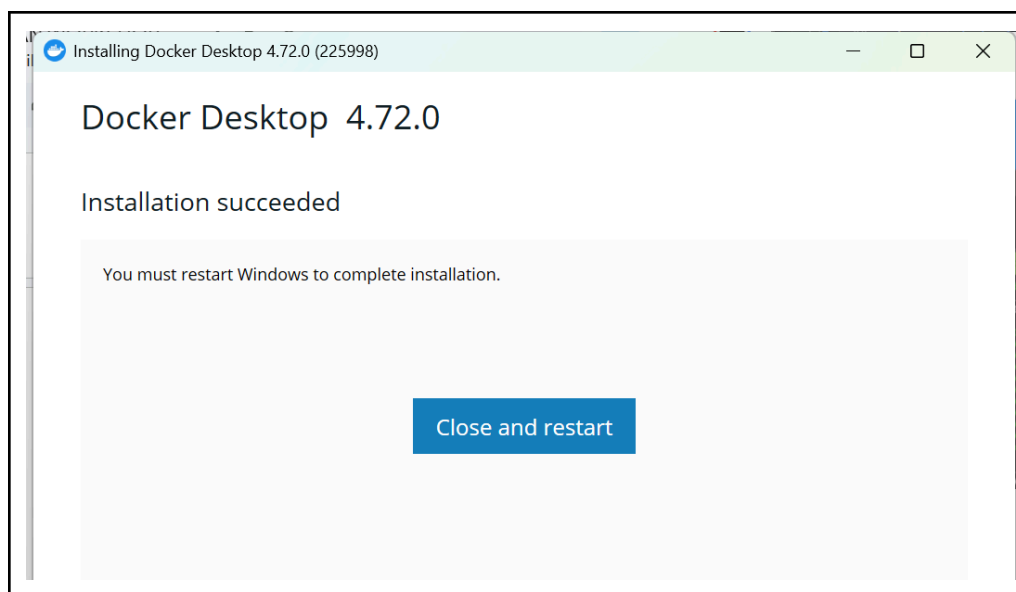
PS C:\WINDOWS\system32> wsl --set-default-version 2
For information on key differences with WSL 2 please visit https://aka.ms/ws12
The operation completed successfully.
PS C:\WINDOWS\system32> wsl --list --verbose
  NAME      STATE      VERSION
* Ubuntu    Running    2
PS C:\WINDOWS\system32>

```

Langkah ini dilakukan untuk mengaktifkan Windows Subsystem for Linux (WSL) versi 2 yang dibutuhkan agar Docker Desktop dapat berjalan dengan optimal. WSL2 memungkinkan Linux berjalan langsung di Windows dengan performa yang lebih baik. Setelah diaktifkan, komputer perlu di-restart agar konfigurasi diterapkan.

2. Download dan Instal Docker Desktop

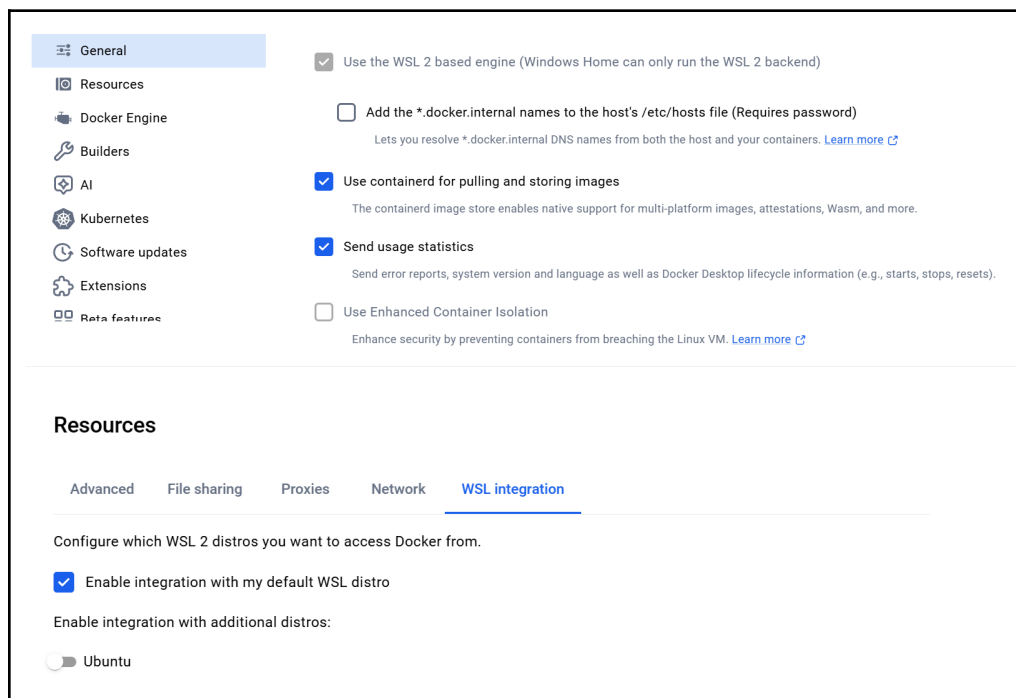
- Buka <https://www.docker.com/products/docker-desktop/>
- Download Docker Desktop for Windows
- Jalankan installer, centang "Use WSL 2 instead of Hyper-V"
- Klik Install → tunggu selesai → Restart jika diminta



Pada tahap ini dilakukan pengunduhan dan instalasi Docker Desktop pada sistem Windows. Docker Desktop berfungsi sebagai antarmuka GUI untuk mengelola container dan image Docker. Selama instalasi, opsi WSL2 dipilih agar integrasi dengan Linux subsystem dapat berjalan dengan baik.

3. Konfigurasi Docker Desktop

- Buka Docker Desktop dari Start Menu
- Masuk ke Settings → General → pastikan "Use the WSL 2 based engine" aktif
- Masuk ke Settings → Resources → WSL Integration → aktifkan distro yang diinginkan



Setelah instalasi selesai, dilakukan konfigurasi awal pada Docker Desktop seperti mengaktifkan WSL2 backend dan integrasi dengan distribusi Linux yang digunakan. Konfigurasi ini memastikan Docker dapat berjalan baik di Windows dan dapat diakses melalui terminal WSL maupun PowerShell.

4. Verifikasi dari PowerShell / WSL Terminal

Dari PowerShell

```
PS C:\WINDOWS\system32> docker version

Client:
 Version:           29.4.2
 API version:       1.54
 Go version:        go1.26.2
 Git commit:        055a478
 Built:            Fri May 1 01:27:10 2026
 OS/Arch:           windows/amd64
 Context:           desktop-linux

Server: Docker Desktop 4.72.0 (225998)
 Engine:
  Version:          29.4.2
  API version:      1.54 (minimum version 1.40)
  Go version:       go1.26.2
  Git commit:       d329809
```

```
PS C:\WINDOWS\system32> docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
4f55086f7dd0: Pull complete
d5e71e642bf5: Download complete
Digest: sha256:f9078146db2e05e794366b1bfe584a14ea6317f44027d10ef7dad65279026885
Status: Downloaded newer image for hello-world:latest
```

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:

1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
(amd64)
3. The Docker daemon created a new container from that image which runs the executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
\$ docker run -it ubuntu bash

Dari WSL Terminal

```
nyndyaa@nindyA:~$ docker version

Client:
 Version:           29.4.2
 API version:       1.54
 Go version:        go1.26.2
 Git commit:        055a478
 Built:            Fri May 1 01:23:32 2026
 OS/Arch:           linux/amd64
 Context:           default

Server: Docker Desktop 4.72.0 (225998)
 Engine:
  Version:          29.4.2
  API version:      1.54 (minimum version 1.40)
  Go version:       go1.26.2
  Git commit:       d329809
  Built:            Fri May 1 01:24:36 2026
  OS/Arch:          linux/amd64
  Experimental:     false
```

```
nyndyaa@nindyA:~$ docker run hello-world

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
   (amd64)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/
```

Tahap ini bertujuan untuk memastikan Docker Desktop sudah terinstal dan berjalan dengan benar di Windows. Perintah `docker version` dan `docker run hello-world` digunakan untuk menguji apakah Docker Engine dapat menjalankan container dengan sukses dari lingkungan Windows maupun WSL.

Langkah 3: Operasi Dasar Docker Image

1. Pull image dari Docker Hub

```

elfandi@elfandi-Latitude-7414:~$ docker pull nginx
Using default tag: latest
latest: Pulling from library/nginx
8e9b53b630de: Pull complete
382a36159731: Pull complete
6e9f32fdbd61: Pull complete
20d41cd6829d: Pull complete
38a7e44929f3: Pull complete
83742381311a: Pull complete
57fb71246055: Pull complete
94a543ead0f2: Download complete
46ca9676efb5: Download complete
Digest: sha256:1881968aff6f7cdcc4b888c00a11f4ce241ad7ec957e0cb4a9e19e93a3ff87ea
Status: Downloaded newer image for nginx:latest
docker.io/library/nginx:latest
elfandi@elfandi-Latitude-7414:~$ docker pull nginx:1.26
1.26: Pulling from library/nginx
4fd410795c0f: Pull complete
d44088bb6ae8: Pull complete
9ebfb40fb06b: Pull complete
5e98d206134b: Pull complete
8a628cdd7ccc: Pull complete
7a0654aeb922: Pull complete
6923759e66ab: Pull complete
f17adcc09044: Download complete
5c2c3686e536: Download complete
Digest: sha256:41b194461e4bae16f9b25d68b0976ed4735b89ca625c89aad88e1c1c3b7e8860
Status: Downloaded newer image for nginx:1.26
docker.io/library/nginx:1.26
elfandi@elfandi-Latitude-7414:~$ docker pull ubuntu:22.04
22.04: Pulling from library/ubuntu
f63eb04151bc: Pull complete
bf0d6867143e: Download complete
Digest: sha256:962f6cadeae0ea6284001009daa4cc9a8c37e75d1f5191cf0eb83fe565b63dd7
Status: Downloaded newer image for ubuntu:22.04
docker.io/library/ubuntu:22.04
elfandi@elfandi-Latitude-7414:~$ docker pull alpine:3.20
3.20: Pulling from library/alpine
25fd6b1951a: Pull complete
3a030ca3f633: Download complete
d4050d56ebf2: Download complete
Digest: sha256:d9e853e87e55526f6b2917df91a2115c36dd7c696a35be12163d44e6e2a4b6bc
Status: Downloaded newer image for alpine:3.20
docker.io/library/alpine:3.20

```

Langkah ini digunakan untuk mengunduh image dari Docker Hub ke lokal komputer. Image seperti nginx, ubuntu, dan alpine diunduh untuk digunakan sebagai base image dalam pembuatan container. Perintah docker pull memastikan image tersedia secara lokal sebelum dijalankan.

2. Manajemen image

```
elfandi@elfandi-Latitude-7414:~$ docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
alpine:3.20	d9e853e87e55	12.2MB	3.71MB	
hello-world:latest	f9078146db2e	25.9kB	9.49kB	U
nginx:1.26	41b194461e4b	282MB	75.2MB	
nginx:latest	1881968aff6f	240MB	65.8MB	
ubuntu:22.04	962f6cadeae0	119MB	31.7MB	

```
elfandi@elfandi-Latitude-7414:~$ docker images --format "table {{.Repository}}\t{{.Tag}}\t{{.Size}}"
```

REPOSITORY	TAG	SIZE
nginx	latest	240MB
alpine	3.20	12.2MB
ubuntu	22.04	119MB
hello-world	latest	25.9kB
nginx	1.26	282MB

```
elfandi@elfandi-Latitude-7414:~$ docker image inspect nginx
```

```
[
  {
    "Id": "sha256:1881968aff6f7cdcc4b888c00a11f4ce241ad7ec957e0cb4a9e19e93a3ff87ea",
    "RepoTags": [
      "nginx:latest"
    ],
    "RepoDigests": [
      "nginx@sha256:1881968aff6f7cdcc4b888c00a11f4ce241ad7ec957e0cb4a9e19e93a3ff87ea"
    ],
    "Comment": "buildkit.dockerfile.v0",
    "Created": "2026-05-08T19:22:39.549188019Z",
    "Config": {
      "ExposedPorts": {
        "80/tcp": {}
      },
      "Env": [
        "PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin",
        "NGINX_VERSION=1.29.8",
        "NJS_VERSION=0.9.6",
        "NJS_RELEASE=1~trixie",
        "ACME_VERSION=0.3.1",
        "PKG_RELEASE=1~trixie",
        "DYNPKG_RELEASE=1~trixie"
      ],
      "Entrypoint": [
        "/docker-entrypoint.sh"
      ],
      "Cmd": [
        "nginx",
        "-g",
        "daemon off;"
      ],
      "Labels": {
        "maintainer": "NGINX Docker Maintainers <docker-maint@nginx.com>"
      },
      "StopSignal": "SIGQUIT"
    },
    "Architecture": "amd64",
    "Os": "linux",
    "Size": 629640900
  }
]
```

```

elfandigelfandi-Latitude-7414:~$ docker image history nginx
IMAGE          CREATED          CREATED BY                                      SIZE      COMMENT
1881968aff6f   33 hours ago    CMD ["nginx" "-g" "daemon off;"]              0B        buildkit.dockerfile.v0
<missing>      33 hours ago    STOPSIGNAL SIGQUIT                             0B        buildkit.dockerfile.v0
<missing>      33 hours ago    EXPOSE map[80/tcp:{}]                          0B        buildkit.dockerfile.v0
<missing>      33 hours ago    ENTRYPOINT ["/docker-entrypoint.sh"]           0B        buildkit.dockerfile.v0
<missing>      33 hours ago    COPY 30-tune-worker-processes.sh /docker-ent... 16.4kB    buildkit.dockerfile.v0
<missing>      33 hours ago    COPY 20-envsubst-on-templates.sh /docker-ent... 12.3kB    buildkit.dockerfile.v0
<missing>      33 hours ago    COPY 15-local-resolvers.envsh /docker-entryp... 12.3kB    buildkit.dockerfile.v0
<missing>      33 hours ago    COPY 10-listen-on-ipv6-by-default.sh /docker... 12.3kB    buildkit.dockerfile.v0
<missing>      33 hours ago    COPY docker-entrypoint.sh / # buildkit         8.19kB    buildkit.dockerfile.v0
<missing>      33 hours ago    RUN /bin/sh -c set -x && groupadd --syst...     86.7MB    buildkit.dockerfile.v0
<missing>      33 hours ago    ENV DYNPKG_RELEASE=1-trixie                    0B        buildkit.dockerfile.v0
<missing>      33 hours ago    ENV PKG_RELEASE=1-trixie                      0B        buildkit.dockerfile.v0
<missing>      33 hours ago    ENV ACME_VERSION=0.3.1                        0B        buildkit.dockerfile.v0
<missing>      33 hours ago    ENV NJS_RELEASE=1-trixie                      0B        buildkit.dockerfile.v0
<missing>      33 hours ago    ENV NJS_VERSION=0.9.6                         0B        buildkit.dockerfile.v0
<missing>      33 hours ago    ENV NGINX_VERSION=1.29.8                      0B        buildkit.dockerfile.v0
<missing>      33 hours ago    LABEL maintainer=NGINX Docker Maintainers <d... 0B        buildkit.dockerfile.v0
<missing>      5 days ago     # debian.sh --arch 'amd64' out/ 'trixie' '@1... 87.4MB    debuerreotype 0.17
elfandigelfandi-Latitude-7414:~$ docker rmi alpine:3.20
Untagged: alpine:3.20
Deleted: sha256:d9e853e87e55526f6b2917df91a2115c36dd7c696a35be12163d44e6e2a4b6bc
elfandigelfandi-Latitude-7414:~$ docker image prune -a
WARNING! This will remove all images without at least one container associated to them.
Are you sure you want to continue? [y/N] y
Deleted Images:
untagged: nginx:1.26
deleted: sha256:41b194461e4bae16f9b25d68b0976ed4735b89ca625c89aad88e1c1c3b7e8860
deleted: sha256:c8042489f41ddaf05b7dfbe1e32d5227f94f1b002aca8e3b7b9c9d0cee6b0fed
deleted: sha256:c28381c5787d56e9bf825ecd4b8b1f1a64caec040b959a54674ac5a43954cfd3
untagged: nginx:latest
deleted: sha256:1881968aff6f7cdcc4b888c00a11f4ce241ad7ec957e0cb4a9e19e93a3ff07ea
deleted: sha256:ab15d428b6a7121511a38221591a41f835933ac16f996fafd102d128a0fa20f7
deleted: sha256:bf34a2efe16261f3df780ef1aaa3d34d2e2f85d7257b3ffc1b20f78de52a41f6
untagged: ubuntu:22.04
deleted: sha256:962f6cadeae0ea6284001009daa4cc9a8c37e75d1f5191cf0eb83fe565b63dd7
deleted: sha256:14be402d3f1eeeb5e7da73d3260322c68e7b51c88388f53e88eb21d6450bd520
deleted: sha256:268e076660f3cd225015df43cfe7198e5f787f24db00338d433299687237e538

Total reclaimed space: 172.7MB
elfandigelfandi-Latitude-7414:~$

```

Pada tahap ini dilakukan pengelolaan image Docker seperti melihat daftar image, mengecek detail, melihat history layer, dan menghapus image yang tidak digunakan. Hal ini bertujuan untuk memahami struktur image serta mengelola storage Docker secara efisien.

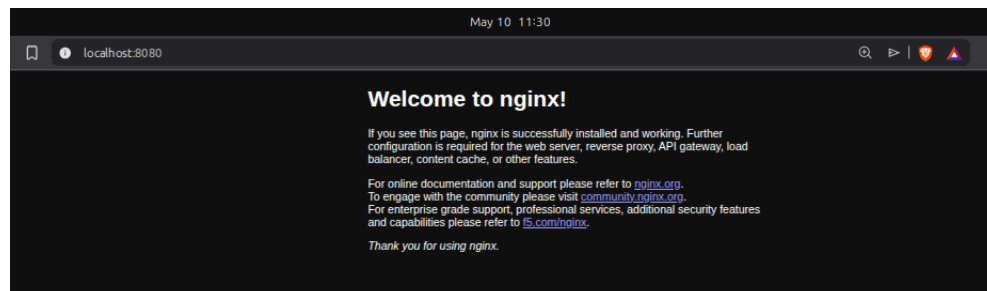
Langkah 4: Menjalankan dan Mengelola Container

1. Menjalankan container dasar

```

elfandigelfandi-Latitude-7414:~$ docker run nginx
Unable to find image 'nginx:latest' locally
latest: Pulling from library/nginx
8e9b53b630de: Pull complete
6e9f32fdbd61: Pull complete
20d41cd6829d: Pull complete
38a7e44929f3: Pull complete
382a36159731: Pull complete
57fb71246055: Pull complete
83742381311a: Pull complete
94a543ead0f2: Download complete
46ca9676efb5: Download complete
Digest: sha256:1881968aff6f7cdcc4b888c00a11f4ce241ad7ec957e0cb4a9e19e93a3ff87ea
Status: Downloaded newer image for nginx:latest
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/05/10 04:27:39 [notice] 1#1: using the "epoll" event method
2026/05/10 04:27:39 [notice] 1#1: nginx/1.29.8
2026/05/10 04:27:39 [notice] 1#1: built by gcc 14.2.0 (Debian 14.2.0-19)
2026/05/10 04:27:39 [notice] 1#1: OS: Linux 6.17.0-22-generic
2026/05/10 04:27:39 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1024:524288
2026/05/10 04:27:39 [notice] 1#1: start worker processes
2026/05/10 04:27:39 [notice] 1#1: start worker process 29
2026/05/10 04:27:39 [notice] 1#1: start worker process 30
2026/05/10 04:27:39 [notice] 1#1: start worker process 31
2026/05/10 04:27:39 [notice] 1#1: start worker process 32
^C2026/05/10 04:29:24 [notice] 1#1: signal 2 (SIGINT) received, exiting
2026/05/10 04:29:24 [notice] 29#29: exiting
2026/05/10 04:29:24 [notice] 29#29: exit
2026/05/10 04:29:24 [notice] 30#30: exiting
2026/05/10 04:29:24 [notice] 31#31: exiting
2026/05/10 04:29:24 [notice] 30#30: exit
2026/05/10 04:29:24 [notice] 31#31: exit
2026/05/10 04:29:24 [notice] 32#32: exiting
2026/05/10 04:29:24 [notice] 32#32: exit
2026/05/10 04:29:24 [notice] 1#1: signal 17 (SIGCHLD) received from 32
2026/05/10 04:29:24 [notice] 1#1: worker process 29 exited with code 0
2026/05/10 04:29:24 [notice] 1#1: worker process 30 exited with code 0
2026/05/10 04:29:24 [notice] 1#1: worker process 31 exited with code 0
2026/05/10 04:29:24 [notice] 1#1: worker process 32 exited with code 0
2026/05/10 04:29:24 [notice] 1#1: exit
elfandigelfandi-Latitude-7414:~$ docker run -d --name web-server nginx
febab81a5ceda83a0246ecf84cbf6417cc39ad19556bb2f1a57023e61aee9051
elfandigelfandi-Latitude-7414:~$ docker run -d --name web-public -p 8080:80 nginx
f8d4b60c4c9b989b0dce378bff7279c5662d1f42bfb8c854fea6797ccf9b3286
elfandigelfandi-Latitude-7414:~$

```



Langkah ini digunakan untuk menjalankan container dari image yang sudah di-pull. Container dapat dijalankan dalam mode foreground maupun

background (detached). Selain itu dilakukan port mapping agar container dapat diakses melalui browser pada port tertentu.

2. Container interaktif

```
elfandi@elfandi-Latitude-7414:~$ docker run -it --name ubuntu-test ubuntu:22.04 /bin/bash
Unable to find image 'ubuntu:22.04' locally
22.04: Pulling from library/ubuntu
f63eb04151bc: Pull complete
bf0d6867143e: Download complete
Digest: sha256:962f6cadeae0ea6284001009daa4cc9a8c37e75df5191cf0eb83fe565b63dd7
Status: Downloaded newer image for ubuntu:22.04
root@a0538840c17b:/# cat /etc/os-release
apt update && apt install -y curl
curl http://web-server # akses container lain (jika di network sama)
exit
PRETTY_NAME="Ubuntu 22.04.5 LTS"
NAME="Ubuntu"
VERSION_ID="22.04"
VERSION="22.04.5 LTS (Jammy Jellyfish)"
VERSION_CODENAME=jammy
ID=ubuntu
ID_LIKE=debian
HOME_URL="https://www.ubuntu.com/"
SUPPORT_URL="https://help.ubuntu.com/"
BUG_REPORT_URL="https://bugs.launchpad.net/ubuntu/"
PRIVACY_POLICY_URL="https://www.ubuntu.com/legal/terms-and-policies/privacy-policy"
UBUNTU_CODENAME=jammy
Get:1 http://archive.ubuntu.com/ubuntu jammy InRelease [270 kB]
Get:2 http://security.ubuntu.com/ubuntu jammy-security InRelease [129 kB]
Get:3 http://security.ubuntu.com/ubuntu jammy-security/restricted amd64 Packages [7004 kB]
Get:4 http://archive.ubuntu.com/ubuntu jammy-updates InRelease [128 kB]
Get:5 http://archive.ubuntu.com/ubuntu jammy-backports InRelease [127 kB]
Get:6 http://archive.ubuntu.com/ubuntu jammy/universe amd64 Packages [17.5 MB]
Get:7 http://security.ubuntu.com/ubuntu jammy-security/multiverse amd64 Packages [61.6 kB]
Get:8 http://security.ubuntu.com/ubuntu jammy-security/main amd64 Packages [3915 kB]
Get:9 http://security.ubuntu.com/ubuntu jammy-security/universe amd64 Packages [1294 kB]
Get:10 http://archive.ubuntu.com/ubuntu jammy/restricted amd64 Packages [164 kB]
Get:11 http://archive.ubuntu.com/ubuntu jammy/multiverse amd64 Packages [266 kB]
```

Pada tahap ini container dijalankan dalam mode interaktif menggunakan terminal bash. Hal ini memungkinkan pengguna masuk langsung ke dalam sistem container untuk menjalankan perintah seperti di Linux biasa. Container akan berhenti ketika pengguna keluar dari shell.

3. Monitoring container


```
alfandigelfandi-Latitude-7414: $ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS                               NAMES
f8d4b60c4c9b   nginx    "/docker-entrypoint..." 6 minutes ago  Up 6 minutes  0.0.0.0:8080->80/tcp, [::]:8080->80/tcp  web-public
febab81a5ced   nginx    "/docker-entrypoint..." 6 minutes ago  Up 6 minutes  80/tcp                               web-server

alfandigelfandi-Latitude-7414: $ docker ps -a
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS                               NAMES
005388b0c17b   ubuntu:22.04 "/bin/bash"             4 minutes ago  Exited (6) 2 minutes ago              ubuntu-test
f8d4b60c4c9b   nginx    "/docker-entrypoint..." 6 minutes ago  Up 6 minutes  0.0.0.0:8080->80/tcp, [::]:8080->80/tcp  web-public
febab81a5ced   nginx    "/docker-entrypoint..." 7 minutes ago  Up 7 minutes  80/tcp                               web-server
97ba943a8ee0   nginx    "/docker-entrypoint..." 8 minutes ago  Exited (0) 7 minutes ago              bold_ardinghelli
95c9fd35f906   hello-world "/hello"                31 minutes ago  Exited (0) 31 minutes ago              great_williams

alfandigelfandi-Latitude-7414: $ docker logs web-server
docker logs -f web-server # follow (real-time)
docker logs --tail 20 web-server # 20 baris terakhir
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/05/10 04:29:33 [notice] 1#1: using the "epoll" event method
2026/05/10 04:29:33 [notice] 1#1: nginx/1.29.8
2026/05/10 04:29:33 [notice] 1#1: built by gcc 14.2.0 (Debian 14.2.0-19)
2026/05/10 04:29:33 [notice] 1#1: OS: Linux 6.17.0-22-generic
2026/05/10 04:29:33 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1024:524288
2026/05/10 04:29:33 [notice] 1#1: start worker processes
2026/05/10 04:29:33 [notice] 1#1: start worker process 29
2026/05/10 04:29:33 [notice] 1#1: start worker process 30
2026/05/10 04:29:33 [notice] 1#1: start worker process 31
2026/05/10 04:29:33 [notice] 1#1: start worker process 32
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/05/10 04:29:33 [notice] 1#1: using the "epoll" event method
2026/05/10 04:29:33 [notice] 1#1: nginx/1.29.8
2026/05/10 04:29:33 [notice] 1#1: built by gcc 14.2.0 (Debian 14.2.0-19)
2026/05/10 04:29:33 [notice] 1#1: OS: Linux 6.17.0-22-generic
2026/05/10 04:29:33 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1024:524288
2026/05/10 04:29:33 [notice] 1#1: start worker processes
2026/05/10 04:29:33 [notice] 1#1: start worker process 29
2026/05/10 04:29:33 [notice] 1#1: start worker process 30
2026/05/10 04:29:33 [notice] 1#1: start worker process 31
2026/05/10 04:29:33 [notice] 1#1: start worker process 32
|
```

CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O	PIDS
f8d4b60c4c9b	web-public	0.00%	4.809MiB / 15.49GiB	0.03%	14.4kB / 2.94kB	0B / 12.3kB	5
febab81a5ced	web-server	0.00%	4.797MiB / 15.49GiB	0.03%	13.7kB / 126B	0B / 12.3kB	5

```

^Celfandi@elfandi-Latitude-7414:~$ docker inspect web-server
[
  {
    "Id": "febab81a5ceda83a0246ecf84cbf6417cc39ad19556bb2f1a57023e61aee9051",
    "Created": "2026-05-10T04:29:32.928985585Z",
    "Path": "/docker-entrypoint.sh",
    "Args": [
      "nginx",
      "-g",
      "daemon off;"
    ],
    "State": {
      "Status": "running",
      "Running": true,
      "Paused": false,
      "Restarting": false,
      "OOMKilled": false,
      "Dead": false,
      "Pid": 32641,
      "ExitCode": 0,
      "Error": "",
      "StartedAt": "2026-05-10T04:29:32.992540076Z",
      "FinishedAt": "0001-01-01T00:00:00Z"
    },
    "Image": "sha256:1881968aff6f7cdcc4b888c00a11f4ce241ad7ec957e0cb4a9e19e93",
    "ResolvConfPath": "/var/lib/docker/containers/febab81a5ceda83a0246ecf84cbf6417cc39ad19556bb2f1a57023e61aee9051/resolv.conf",
    "HostnamePath": "/var/lib/docker/containers/febab81a5ceda83a0246ecf84cbf6417cc39ad19556bb2f1a57023e61aee9051/hostname",
    "HostsPath": "/var/lib/docker/containers/febab81a5ceda83a0246ecf84cbf6417cc39ad19556bb2f1a57023e61aee9051/hosts",
    "LogPath": "/var/lib/docker/containers/febab81a5ceda83a0246ecf84cbf6417cc39ad19556bb2f1a57023e61aee9051/json-file.log",
    "Name": "/web-server",
    "RestartCount": 0,
    "Driver": "overlayfs",
    "Platform": "linux",
    "MountLabel": "",
    "ProcessLabel": "",
    "AppArmorProfile": "docker-default",
    "ExecIDs": null,
    "HostConfig": {
      "Binds": null,
      "ContainerIDFile": "",
      "LogConfig": {
        "Type": "json-file",
        "Config": {}
      },
      "NetworkMode": "bridge",

```

Langkah ini digunakan untuk memantau kondisi container yang sedang berjalan. Perintah seperti docker ps, logs, stats, dan inspect digunakan untuk melihat status, log aktivitas, penggunaan resource, serta detail konfigurasi container.

4. Interaksi dengan container berjalan

```

elfandi@elfandi-Latitude-7414:~$ docker exec web-server cat /etc/nginx/nginx.conf

user  nginx;
worker_processes  auto;

error_log  /var/log/nginx/error.log notice;
pid        /run/nginx.pid;

events {
    worker_connections  1024;
}

http {
    include        /etc/nginx/mime.types;
    default_type  application/octet-stream;

    log_format main '$remote_addr - $remote_user [$time_local] "$request" '
                    '$status $body_bytes_sent "$http_referer" '
                    '"$http_user_agent" "$http_x_forwarded_for"';

    access_log  /var/log/nginx/access.log  main;

    sendfile        on;
    #tcp_nopush     on;

    keepalive_timeout  65;

    #gzip           on;

    include /etc/nginx/conf.d/*.conf;
}
elfandi@elfandi-Latitude-7414:~$ docker exec -it web-server /bin/bash
root@febab81a5ced:/# exit
exit
elfandi@elfandi-Latitude-7414:~$ docker cp index.html web-server:/usr/share/nginx/html/index.html
Successfully copied 22B (transferred 2.05kB) to web-server:/usr/share/nginx/html/index.html
elfandi@elfandi-Latitude-7414:~$ docker cp web-server:/etc/nginx/nginx.conf ./nginx.conf
Successfully copied 644B (transferred 2.56kB) to /home/elfandi/nginx.conf
elfandi@elfandi-Latitude-7414:~$

```

Tahap ini memungkinkan eksekusi perintah langsung di dalam container tanpa masuk ke shell, serta melakukan transfer file antara host dan container menggunakan `docker cp`. Hal ini penting untuk mengelola file konfigurasi atau data di dalam container.

5. Lifecycle management

```

elfandi@elfandi-Latitude-7414:~$ docker stop web-server
web-server
elfandi@elfandi-Latitude-7414:~$ docker start web-server
web-server
elfandi@elfandi-Latitude-7414:~$ docker restart web-server
web-server
elfandi@elfandi-Latitude-7414:~$ docker kill web-server
web-server
elfandi@elfandi-Latitude-7414:~$ docker stop web-server && docker rm web-server
web-server
web-server
elfandi@elfandi-Latitude-7414:~$ docker rm -f web-server
Error response from daemon: No such container: web-server
elfandi@elfandi-Latitude-7414:~$ docker container prune
WARNING! This will remove all stopped containers.
Are you sure you want to continue? [y/N] y
Deleted Containers:
a0538840c17bd39b1d2a32cfa2188a8fc7cc4ccf07be4a9af14025e2b34a2d20
97ba943a8ee0b7d52e27ece50b455ee0564dea48dcc051e9f94dbb6a33c88028
95c9fd35f9061310d71d46255c10d5571aca431a6458e4cbaa6a8ee21f4f3309

Total reclaimed space: 84.75MB

```

Langkah ini menjelaskan pengelolaan siklus hidup container seperti stop, start, restart, kill, dan remove. Selain itu juga dilakukan pembersihan container yang sudah tidak digunakan untuk menghemat resource sistem.

Langkah 5: Membuat Custom Image dengan Dockerfile

1. Buat project directory

```

elfandi@elfandi-Latitude-7414:~$ mkdir -p ~/docker-lab/custom-web && cd ~/docker-lab/custom-web

```

Langkah ini membuat folder khusus sebagai workspace proyek Docker. Direktori ini digunakan untuk menyimpan file HTML dan Dockerfile yang akan digunakan dalam proses build image custom.

2. Buat halaman web

```

elfandi@elfandi-Latitude-7414:~/docker-lab/custom-web$ cat > index.html << 'EOF'
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <title>Docker Lab - PENS</title>
  <style>
    body { font-family: Arial, sans-serif; text-align: center; padding: 50px;
      background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
      color: white; }
    .container { background: rgba(255,255,255,0.1); border-radius: 15px;
      padding: 40px; max-width: 600px; margin: 0 auto; }
    h1 { font-size: 2.5em; }
    .info { background: rgba(0,0,0,0.2); padding: 15px; border-radius: 8px;
      margin-top: 20px; text-align: left; }
  </style>
</head>
<body>
  <div class="container">
    <h1>🐳 Docker Lab PENS</h1>
    <p>Container berhasil berjalan!</p>
    <div class="info">
      <p><strong>Hostname:</strong> <span id="host"></span></p>
      <p><strong>Server:</strong> Nginx on Docker</p>
      <p><strong>Praktikum:</strong> Modul 1 – Instalasi Docker</p>
    </div>
  </div>
  <script>
    document.getElementById('host').textContent = location.hostname;
  </script>
</body>
</html>
EOF

```

Pada tahap ini dibuat file HTML sederhana yang akan digunakan sebagai konten website di dalam container. File ini akan menggantikan halaman default Nginx untuk menampilkan web custom.

3. Buat Dockerfile

```

elfandi@elfandi-Latitude-7414:~/docker-lab/custom-web$ cat > Dockerfile << 'EOF'
# Gunakan base image nginx versi stabil
FROM nginx:1.26-alpine

# Metadata
LABEL maintainer="admin@pens.ac.id"
LABEL description="Custom Nginx untuk praktikum Docker PENS"
LABEL version="1.0"

# Hapus halaman default dan ganti dengan halaman custom
RUN rm -rf /usr/share/nginx/html/*
COPY index.html /usr/share/nginx/html/index.html

# Expose port 80
EXPOSE 80

# Command default (inherited dari base image, tapi kita tulis eksplisit)
CMD ["nginx", "-g", "daemon off;"]
EOF

```

Dockerfile dibuat sebagai blueprint untuk membangun custom image. Di dalamnya terdapat instruksi seperti base image, metadata, copy file, dan konfigurasi server. Dockerfile ini menentukan bagaimana image akan dibuat.

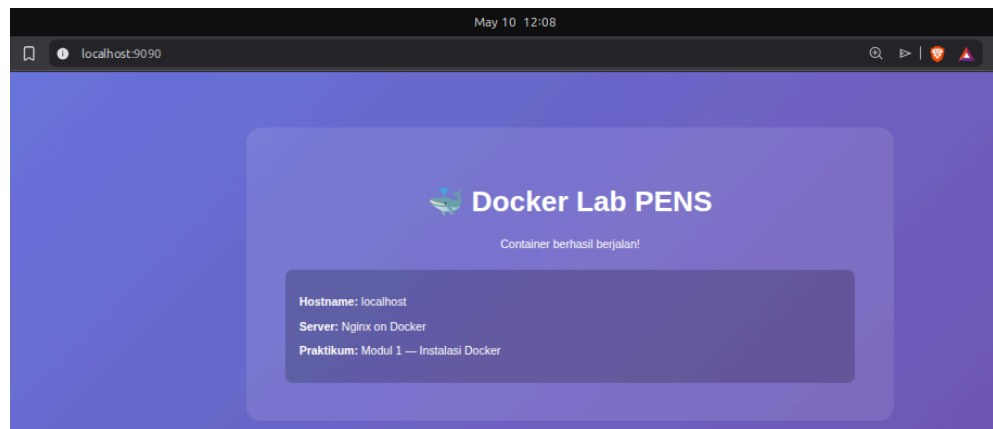
4. Build dan jalankan

```

elfandi@elfandi-Latitude-7414:~/docker-lab/custom-web$ docker build -t pens-web:1.0 .
[+] Building 25.9s (8/8) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 509B
=> [internal] load metadata for docker.io/library/nginx:1.26-alpine
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [internal] load build context
=> => transferring context: 1.18kB
=> [1/3] FROM docker.io/library/nginx:1.26-alpine@sha256:1eadbb07820339e8bbfed18c771691970baee292ec4ab2558f14
=> => resolve docker.io/library/nginx:1.26-alpine@sha256:1eadbb07820339e8bbfed18c771691970baee292ec4ab2558f14
=> => sha256:a7c661bcc4ef10a5e7ee3b4f80a8d77754a4e6b1a598271b70bd220d804316eb 1.40kB / 1.40kB
=> => sha256:5ab072e0bc4f67351f9c935c8834e3e1bcacbb0e013a124ef309607d6210f218d 15.19MB / 15.19MB
=> => sha256:c35c2aa19c802e9ca054353d84a4b58904f72807055c441be84f27401714f25d 1.21kB / 1.21kB
=> => sha256:d987ab110a5ba4aebf9df7115759f01e8a5d8be0a70045f4dc7a5b959be1b04c 404B / 404B
=> => sha256:b8651333baae33f196ce13e6d5cfc16e276b640bbca95329625fbb6a3f13855 956B / 956B
=> => sha256:914d0eaf60479e4b17101b02ed1e82400c02231d11c32276fb24867f5f39e97b 1.76MB / 1.76MB
=> => sha256:5e10ef93c435d9cd1c48d1ae900cf7e92ee61ae11be6b3bdefc899f4be8efad9 628B / 628B
=> => sha256:0a9a5dfd008f05ebc27e4790db0709a29e527690c21bc0cd01481eae6bb49dc 3.63MB / 3.63MB
=> => extracting sha256:0a9a5dfd008f05ebc27e4790db0709a29e527690c21bc0cd01481eae6bb49dc
=> => extracting sha256:914d0eaf60479e4b17101b02ed1e82400c02231d11c32276fb24867f5f39e97b
=> => extracting sha256:5e10ef93c435d9cd1c48d1ae900cf7e92ee61ae11be6b3bdefc899f4be8efad9
=> => extracting sha256:b8651333baae33f196ce13e6d5cfc16e276b640bbca95329625fbb6a3f13855
=> => extracting sha256:d987ab110a5ba4aebf9df7115759f01e8a5d8be0a70045f4dc7a5b959be1b04c
=> => extracting sha256:c35c2aa19c802e9ca054353d84a4b58904f72807055c441be84f27401714f25d
=> => extracting sha256:a7c661bcc4ef10a5e7ee3b4f80a8d77754a4e6b1a598271b70bd220d804316eb
=> => extracting sha256:5ab072e0bc4f67351f9c935c8834e3e1bcacbb0e013a124ef309607d6210f218d
=> [2/3] RUN rm -rf /usr/share/nginx/html/*
=> [3/3] COPY index.html /usr/share/nginx/html/index.html
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:7137beeacff26e6350fadfa21c7026104bece498fe5bd578645e31192fe276d4
=> => exporting config sha256:59ace2cee4ce7894c93939c87cbae44d65a3f12633f11b6a1152bde7b51869f3
=> => exporting attestation manifest sha256:5016d63c6504a29d242e3a0a69df4842b5af2312f04815aaff2f8682dde20a57
=> => exporting manifest list sha256:e0c5a30fd1dd4ce6f8db5ca90fd4bbbe91ec73621a5ca4b31ec12a44622e0135
=> => naming to docker.io/library/pens-web:1.0
=> => unpacking to docker.io/library/pens-web:1.0
elfandi@elfandi-Latitude-7414:~/docker-lab/custom-web$ docker images | grep pens-web
WARNING: This output is designed for human readability. For machine-readable output, please use --format.

```

```
elfandi@elfandi-Latitude-7414:~/docker-lab/custom-web$ docker run -d --name pens-app -p 9090:80 pens-web:1.0
dc02a695ed2d3738ee569bc1719fb9b3ac7da423e610a8017b949a28387069cb
elfandi@elfandi-Latitude-7414:~/docker-lab/custom-web$ curl http://localhost:9090
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <title>Docker Lab - PENS</title>
  <style>
    body { font-family: Arial, sans-serif; text-align: center; padding: 50px;
      background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
      color: white; }
    .container { background: rgba(255,255,255,0.1); border-radius: 15px;
      padding: 40px; max-width: 600px; margin: 0 auto; }
    h1 { font-size: 2.5em; }
    .info { background: rgba(0,0,0,0.2); padding: 15px; border-radius: 8px;
      margin-top: 20px; text-align: left; }
  </style>
</head>
<body>
  <div class="container">
    <h1>🐳 Docker Lab PENS</h1>
    <p>Container berhasil berjalan!</p>
    <div class="info">
      <p><strong>Hostname:</strong> <span id="host"></span></p>
      <p><strong>Server:</strong> Nginx on Docker</p>
      <p><strong>Praktikum:</strong> Modul 1 — Instalasi Docker</p>
    </div>
  </div>
  <script>
    document.getElementById('host').textContent = location.hostname;
  </script>
</body>
</html>
```



Pada tahap ini dilakukan proses build image dari Dockerfile menggunakan docker build. Setelah image berhasil dibuat, container dijalankan dan dipetakan ke port host agar dapat diakses melalui browser.

5. Lihat layer image

```

elfandi@elfandi-Latitude-7414:~/docker-lab/custom-web$ docker image history nginx:1.26-alpine
IMAGE          CREATED          CREATED BY                                      SIZE      COMMENT
1e4dbb078203   15 months ago   RUN /bin/sh -c set -x      && apkArch="$(cat ... 37.7MB     buildkit.dockerfile.v0
<missing>      15 months ago   ENV NJS_RELEASE=1          0B        buildkit.dockerfile.v0
<missing>      15 months ago   ENV NJS_VERSION=0.8.9      0B        buildkit.dockerfile.v0
<missing>      15 months ago   CMD ["nginx" "-g" "daemon off;"] 0B        buildkit.dockerfile.v0
<missing>      15 months ago   STOPSIGNAL SIGQUIT         0B        buildkit.dockerfile.v0
<missing>      15 months ago   EXPOSE map[80/tcp:{}]      0B        buildkit.dockerfile.v0
<missing>      15 months ago   ENTRYPOINT ["/docker-entrypoint.sh"] 0B        buildkit.dockerfile.v0
<missing>      15 months ago   COPY 30-tune-worker-processes.sh /docker-ent... 16.4kB     buildkit.dockerfile.v0
<missing>      15 months ago   COPY 20-envsubst-on-templates.sh /docker-ent... 12.3kB     buildkit.dockerfile.v0
<missing>      15 months ago   COPY 15-local-resolvers.envsh /docker-entryp... 12.3kB     buildkit.dockerfile.v0
<missing>      15 months ago   COPY 10-listen-on-ipv6-by-default.sh /docker... 12.3kB     buildkit.dockerfile.v0
<missing>      15 months ago   COPY docker-entrypoint.sh / # buildkit      8.19kB     buildkit.dockerfile.v0
<missing>      15 months ago   RUN /bin/sh -c set -x      && addgroup -g 101... 5.28MB     buildkit.dockerfile.v0
<missing>      15 months ago   ENV DYNPKG_RELEASE=2       0B        buildkit.dockerfile.v0
<missing>      15 months ago   ENV PKG_RELEASE=1         0B        buildkit.dockerfile.v0
<missing>      15 months ago   ENV NGINX_VERSION=1.26.3   0B        buildkit.dockerfile.v0
<missing>      15 months ago   LABEL maintainer=NGINX Docker Maintainers <d... 0B        buildkit.dockerfile.v0
<missing>      15 months ago   CMD ["/bin/sh"]           0B        buildkit.dockerfile.v0
<missing>      15 months ago   ADD alpine-minirootfs-3.20.6-x86_64.tar.gz /... 8.47MB     buildkit.dockerfile.v0
elfandi@elfandi-Latitude-7414:~/docker-lab/custom-web$ docker image history pens-web:1.0
IMAGE          CREATED          CREATED BY                                      SIZE      COMMENT
e0c5a30fd1d4   2 minutes ago   CMD ["nginx" "-g" "daemon off;"] 0B        buildkit.dockerfile.v0
<missing>      2 minutes ago   EXPOSE [80/tcp]           0B        buildkit.dockerfile.v0
<missing>      2 minutes ago   COPY index.html /usr/share/nginx/html/index... 24.6kB     buildkit.dockerfile.v0
<missing>      2 minutes ago   RUN /bin/sh -c rm -rf /usr/share/nginx/html/... 20.5kB     buildkit.dockerfile.v0
<missing>      2 minutes ago   LABEL version=1.0         0B        buildkit.dockerfile.v0
<missing>      2 minutes ago   LABEL description=Custom Nginx untuk praktik... 0B        buildkit.dockerfile.v0
<missing>      2 minutes ago   LABEL maintainer=admin@pens.ac.id          0B        buildkit.dockerfile.v0
<missing>      15 months ago   RUN /bin/sh -c set -x      && apkArch="$(cat ... 37.7MB     buildkit.dockerfile.v0
<missing>      15 months ago   ENV NJS_RELEASE=1          0B        buildkit.dockerfile.v0
<missing>      15 months ago   ENV NJS_VERSION=0.8.9      0B        buildkit.dockerfile.v0
<missing>      15 months ago   CMD ["nginx" "-g" "daemon off;"] 0B        buildkit.dockerfile.v0
<missing>      15 months ago   STOPSIGNAL SIGQUIT         0B        buildkit.dockerfile.v0
<missing>      15 months ago   EXPOSE map[80/tcp:{}]      0B        buildkit.dockerfile.v0
<missing>      15 months ago   ENTRYPOINT ["/docker-entrypoint.sh"] 0B        buildkit.dockerfile.v0
<missing>      15 months ago   COPY 30-tune-worker-processes.sh /docker-ent... 16.4kB     buildkit.dockerfile.v0
<missing>      15 months ago   COPY 20-envsubst-on-templates.sh /docker-ent... 12.3kB     buildkit.dockerfile.v0
<missing>      15 months ago   COPY 15-local-resolvers.envsh /docker-entryp... 12.3kB     buildkit.dockerfile.v0
<missing>      15 months ago   COPY 10-listen-on-ipv6-by-default.sh /docker... 12.3kB     buildkit.dockerfile.v0
<missing>      15 months ago   COPY docker-entrypoint.sh / # buildkit      8.19kB     buildkit.dockerfile.v0
<missing>      15 months ago   RUN /bin/sh -c set -x      && addgroup -g 101... 5.28MB     buildkit.dockerfile.v0
<missing>      15 months ago   ENV DYNPKG_RELEASE=2       0B        buildkit.dockerfile.v0
<missing>      15 months ago   ENV PKG_RELEASE=1         0B        buildkit.dockerfile.v0
<missing>      15 months ago   ENV NGINX_VERSION=1.26.3   0B        buildkit.dockerfile.v0
<missing>      15 months ago   LABEL maintainer=NGINX Docker Maintainers <d... 0B        buildkit.dockerfile.v0
<missing>      15 months ago   CMD ["/bin/sh"]           0B        buildkit.dockerfile.v0
<missing>      15 months ago   ADD alpine-minirootfs-3.20.6-x86_64.tar.gz /... 8.47MB     buildkit.dockerfile.v0

```

Langkah ini digunakan untuk melihat struktur layer dari image Docker. Dengan perintah history, kita dapat memahami setiap proses yang terjadi saat image dibangun, termasuk perubahan dari base image hingga custom image.

Langkah 6: Docker System Cleanup

1. Docker System Cleanup


```

elfandigelfandi-Latitude-7414:~/docker-lab/custom-web$ docker system df

```

TYPE	TOTAL	ACTIVE	SIZE	RECLAIMABLE
Images	5	2	432.9MB	140.9MB (32%)
Containers	2	2	163.8kB	0B (0%)
Local Volumes	0	0	0B	0B
Build Cache	13	0	72.22MB	20.48kB

```

elfandigelfandi-Latitude-7414:~/docker-lab/custom-web$ docker system df -v

```

Images space usage:

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE	SHARED SIZE	UNIQUE SIZE	CONTAINERS
pens-web	1.0	e0c5a30fd1d4	2 minutes ago	72.2MB	51.55MB	20.64MB	1
nginx	latest	1881968aff6f	34 hours ago	240MB	0B	240MB	1
ubuntu	22.04	962f6cadeae0	4 weeks ago	119MB	0B	119.4MB	0
hello-world	latest	f9078146db2e	6 weeks ago	25.9kB	0B	25.87kB	0
nginx	1.26-alpine	1eadbb078203	15 months ago	73MB	51.55MB	21.48MB	0

Containers space usage:

CONTAINER ID	IMAGE	COMMAND	LOCAL VOLUMES	SIZE	CREATED	STATUS	NAMES
dc02a695ed2d	pens-web:1.0	"/docker-entrypoint..."	0	81.9kB	2 minutes ago	Up 2 minutes	pens-app
f8d4b60c4c9b	nginx	"/docker-entrypoint..."	0	81.9kB	41 minutes ago	Up 41 minutes	web-public

Local Volumes space usage:

VOLUME NAME	LINKS	SIZE
Build cache usage: 72.22MB		

CACHE ID	CACHE TYPE	SIZE	CREATED	LAST USED	USAGE	SHARED
ch6lc4bas3ev	regular	12.1MB	3 minutes ago	3 minutes ago	1	true
k99tfamuecbr	regular	7.04MB	3 minutes ago	3 minutes ago	1	true
vt7nuv4u4xuj	regular	8.82kB	3 minutes ago	3 minutes ago	1	true
kp01af3ah8f6	regular	13.2kB	3 minutes ago	3 minutes ago	1	true
0dchrtc6vcsc	regular	12.7kB	3 minutes ago	3 minutes ago	1	true
d0qjz4lwksdo	regular	13.5kB	3 minutes ago	3 minutes ago	1	true
ssk3nnkox0fm	regular	17.8kB	3 minutes ago	3 minutes ago	1	true
0a6rz6kxswfo	regular	37.1kB	2 minutes ago	2 minutes ago	1	true
jyn6chghjbvs	source.local	8.19kB	3 minutes ago	2 minutes ago	1	false
wjccqzkm123y	regular	37.7kB	2 minutes ago	2 minutes ago	1	true
jdbtrpk7axy7	source.local	8.19kB	3 minutes ago	2 minutes ago	1	false
uf2fbcttt12z	source.local	4.1kB	3 minutes ago	2 minutes ago	1	false
s5kpoe1amdma	regular	52.9MB	3 minutes ago	2 minutes ago	1	true

```

elfandi@elfandi-Latitude-7414:~/docker-lab/custom-web$ docker system prune -a
WARNING! This will remove:
 - all stopped containers
 - all networks not used by at least one container
 - all images without at least one container associated to them
 - all build cache

Are you sure you want to continue? [y/N] y
Deleted Images:
untagged: ubuntu:22.04
deleted: sha256:962f6cadeae0ea6284001009daa4cc9a8c37e75d1f5191cf0eb83fe565b63dd7
deleted: sha256:14be402d3f1eeeb5e7da73d3260322c68e7b51c88388f53e88eb21d6450bd520
deleted: sha256:268e076660f3cd225015df43cfe7198e5f787f24db00338d433299687237e538
untagged: hello-world:latest
deleted: sha256:f9078146db2e05e794366b1bfe584a14ea6317f44027d10ef7dad65279026885
deleted: sha256:d1a8d0a4eeb63aff09f5f34d4d80505e0ba81905f36158cc3970d8e07179e59e
deleted: sha256:8e752a1cddeafc02597e756f4a0ec96e29f63ac4bc4af87682daf3f1de843bb7
untagged: nginx:1.26-alpine

Deleted build cache objects:
wjccqzkml23yeb0lilabnmbad
jyn6chghjbvslhxje99y95r77
jdbtrpk7axy7flwxj8whik3n
uf2fbcttt12z2son4k4no3h3d
0q6rz6kxswfoxj2ex6c5q6oid
s5kpoe1amdahaqa9nj04cj7f
ssk3nnkox0fms64zbzpslk9y8
d0qjz4lwksdosxf4b974l8hgg
0dchrtc6vcsc43fqjvi7owguy
kp01af3ah8f6iqwfwuvqr0yk6
vt7nuv4u4xujz9c6v51dd7nee
k99tfamuecbn19tunt2xgj1db
ch6lc4bas3evx1ay3rj12n51t

Total reclaimed space: 104MB
elfandi@elfandi-Latitude-7414:~/docker-lab/custom-web$ docker system prune -a -f
Total reclaimed space: 0B

```

Langkah ini digunakan untuk melihat penggunaan storage Docker pada sistem. Informasi ini membantu mengetahui seberapa besar ruang yang digunakan oleh image, container, dan cache.

Tahap ini juga dilakukan untuk membersihkan resource Docker yang tidak digunakan seperti container berhenti, image lama, dan cache build. Hal ini bertujuan untuk menghemat ruang penyimpanan dan menjaga sistem tetap bersih.

E. PERTANYAAN

Pre-Lab

1. Sebutkan minimal 3 perbedaan antara Virtual Machine dan Container.

Virtual Machine (VM) menjalankan sistem operasi lengkap di atas hypervisor sehingga lebih berat dan membutuhkan resource besar, sedangkan container hanya menjalankan aplikasi dengan berbagi kernel host sehingga lebih ringan dan cepat. VM memiliki isolasi lebih kuat karena masing-masing

punya OS sendiri, sedangkan container lebih efisien tetapi isolasinya lebih terbatas.

2. Apa fungsi dari containerd dan runc dalam arsitektur Docker?

containerd berfungsi sebagai runtime tingkat tinggi yang mengelola lifecycle container seperti pull image, start, stop, dan storage. runc adalah runtime tingkat rendah yang bertugas mengeksekusi container berdasarkan spesifikasi OCI dan langsung berinteraksi dengan kernel Linux untuk menjalankan proses container.

3. Mengapa Docker membutuhkan kernel Linux? Bagaimana Docker Desktop di Windows mengatasi hal ini?

Docker membutuhkan kernel Linux karena fitur utama seperti namespaces dan cgroups hanya tersedia di Linux. Pada Windows, Docker Desktop mengatasi hal ini dengan menggunakan WSL2 atau Hyper-V yang menjalankan kernel Linux virtual sehingga container tetap bisa berjalan seperti di Linux asli.

4. Apa keuntungan layered filesystem pada Docker Image?

Layered filesystem memungkinkan setiap perubahan pada image disimpan sebagai layer terpisah sehingga proses build menjadi lebih efisien. Jika ada bagian image yang sama, Docker dapat melakukan reuse layer sehingga menghemat storage dan mempercepat proses download maupun build.

5. Jelaskan perbedaan antara docker run dan docker exec.

docker run digunakan untuk membuat dan menjalankan container baru dari sebuah image, sedangkan docker exec digunakan untuk menjalankan perintah di dalam container yang sudah berjalan tanpa membuat container baru.

Post-Lab

1. Bandingkan output docker image history nginx dengan docker image history pens-web:1.0. Layer mana saja yang di-share?

Image nginx memiliki banyak layer default dari base image seperti konfigurasi nginx, entrypoint, dan system alpine. Sedangkan pens-web:1.0 hanya menambahkan beberapa layer custom seperti COPY index.html, RUN cleanup, dan metadata. Layer yang di-share adalah layer base image nginx-alpine karena digunakan sebagai dasar dari pens-web.

2. Apa yang terjadi pada data di dalam container setelah container dihapus dengan docker rm? Bagaimana solusinya?

Ketika container dihapus menggunakan docker rm, semua data yang ada di dalam container juga akan hilang karena container bersifat ephemeral.

Solusinya adalah menggunakan volume atau bind mount agar data tetap persisten meskipun container dihapus.

3. Jelaskan perbedaan antara EXPOSE di Dockerfile dan flag -p pada docker run. Apakah EXPOSE cukup untuk membuat port dapat diakses dari host?

EXPOSE hanya berfungsi sebagai dokumentasi untuk menunjukkan port yang digunakan di dalam container, sedangkan -p pada docker run melakukan port mapping dari host ke container sehingga port benar-benar dapat diakses dari luar. EXPOSE saja tidak cukup untuk membuka akses dari host.

4. Mengapa menggunakan tag spesifik (misal nginx:1.26) lebih baik daripada nginx:latest untuk production?

Tag latest tidak stabil karena dapat berubah ketika image baru dirilis, sehingga bisa menyebabkan perubahan behavior aplikasi secara tidak terduga. Menggunakan tag spesifik seperti nginx:1.26 memastikan versi image konsisten dan stabil untuk production.

5. Berapa ukuran image alpine:3.20 dibanding ubuntu:22.04? Apa trade-off menggunakan Alpine?

Image alpine:3.20 berukuran sangat kecil (~5–10 MB), sedangkan ubuntu:22.04 jauh lebih besar (~70–80 MB). Alpine lebih ringan dan cepat tetapi menggunakan musl libc yang kadang tidak kompatibel dengan beberapa aplikasi, sedangkan Ubuntu lebih kompatibel tetapi lebih berat.

Modul 2

Docker Service Mount



A. TUJUAN PEMBELAJARAN

Setelah praktikum ini, mahasiswa mampu:

1. Memahami dan mengelola Docker Network: bridge, host, dan none
2. Menghubungkan antar container menggunakan user-defined bridge network
3. Memahami perbedaan tiga jenis mount: Volume, Bind Mount, dan tmpfs
4. Membuat dan mengelola Docker Volume untuk persistensi data
5. Menggunakan Bind Mount untuk development workflow (live-reload)
6. Menginstal dan menggunakan Docker Compose untuk orkestrasi multi-container
7. Menulis file docker-compose.yml dengan definisi service, network, dan volume
8. Mengelola lifecycle aplikasi multi-container dengan docker compose up/down/restart

B. DASAR TEORI

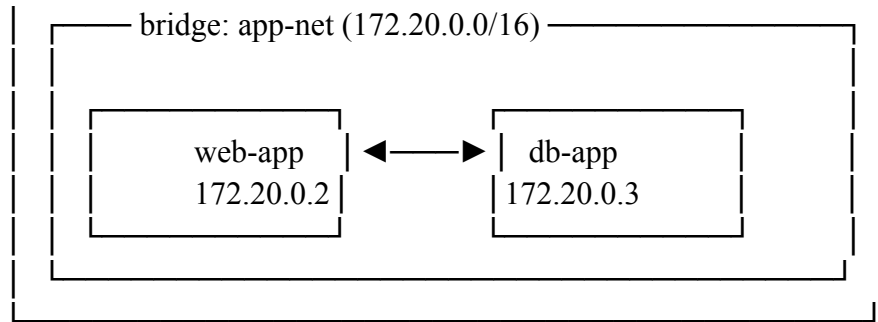
1. Docker Network

Docker menyediakan beberapa driver network untuk menghubungkan container:

Driver	Deskripsi	Use Case
bridge	Network default, container terisolasi dari host	Container standalone yang butuh komunikasi antar container
host	Container langsung pakai network host	Performa maksimal, tidak ada NAT overhead
none	Tidak ada network sama sekali	Container yang hanya butuh proses lokal
overlay	Multi-host networking	Docker Swarm / cluster

User-defined bridge lebih unggul dari default bridge karena container bisa saling resolve by nama (automatic DNS), isolasi lebih baik, dan bisa di-connect/disconnect saat container running.

Host Machine



2. Docker Mount: Volume vs Bind Mount vs tmpfs

Data di dalam container bersifat ephemeral — hilang saat container dihapus. Docker menyediakan tiga mekanisme mount untuk persistensi dan sharing data:

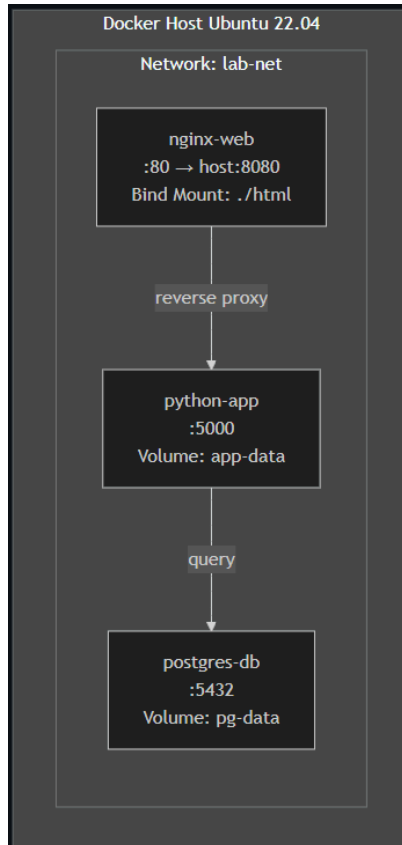
Aspek	Volume	Bind Mount	tmpfs
Lokasi di host	<code>/var/lib/docker/volumes/</code>	Path bebas di host	RAM (memory)
Dikelola oleh	Docker Engine	User / OS	Kernel
Portabilitas	Tinggi (managed)	Rendah (path-dependent)	Tidak persisten
Permission	Docker handle	Bisa conflict UID/GID	Docker handle
Best for	Database, persistent data	Source code, config dev	Secrets, cache sementara
Syntax <code>-v</code>	<code>-v vol_name:/path</code>	<code>-v /host/path:/path</code>	<code>--tmpfs /path</code>

3. Docker Compose

Docker Compose adalah tool untuk mendefinisikan dan menjalankan aplikasi multi-container menggunakan satu file YAML (`docker-compose.yml`). Keuntungan: satu file mendefinisikan seluruh stack, satu perintah untuk start/stop, environment reproducible dan version-controlled, serta dependency antar service otomatis dengan `depends_on`.

Catatan: Gunakan docker compose (spasi, plugin v2) bukan docker-compose (hyphen, v1 standalone yang sudah deprecated).

C. TOPOLOGI LAB



D. PRAKTIKUM

Langkah 1: Docker Network

1. Eksplorasi network default


```

elfandi@elfandi-Latitude-7414:~$ docker network ls
NETWORK ID        NAME          DRIVER        SCOPE
8451204ba590     bridge       bridge        local
3679be4c9975     host         host          local
bc6cf0dfd4f5     none         null          local
elfandi@elfandi-Latitude-7414:~$ docker network inspect bridge
[
  {
    "Name": "bridge",
    "Id": "8451204ba590ab76709b1ada996864f774a79e62022e87f22a2f8081e9530819",
    "Created": "2026-05-10T11:03:20.296763804+07:00",
    "Scope": "local",
    "Driver": "bridge",
    "EnableIPv4": true,
    "EnableIPv6": false,
    "IPAM": {
      "Driver": "default",
      "Options": null,
      "Config": [
        {
          "Subnet": "172.17.0.0/16",
          "Gateway": "172.17.0.1"
        }
      ]
    },
    "Internal": false,
    "Attachable": false,
    "Ingress": false,
    "ConfigFrom": {
      "Network": ""
    },
    "ConfigOnly": false,
    "Options": {
      "com.docker.network.bridge.default_bridge": "true",
      "com.docker.network.bridge.enable_icc": "true",
      "com.docker.network.bridge.enable_ip_masquerade": "true",
      "com.docker.network.bridge.host_binding_ipv4": "0.0.0.0",
      "com.docker.network.bridge.name": "docker0",
      "com.docker.network.driver.mtu": "1500"
    },
    "Labels": {},
    "Containers": {
      "dc02a695ed2d3738ee569bc1719fb9b3ac7da423e610a8017b949a28387069cb": {
        "Name": "pens-app",
        "EndpointID": "6dae0544972369fad0fa2e7bc637d573fd79690a2edb7b94fc4db4ec3dfefe25",
        "MacAddress": "de:98:c8:5e:b3:77",
        "IPv4Address": "172.17.0.2/16",
        "IPv6Address": ""
      },
      "f8d4b60c4c9b989b0dce378bffa7279c5662d1f42bfb8c854fea6797ccf9b3286": {
        "Name": "web-public",
        "EndpointID": "35019f0e59f248286fb84b1481ca2382b5f2f379fc435acfc58c6e682aaf9f28",
        "MacAddress": "66:61:3d:41:41:96",
        "IPv4Address": "172.17.0.3/16",
        "IPv6Address": ""
      }
    },
    "Status": {
      "IPAM": {
        "Subnets": [
          {
            "172.17.0.0/16": {
              "IPsInUse": 5,
              "DynamicIPsAvailable": 65531
            }
          }
        ]
      }
    }
  }
]

```

Langkah ini digunakan untuk melihat jaringan default yang tersedia pada Docker seperti bridge, host, dan none. Network bridge merupakan network utama yang digunakan container secara default jika tidak ditentukan network khusus. Inspect dilakukan untuk melihat konfigurasi detail seperti subnet, gateway, dan container yang terhubung.

2. Buat user-defined bridge network

```
elfandi@elfandi-Latitude-7414:~$ docker network create --driver bridge --subnet 172.20.0.0/16 lab-net
d24082dc4430f16e5cd852eec97185d0edbf26be82f2e16f7b7c5f7306eebb6f
elfandi@elfandi-Latitude-7414:~$ docker network ls
docker network inspect lab-net
NETWORK ID        NAME        DRIVER    SCOPE
8451204ba590     bridge     bridge    local
3679be4c9975     host       host      local
d24082dc4430     lab-net    bridge    local
bc6cf0dfd4f5     none      null      local
[
  {
    "Name": "lab-net",
    "Id": "d24082dc4430f16e5cd852eec97185d0edbf26be82f2e16f7b7c5f7306eebb6f",
    "Created": "2026-05-10T12:46:43.907783539+07:00",
    "Scope": "local",
    "Driver": "bridge",
    "EnableIPv4": true,
    "EnableIPv6": false,
    "IPAM": {
      "Driver": "default",
      "Options": {},
      "Config": [
        {
          "Subnet": "172.20.0.0/16",
          "Gateway": "172.20.0.1"
        }
      ]
    },
    "Internal": false,
    "Attachable": false,
    "Ingress": false,
    "ConfigFrom": {
      "Network": ""
    },
    "ConfigOnly": false,
    "Options": {},
    "Labels": {},
    "Containers": {},
    "Status": {
      "IPAM": {
        "Subnets": [
          {
            "172.20.0.0/16": {
              "IPsInUse": 3,
              "DynamicIPsAvailable": 65533
            }
          }
        ]
      }
    }
  }
]
```

Pada tahap ini dibuat network baru bernama lab-net dengan driver bridge dan subnet tertentu. User-defined network memungkinkan container saling berkomunikasi menggunakan nama container (DNS internal). Hal ini memberikan isolasi dan kontrol jaringan yang lebih baik dibanding default bridge.

3. Test DNS resolution antar container

```
elfandi@elfandi-Latitude-7414:~$ docker run -d --name server-a --network lab-net nginx:alpine
6559568fc756b6fd8b325a76a8fe4182c27b4774e1605dc0dfc93cca9b65c4f5
elfandi@elfandi-Latitude-7414:~$ docker run -d --name server-b --network lab-net nginx:alpine
6e9050677cc77c1db51611c0c95c258a805c6eb56291dfcb9b78068b9cef63ce
```

```
elfandi@elfandi-Latitude-7414:~$ docker exec server-a ping -c 3 server-b
PING server-b (172.20.0.3): 56 data bytes
64 bytes from 172.20.0.3: seq=0 ttl=64 time=0.109 ms
64 bytes from 172.20.0.3: seq=1 ttl=64 time=0.076 ms
64 bytes from 172.20.0.3: seq=2 ttl=64 time=0.098 ms
```

```
--- server-b ping statistics ---
3 packets transmitted, 3 packets received, 0% packet loss
round-trip min/avg/max = 0.076/0.094/0.109 ms
elfandi@elfandi-Latitude-7414:~$ docker exec server-b ping -c 3 server-a
PING server-a (172.20.0.2): 56 data bytes
64 bytes from 172.20.0.2: seq=0 ttl=64 time=0.076 ms
64 bytes from 172.20.0.2: seq=1 ttl=64 time=0.072 ms
64 bytes from 172.20.0.2: seq=2 ttl=64 time=0.069 ms

--- server-a ping statistics ---
3 packets transmitted, 3 packets received, 0% packet loss
round-trip min/avg/max = 0.069/0.072/0.076 ms
```

```
elfandi@elfandi-Latitude-7414:~$ docker run -d --name server-c nginx:alpine
324ac55b5bc3e900c68698b2ba43e17f90564c1337fd86e1666449b2c854b3e8
elfandi@elfandi-Latitude-7414:~$ docker run -d --name server-d nginx:alpine
0a3adf30108eb272447fca635e257b1b14f496ffa760bdab01832eebab9cae50
elfandi@elfandi-Latitude-7414:~$ docker exec server-c ping -c 3 server-d # GAGAL!
ping: bad address 'server-d'
elfandi@elfandi-Latitude-7414:~$
```

```
elfandi@elfandi-Latitude-7414:~$ docker rm -f server-a server-b server-c server-d
server-a
server-b
server-c
server-d
```

Langkah ini menguji kemampuan container dalam satu network untuk saling berkomunikasi menggunakan nama container sebagai hostname. Container server-a dan server-b dapat saling ping karena berada dalam network yang sama. Sedangkan container di default bridge tidak dapat resolve nama container lain karena tidak memiliki DNS internal seperti user-defined network.

Langkah 2: Docker Volume

1. Buat dan kelola volume

```
elfandi@elfandi-Latitude-7414:~$ docker volume create data-vol
data-vol
elfandi@elfandi-Latitude-7414:~$ docker volume ls
DRIVER      VOLUME NAME
local       data-vol
elfandi@elfandi-Latitude-7414:~$ docker volume inspect data-vol
[
  {
    "CreatedAt": "2026-05-10T12:56:21+07:00",
    "Driver": "local",
    "Labels": null,
    "Mountpoint": "/var/lib/docker/volumes/data-vol/_data",
    "Name": "data-vol",
    "Options": null,
    "Scope": "local"
  }
]
```

Volume Docker dibuat untuk menyimpan data secara persisten di luar container. Volume dapat digunakan kembali oleh container lain dan tidak terhapus meskipun container dihapus. Inspect digunakan untuk melihat lokasi dan konfigurasi volume di sistem Docker.

2. Gunakan volume di container

```
elfandi@elfandi-Latitude-7414:~$ docker run -d --name writer \
-v data-vol:/app/data \
  alpine:3.20 sh -c "while true; do date >> /app/data/log.txt; sleep 5; done"
Unable to find image 'alpine:3.20' locally
3.20: Pulling from library/alpine
25f1d6b1951a: Pull complete
d4050d56ebf2: Download complete
3a030ca3f633: Download complete
Digest: sha256:d9e853e87e55526f6b2917df91a2115c36dd7c696a35be12163d44e6e2a4b6bc
Status: Downloaded newer image for alpine:3.20
c2f0fd717ee6244d8b40aac83c7bba6c23551a8732d0a4eee725b63942ce519d
elfandi@elfandi-Latitude-7414:~$ sleep 15
docker run --rm -v data-vol:/data alpine:3.20 cat /data/log.txt
Sun May 10 05:57:01 UTC 2026
Sun May 10 05:57:06 UTC 2026
Sun May 10 05:57:11 UTC 2026
Sun May 10 05:57:16 UTC 2026
Sun May 10 05:57:21 UTC 2026
Sun May 10 05:57:26 UTC 2026
```

Pada tahap ini volume dipasang ke container writer untuk menyimpan data

log secara berulang. Container akan menulis data ke volume setiap beberapa detik. Container lain dapat membaca data yang sama dari volume tersebut, menunjukkan bahwa volume dapat digunakan bersama antar container.

3. Volume persist setelah container dihapus

```
elfandi@elfandi-Latitude-7414:~$ docker rm -f writer
writer
elfandi@elfandi-Latitude-7414:~$ docker run --rm -v data-vol:/data alpine:3.20 cat /data/log.txt
Sun May 10 05:57:01 UTC 2026
Sun May 10 05:57:06 UTC 2026
Sun May 10 05:57:11 UTC 2026
Sun May 10 05:57:16 UTC 2026
Sun May 10 05:57:21 UTC 2026
Sun May 10 05:57:26 UTC 2026
Sun May 10 05:57:31 UTC 2026
Sun May 10 05:57:36 UTC 2026
Sun May 10 05:57:41 UTC 2026
Sun May 10 05:57:46 UTC 2026
Sun May 10 05:57:51 UTC 2026
Sun May 10 05:57:56 UTC 2026
Sun May 10 05:58:01 UTC 2026
Sun May 10 05:58:06 UTC 2026
Sun May 10 05:58:11 UTC 2026
Sun May 10 05:58:16 UTC 2026
Sun May 10 05:58:21 UTC 2026
Sun May 10 05:58:26 UTC 2026
Sun May 10 05:58:31 UTC 2026
Sun May 10 05:58:36 UTC 2026
Sun May 10 05:58:41 UTC 2026
Sun May 10 05:58:46 UTC 2026
Sun May 10 05:58:51 UTC 2026
```

Langkah ini membuktikan bahwa data dalam volume tidak hilang meskipun container yang menggunakannya dihapus. Volume tetap menyimpan data secara permanen sampai dihapus secara manual. Hal ini membuktikan sifat persistent storage dari Docker volume.

4. Backup dan restore volume

```

elfandi@elfandi-Latitude-7414:~$ docker run --rm \
-v data-vol:/source:ro \
-v $(pwd):/backup \
alpine:3.20 tar czf /backup/data-vol-backup.tar.gz -C /source .
elfandi@elfandi-Latitude-7414:~$ docker volume create data-vol-restored
elfandi@elfandi-Latitude-7414:~$ docker run --rm \
-v data-vol-restored:/target \
-v $(pwd):/backup:ro \
alpine:3.20 tar xzf /backup/data-vol-backup.tar.gz -C /target
data-vol-restored
elfandi@elfandi-Latitude-7414:~$ docker run --rm -v data-vol-restored:/data alpine:3.20 cat /data/log.txt
Sun May 10 05:57:01 UTC 2026
Sun May 10 05:57:06 UTC 2026
Sun May 10 05:57:11 UTC 2026
Sun May 10 05:57:16 UTC 2026
Sun May 10 05:57:21 UTC 2026
Sun May 10 05:57:26 UTC 2026
Sun May 10 05:57:31 UTC 2026
Sun May 10 05:57:36 UTC 2026
Sun May 10 05:57:41 UTC 2026
Sun May 10 05:57:46 UTC 2026
Sun May 10 05:57:51 UTC 2026
Sun May 10 05:57:56 UTC 2026
Sun May 10 05:58:01 UTC 2026
Sun May 10 05:58:06 UTC 2026
Sun May 10 05:58:11 UTC 2026
Sun May 10 05:58:16 UTC 2026
Sun May 10 05:58:21 UTC 2026
Sun May 10 05:58:26 UTC 2026
Sun May 10 05:58:31 UTC 2026
Sun May 10 05:58:36 UTC 2026
Sun May 10 05:58:41 UTC 2026
Sun May 10 05:58:46 UTC 2026
Sun May 10 05:58:51 UTC 2026

```

Pada tahap ini dilakukan backup isi volume ke file archive dan kemudian direstore ke volume baru. Proses ini berguna untuk migrasi data atau backup aplikasi. Setelah restore, data dapat diverifikasi kembali untuk memastikan tidak ada kehilangan data.

Langkah 3: Bind Mount

1. Buat project

```

elfandi@elfandi-Latitude-7414:~$ mkdir -p ~/docker-lab/web-dev/html && cd ~/docker-lab/web-dev
elfandi@elfandi-Latitude-7414:~/docker-lab/web-dev$ cat > html/index.html << 'EOF'
<html>
<body>
  <h1>Hello dari Bind Mount!</h1>
  <p>Timestamp: VERSI-1</p>
</body>
</html>
EOF

```

Langkah ini membuat struktur folder project dan file HTML yang akan digunakan sebagai konten website. File ini disiapkan di host agar bisa dihubungkan langsung ke container menggunakan bind mount.

2. Jalankan container dengan bind mount

```
elfandi@elfandi-Latitude-7414:~/docker-lab/web-dev$ docker run -d --name dev-server -p 8080:80 -v $(pwd)/html:/usr/share/nginx/html:ro nginx:alpine
```

Bind mount digunakan untuk menghubungkan folder di host langsung ke dalam container. Perubahan file di host akan langsung terlihat di container tanpa perlu rebuild image atau restart container. Hal ini sangat berguna untuk development.

3. Test live-reload

```
elfandi@elfandi-Latitude-7414:~/docker-lab/web-dev$ sed -i 's/VERSI-1/VERSI-2 (diedit live)/' html/index.html
curl http://localhost:8080
<html>
<body>
  <h1>Hello dari Bind Mount!</h1>
  <p>Timestamp: VERSI-2 (diedit live)</p>
</body>
</html>
elfandi@elfandi-Latitude-7414:~/docker-lab/web-dev$ docker rm -f dev-server
dev-server
```

Pada tahap ini dilakukan pengujian perubahan file secara real-time. Ketika file HTML diubah di host, perubahan langsung muncul di container karena bind mount bersifat real-time sync. Setelah pengujian, container dihentikan dan dihapus.

Langkah 4: tmpfs Mount

1. tmpfs Mount

```
elfandi@elfandi-Latitude-7414:~/docker-lab/web-dev$ docker run -d --name tmpfs-demo \
  --tmpfs /app/cache:size=64m \
  alpine:3.20 sh -c "echo 'secret-data' > /app/cache/token.txt && sleep 3600"
ccfce4ab1a73fdc324683935c008462f3de08328148bbe70657402c4990fed6e
elfandi@elfandi-Latitude-7414:~/docker-lab/web-dev$ docker exec tmpfs-demo cat /app/cache/token.txt
secret-data
elfandi@elfandi-Latitude-7414:~/docker-lab/web-dev$ docker stop tmpfs-demo && docker start tmpfs-demo
docker exec tmpfs-demo cat /app/cache/token.txt # File tidak ada!
tmpfs-demo
tmpfs-demo
secret-data
elfandi@elfandi-Latitude-7414:~/docker-lab/web-dev$ docker rm -f tmpfs-demo
tmpfs-demo
elfandi@elfandi-Latitude-7414:~/docker-lab/web-dev$
```

tmpfs mount digunakan untuk menyimpan data di RAM, bukan di disk. Data bersifat sementara dan akan hilang ketika container dihentikan atau restart. Hal ini cocok untuk cache atau data sementara yang tidak perlu disimpan permanen.

Langkah 5: Aplikasi Multi-Container dengan Docker Compose

1. Buat project structure

```
elfandi@elfandi-Latitude-7414:~/docker-lab/web-dev$ mkdir -p ~/docker-lab/compose-app/{html,app} && cd ~/docker-lab/compose-app
```

Langkah ini membuat struktur folder untuk aplikasi multi-service yang terdiri dari web, backend, dan database. Struktur ini digunakan untuk memisahkan komponen aplikasi agar lebih rapi dan mudah dikelola.

2. Buat halaman Nginx

```
elfandi@elfandi-Latitude-7414:~/docker-lab/compose-app$ cat > html/index.html << 'EOF'
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8"><title>Docker Compose Lab</title>
  <style>
    body { font-family: sans-serif; max-width: 700px; margin: 40px auto; padding: 20px; }
    .card { background: white; border-radius: 10px; padding: 25px;
            box-shadow: 0 2px 10px rgba(0,0,0,0.1); margin-bottom: 20px; }
    #result { background: #263238; color: #80CBC4; padding: 15px;
             border-radius: 8px; font-family: monospace; white-space: pre-wrap; }
  </style>
</head>
<body>
<div class="card">
  <h1>🐳 Docker Compose Lab</h1>
  <p>Nginx → Flask → PostgreSQL</p>
  <button onclick="fetchData()">Cek Koneksi Backend</button>
  <div id="result">Klik tombol untuk test...</div>
</div>
<script>
async function fetchData() {
  try {
    const r = await fetch('/api/health');
    document.getElementById('result').textContent = JSON.stringify(await r.json(), null, 2);
  } catch(e) { document.getElementById('result').textContent = 'Error: ' + e.message; }
}
</script>
</body></html>
EOF
```

Pada tahap ini dibuat halaman web sederhana yang akan ditampilkan oleh service Nginx. Halaman ini juga digunakan untuk mengakses API backend melalui tombol yang disediakan di UI.

3. Buat Flask app + Dockerfile


```

EOF
elfandi@elfandi-Latitude-7414:~/docker-lab/compose-app$ cat > app/requirements.txt << 'EOF'
flask==3.1.*
psycopg2-binary==2.9.*
EOF

cat > app/app.py << 'PYEOF'
import os, socket, datetime
from flask import Flask, jsonify
import psycopg2

app = Flask(__name__)

@app.route("/api/health")
def health():
    result = {"status": "ok", "hostname": socket.gethostname(),
             "timestamp": datetime.datetime.now().isoformat()}

    try:
        conn = psycopg2.connect(
            host=os.environ.get("DB_HOST", "db"),
            dbname=os.environ.get("DB_NAME", "labdb"),
            user=os.environ.get("DB_USER", "labuser"),
            password=os.environ.get("DB_PASS", "labpass123"))
        cur = conn.cursor()
        cur.execute("SELECT version();")
        result["database"] = cur.fetchone()[0]
        result["db_status"] = "connected"
        cur.close(); conn.close()
    except Exception as e:
        result["db_status"] = f"error: {e}"
    return jsonify(result)

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
PYEOF

cat > app/Dockerfile << 'EOF'
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
EXPOSE 5000
CMD ["python", "app.py"]
EOF

```

Flask digunakan sebagai backend API yang terhubung ke database PostgreSQL. Dockerfile dibuat untuk membangun image aplikasi Python dengan dependencies yang diperlukan. Aplikasi ini akan memberikan response API kesehatan sistem dan koneksi database.

4. Buat Nginx config

```
elfandi@elfandi-Latitude-7414:~/docker-lab/compose-app$ cat > nginx.conf << 'EOF'
server {
    listen 80;
    location / {
        root /usr/share/nginx/html;
        index index.html;
    }
    location /api/ {
        proxy_pass http://app:5000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
EOF
```

Konfigurasi Nginx dibuat untuk meneruskan request API ke service backend Flask. Selain itu Nginx juga digunakan untuk menyajikan file HTML statis. Proxy_pass digunakan untuk komunikasi antar service di dalam Docker network.

5. Buat docker-compose.yml

```

elfandi@elfandi-Latitude-7414:~/docker-lab/compose-app$ cat > docker-compose.yml << 'EOF'
services:
  web:
    image: nginx:alpine
    container_name: lab-web
    ports:
      - "8080:80"
    volumes:
      - ./html:/usr/share/nginx/html:ro
      - ./nginx.conf:/etc/nginx/conf.d/default.conf:ro
    networks:
      - frontend
    depends_on:
      - app
    restart: unless-stopped

  app:
    build: ./app
    container_name: lab-app
    environment:
      - DB_HOST=db
      - DB_NAME=labdb
      - DB_USER=labuser
      - DB_PASS=labpass123
    networks:
      - frontend
      - backend
    depends_on:
      db:
        condition: service_healthy
    restart: unless-stopped

  db:
    image: postgres:16-alpine
    container_name: lab-db
    environment:
      POSTGRES_DB: labdb
      POSTGRES_USER: labuser
      POSTGRES_PASSWORD: labpass123
    volumes:
      - pg-data:/var/lib/postgresql/data
    networks:
      - backend
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U labuser -d labdb"]
      interval: 5s
      timeout: 5s
      retries: 5
    restart: unless-stopped

volumes:
  pg-data:

networks:
  frontend:
  backend:
EOF

```

Docker Compose digunakan untuk mengelola multi-container secara terstruktur dalam satu file konfigurasi. Terdapat tiga service yaitu web (Nginx), app (Flask), dan database (PostgreSQL). Setiap service terhubung melalui network frontend dan backend serta menggunakan volume untuk persistensi data database.

6. Jalankan dan verifikasi

```
elfandi@elfandi-Latitude-7414:~/docker-lab/compose-app$ docker compose up --build -d
[+] up 13/13
✓ Image postgres:16-alpine Pulled
[+] Building 34.3s (12/12) FINISHED
=> [internal] load local bake definitions
=> => reading from stdin 528B
=> [internal] load build definition from DockerFile
=> => transferring dockerfile: 200B
=> [internal] load metadata for docker.io/library/python:3.11-slim
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/5] FROM docker.io/library/python:3.11-slim@sha256:a5b427ace4900267d93db34138e512325c6fa6af84ad5e4ed5f3b36258cc414
=> => resolve docker.io/library/python:3.11-slim@sha256:a5b427ace4900267d93db34138e512325c6fa6af84ad5e4ed5f3b36258cc414
=> => sha256:6f92665ed17afc6850bfbeb3fb681d6e1038fe59e2020ab126b859ec572da21b 250B / 250B
=> => sha256:fb4c70443787d9baef637d0b257f21b935d5feb6481f1ccdf4d07f48b2e393c1 14.37MB / 14.37MB
=> => sha256:01f59aef9b5c2caa2870aa8b9b8b5006ea3c36d893cd6e2467e252fc1b1fea46 1.29MB / 1.29MB
=> => extracting sha256:01f59aef9b5c2caa2870aa8b9b8b5006ea3c36d893cd6e2467e252fc1b1fea46
=> => extracting sha256:fb4c70443787d9baef637d0b257f21b935d5feb6481f1ccdf4d07f48b2e393c1
=> => extracting sha256:6f92665ed17afc6850bfbeb3fb681d6e1038fe59e2020ab126b859ec572da21b
=> [internal] load build context
=> => transferring context: 1.0kB
=> [2/5] WORKDIR /app
=> [3/5] COPY requirements.txt .
=> [4/5] RUN pip install --no-cache-dir -r requirements.txt
=> [5/5] COPY app.py .
=> => exporting to image
=> => exporting layers
=> => exporting manifest sha256:609abf98abbae489dac6b20790850850fb4aa57779546db9d3091ea1aac3bed0
=> => exporting config sha256:b8595e5feb5f2c99080ed3dc5ba87b0649c08c45130fb04060514ee21fc899b7
=> => exporting attestation manifest sha256:c5508c9633448cb722082087f8c36f8593f26129fab85c46a7ae8c16732b1ef
=> => exporting manifest list sha256:ecdff0b73b472d1238cfc8b14525404ce50ba782858f01bdcab4bdee62269313
=> => naming to docker.io/library/compose-app-app:latest
[+] up 20/20 King to docker.io/library/compose-app-app:latest
✓ Image postgres:16-alpine Pulled
✓ Image compose-app-app Built
✓ Network compose-app_backend Created
✓ Network compose-app_frontend Created
✓ Volume compose-app_pg-data Created
✓ Container lab-db Healthy
✓ Container lab-app Started
✓ Container lab-web Started
```

```
elfandi@elfandi-Latitude-7414:~/docker-lab/compose-app$ docker compose ps
NAME          IMAGE          COMMAND                  SERVICE    CREATED      STATUS      PORTS
lab-app       compose-app-app "python app.py"         app        54 seconds ago Up 48 seconds 5000/tcp
lab-db        postgres:16-alpine "docker-entrypoint.s..." db         54 seconds ago Up 53 seconds (healthy) 5432/tcp
lab-web       nginx:alpine   "/docker-entrypoint..." web        54 seconds ago Up 47 seconds 0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
```

```
elfandi@elfandi-Latitude-7414:~/docker-lab/compose-app$ curl http://localhost:8080
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8"><title>Docker Compose Lab</title>
  <style>
    body { font-family: sans-serif; max-width: 700px; margin: 40px auto; padding: 20px; }
    .card { background: white; border-radius: 10px; padding: 25px;
      box-shadow: 0 2px 10px rgba(0,0,0,0.1); margin-bottom: 20px; }
    #result { background: #263238; color: #80CBC4; padding: 15px;
      border-radius: 8px; font-family: monospace; white-space: pre-wrap; }
  </style>
</head>
<body>
<div class="card">
  <h1>🐳 Docker Compose Lab</h1>
  <p>Nginx → Flask → PostgreSQL</p>
  <button onclick="fetchData()">Cek Koneksi Backend</button>
  <div id="result">Klik tombol untuk test...</div>
</div>
<script>
async function fetchData() {
  try {
    const r = await fetch('/api/health');
    document.getElementById('result').textContent = JSON.stringify(await r.json(), null, 2);
  } catch(e) { document.getElementById('result').textContent = 'Error: ' + e.message; }
}
</script>
</body></html>
elfandi@elfandi-Latitude-7414:~/docker-lab/compose-app$ curl http://localhost:8080/api/health | python3 -m json.tool
{
  "database": "PostgreSQL 16.13 on x86_64-pc-linux-musl, compiled by gcc (Alpine 15.2.0) 15.2.0, 64-bit",
  "db_status": "connected",
  "hostname": "4f79845d13a4",
  "status": "ok",
  "timestamp": "2026-05-10T06:11:00.506645"
}
```

```
elfandi@elfandi-Latitude-7414:~/docker-lab/compose-app$ docker network ls
NETWORK ID          NAME                DRIVER              SCOPE
8451204ba590        bridge              bridge              local
e751c684a78d        compose-app_backend bridge              local
fe3e5a45c6f4        compose-app_frontend bridge              local
3679be4c9975        host                host                local
d24082dc4430        lab-net             bridge              local
bc6cf0dfd4f5        none                null                local
elfandi@elfandi-Latitude-7414:~/docker-lab/compose-app$ docker volume ls
DRIVER              VOLUME NAME
local               compose-app_pg-data
local               data-vol
local               data-vol-restored
```

#lifecycle

```
elfandi@elfandi-Latitude-7414:~/docker-lab/compose-app$ docker compose logs -f
lab-app | * Serving Flask app 'app'
lab-app | * Debug mode: off
lab-app | WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead
lab-app | * Running on all addresses (0.0.0.0)
lab-app | * Running on http://127.0.0.1:5000
lab-app | * Running on http://172.18.0.3:5000
lab-app | Press CTRL+C to quit
lab-app | 172.19.0.3 - - [10/May/2026 06:11:00] "GET /api/health HTTP/1.1" 200 -
lab-web | /docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
lab-web | /docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
lab-web | /docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
lab-web | 10-listen-on-ipv6-by-default.sh: info: can not modify /etc/nginx/conf.d/default.conf (read-only file system?)
lab-web | /docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
lab-web | /docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
lab-web | /docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
lab-web | /docker-entrypoint.sh: Configuration complete; ready for start up
lab-web | 2026/05/10 06:09:48 [notice] 1#1: using the "epoll" event method
lab-web | 2026/05/10 06:09:48 [notice] 1#1: nginx/1.29.8
lab-web | 2026/05/10 06:09:48 [notice] 1#1: built by gcc 15.2.0 (Alpine 15.2.0)
lab-web | 2026/05/10 06:09:48 [notice] 1#1: OS: Linux 6.17.0-22-generic
```

```

elfandigelfandi-Latitude-7414:~/docker-lab/compose-app$ docker compose stop      # stop semua
[+] stop 3/3
✓ Container lab-web Stopped
✓ Container lab-app Stopped
✓ Container lab-db Stopped
elfandigelfandi-Latitude-7414:~/docker-lab/compose-app$ docker compose start    # start kembali
[+] start 3/3
✓ Container lab-db Healthy
✓ Container lab-app Started
✓ Container lab-web Started
elfandigelfandi-Latitude-7414:~/docker-lab/compose-app$ docker compose down    # stop + hapus container/network (volume tetap)
[+] down 5/5
✓ Container lab-web Removed
✓ Container lab-app Removed
✓ Container lab-db Removed
✓ Network compose-app_frontend Removed
✓ Network compose-app_backend Removed
elfandigelfandi-Latitude-7414:~/docker-lab/compose-app$ docker compose down -v  # HATI-HATI: hapus termasuk vol
[+] down 1/1
✓ Volume compose-app_pg-data Removed

```

Pada tahap ini seluruh service dijalankan menggunakan docker compose up. Setelah berjalan, dilakukan pengujian akses web dan API untuk memastikan semua service saling terhubung dengan benar. Selain itu juga dilakukan monitoring network, volume, serta lifecycle container menggunakan perintah compose.

E. PERTANYAAN

Pre-Lab

1. Apa perbedaan default bridge dan user-defined bridge network?

Default bridge adalah network bawaan Docker yang tidak mendukung DNS internal antar container, sehingga container hanya bisa berkomunikasi menggunakan IP. Sedangkan user-defined bridge memungkinkan komunikasi menggunakan nama container (DNS resolution), memiliki isolasi lebih baik, dan konfigurasi network lebih fleksibel.

2. Kapan menggunakan Volume vs Bind Mount vs tmpfs?

Volume digunakan untuk data persisten yang dikelola Docker seperti database. Bind mount digunakan untuk development karena file di host langsung tersinkron ke container. tmpfs digunakan untuk data sementara di memory (RAM) yang akan hilang saat container berhenti, cocok untuk cache atau data sensitif sementara.

3. Apa yang terjadi pada named volume saat docker compose down? Bagaimana jika pakai flag -v?

Saat docker compose down, container dan network akan dihapus tetapi named volume tetap ada sehingga data tidak hilang. Jika menggunakan flag -v, maka volume juga akan dihapus sehingga semua data persisten akan hilang.

4. Apa fungsi depends_on dan healthcheck di docker-compose.yml?

depends_on mengatur urutan startup antar service, misalnya database harus jalan sebelum aplikasi. healthcheck digunakan untuk memastikan service benar-benar siap digunakan, bukan hanya sekadar running, sehingga meningkatkan reliabilitas koneksi antar container.

5. Mengapa user-defined bridge bisa DNS resolve nama container, sedangkan default bridge tidak?

User-defined bridge memiliki embedded DNS server yang otomatis memetakan nama container ke IP internal. Sedangkan default bridge tidak memiliki DNS internal sehingga container hanya bisa berkomunikasi menggunakan IP address secara manual.

Post-Lab

1. Jalankan `docker network inspect lab-frontend`. Sebutkan container dan IP masing-masing.

Hasil `docker network inspect lab-frontend` akan menampilkan daftar container yang terhubung ke network tersebut beserta IP internal masing-masing container. Setiap container mendapatkan IP unik dalam subnet network (misalnya 172.20.0.x) yang digunakan untuk komunikasi internal antar service.

2. Hapus container lab-db lalu `docker compose up -d` lagi. Apakah data PostgreSQL masih ada? Mengapa?

Hasil `docker network inspect lab-frontend` akan menampilkan daftar container yang terhubung ke network tersebut beserta IP internal masing-masing container. Setiap container mendapatkan IP unik dalam subnet network (misalnya 172.20.0.x) yang digunakan untuk komunikasi internal antar service.

3. Tunjukkan perbedaan output `docker inspect` untuk mount type volume vs bind.

Volume ditandai dengan type “volume” dan dikelola oleh Docker di internal storage (misalnya `/var/lib/docker/volumes`). Sedangkan bind mount ditandai dengan type “bind” dan menunjuk langsung ke path di host. Volume lebih aman untuk production, sedangkan bind mount lebih fleksibel untuk development.

4. Jelaskan alur request dari browser → Nginx → Flask → PostgreSQL.

Request dari browser masuk ke Nginx pada port 8080. Jika request API (`/api/health`), Nginx akan meneruskan request ke service Flask. Flask kemudian memproses request dan melakukan query ke PostgreSQL. Hasil dari database dikembalikan ke Flask, lalu diteruskan kembali ke Nginx dan akhirnya ditampilkan ke browser dalam bentuk JSON.

5. Bandingkan ukuran image yang digunakan stack ini. Mana terbesar dan mengapa?

Image PostgreSQL biasanya paling besar karena membawa database engine lengkap dan dependencies sistem. Image Python Flask berada di tengah karena membutuhkan runtime Python dan library tambahan. Image Nginx paling kecil karena hanya berfungsi sebagai web server ringan berbasis Alpine.

Perbedaan ukuran terjadi karena kompleksitas dan jumlah dependency masing-masing image.

Modul 3

Web Service di Docker

– Apache & Nginx



A. TUJUAN PEMBELAJARAN

Setelah praktikum ini, mahasiswa mampu:

1. Men-deploy Apache2 (httpd) sebagai container Docker dengan konfigurasi custom
2. Men-deploy Nginx sebagai container Docker dengan konfigurasi custom
3. Membuat Virtual Host (name-based) di Apache dan Nginx dalam container
4. Mengkonfigurasi SSL/TLS dengan self-signed certificate pada web server container
5. Menggunakan Nginx sebagai reverse proxy untuk backend service di container lain
6. Membandingkan konfigurasi dan performa Apache vs Nginx dalam konteks container
7. Mengatur persistent logging dengan bind mount / volume
8. Mengorkestrasi seluruh stack menggunakan Docker Compose

B. DASAR TEORI

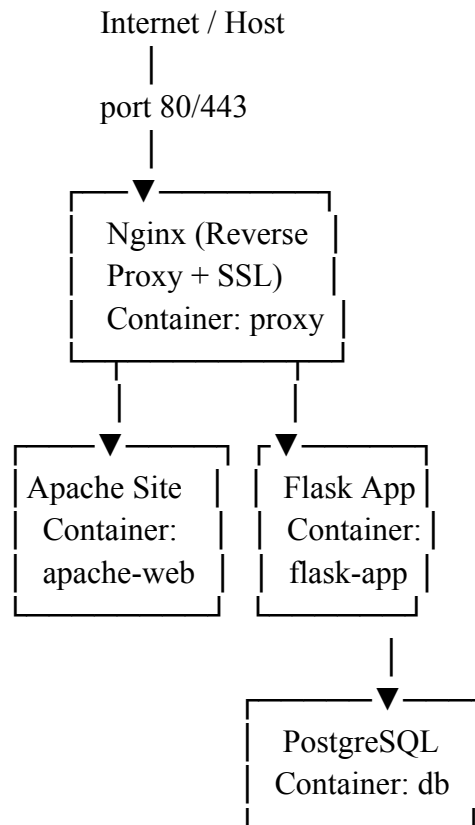
1. Web Server dalam Container

Menjalankan web server di container memberikan beberapa keuntungan: isolasi konfigurasi antar project, kemudahan scaling, reproducible environment, dan deployment yang konsisten. Image resmi Apache (httpd) dan Nginx (nginx) tersedia di Docker Hub dengan berbagai variant (alpine, debian, dll).

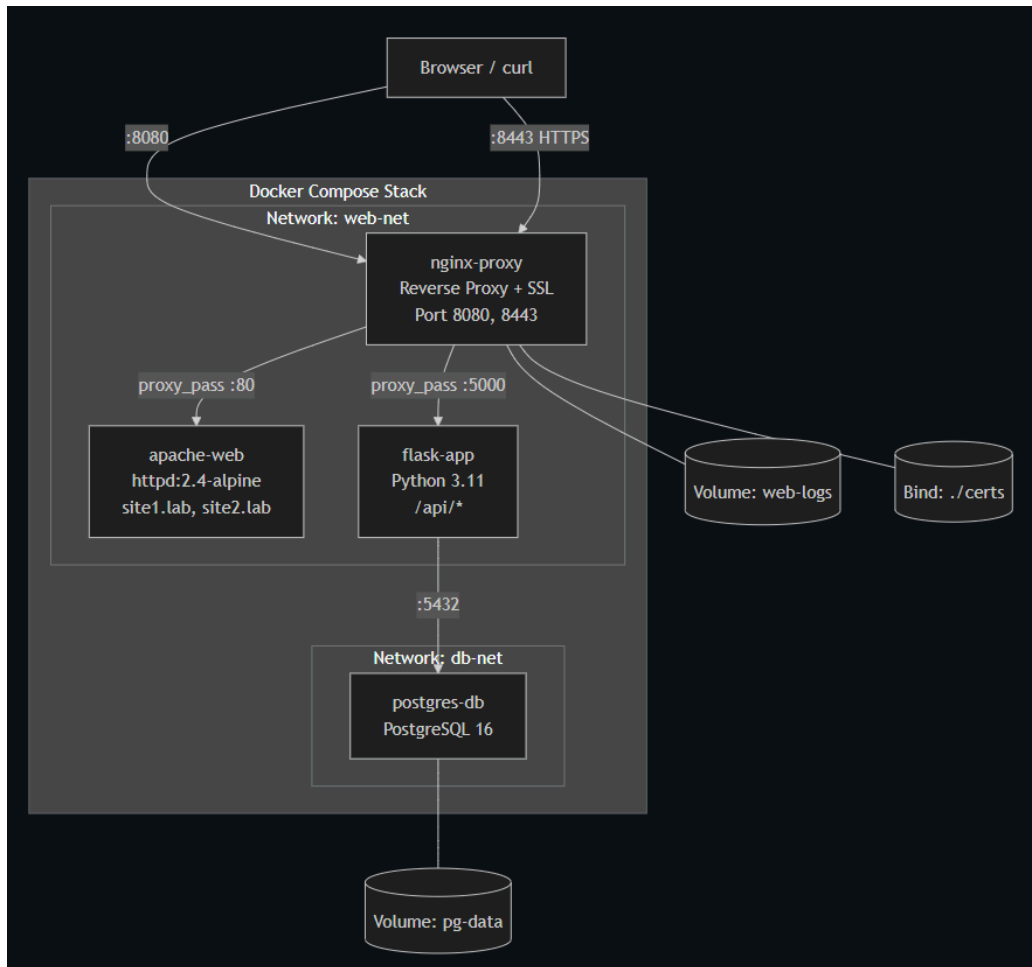
2. Apache vs Nginx di Docker

Aspek	Apache (httpd)	Nginx
Image resmi	httpd:2.4-alpine	nginx:alpine
Config dir	/usr/local/apache2/conf/	/etc/nginx/
Document root	/usr/local/apache2/htdocs/	/usr/share/nginx/html/
Virtual host	httpd-vhosts.conf	conf.d/*.conf
Arsitektur	Process/thread per request	Event-driven, async
Module system	.so modules, .htaccess	Compiled modules
Reverse proxy	mod_proxy	proxy_pass built-in
Ukuran image (alpine)	~60 MB	~45 MB

3. Arsitektur Lab



C. TOPOLOGI LAB



D. LANGKAH PRAKTIKUM

Langkah 0: Persiapan Project

```

PS C:\Users\ASUS\docker-lab\compose-app> mkdir -p "$HOME/docker-lab/web-service/apache/sites", "$HOME/docker-lab/web-service/apache/html-site1", "$HOME/
docker-lab/web-service/apache/html-site2", "$HOME/docker-lab/web-service/nginx", "$HOME/docker-lab/web-service/flask", "$HOME/docker-lab/web-service/cer
ts", "$HOME/docker-lab/web-service/logs"

Directory: C:\Users\ASUS\docker-lab\web-service\apache

Mode                LastWriteTime         Length Name
----                -
d-----          10/05/2026    08.57             sites
d-----          10/05/2026    08.57          html-site1
d-----          10/05/2026    08.57          html-site2

Directory: C:\Users\ASUS\docker-lab\web-service

Mode                LastWriteTime         Length Name
----                -
d-----          10/05/2026    08.57             nginx
d-----          10/05/2026    08.57             flask
d-----          10/05/2026    08.57             certs
d-----          10/05/2026    08.57             logs

PS C:\Users\ASUS\docker-lab\compose-app> cd ~/docker-lab/web-service
PS C:\Users\ASUS\docker-lab\web-service>
  
```

Command `mkdir -p "$HOME/..."` digunakan untuk membuat banyak folder sekaligus sesuai struktur project yang dibutuhkan, seperti folder Apache, Nginx, Flask, certs, dan logs. Opsi `-p` memungkinkan folder parent otomatis dibuat jika

belum ada dan mencegah error jika sebagian folder sudah tersedia.
Command `cd ~/docker-lab/web-service` digunakan untuk berpindah lokasi terminal ke folder web-service di dalam directory docker-lab, sehingga command berikutnya akan dijalankan dari folder tersebut.

Langkah 1: Konfigurasi Apache2 dengan Virtual Host

2. Buat halaman web untuk dua virtual host

Site 1: Company Profile

```
PS C:\Users\ASUS\docker-lab\web-service> @'
>> <!DOCTYPE html>
>> <html lang="id">
>> <head>
>>   <meta charset="UTF-8">
>>   <title>Site 1 - Company</title>
>>   <style>
>>     body{font-family:sans-serif;text-align:center;padding:50px;background:#1a237e;color:white;}
>>     .box{background:rgba(255,255,255,0.1);padding:30px;border-radius:12px;max-width:500px;margin:0 auto;}
>>   </style>
>> </head>
>> <body>
>>   <div class="box">
>>     <h1> 🏢 Site 1 - Company Profile</h1>
>>     <p>Virtual Host: <strong>site1.lab</strong></p>
>>     <p>Server: Apache httpd di Docker</p>
>>   </div>
>> </body>
>> </html>
>> '@ | Set-Content apache/html-site1/index.html
PS C:\Users\ASUS\docker-lab\web-service> |
```

Command diatas digunakan untuk membuat file index.html pada folder apache/html-site1 menggunakan Set-Content. Isi file berupa halaman web sederhana untuk site1.lab yang menampilkan profil company, virtual host, dan styling CSS dasar.

Site 2: Blog

```
PS C:\Users\ASUS\docker-lab\web-service> @'
>> <!DOCTYPE html>
>> <html lang="id">
>> <head>
>>   <meta charset="UTF-8">
>>   <title>Site 2 - Blog</title>
>>   <style>
>>     body{font-family:sans-serif;text-align:center;padding:50px;background:#1b5e20;color:white;}
>>     .box{background:rgba(255,255,255,0.1);padding:30px;border-radius:12px;max-width:500px;margin:0 auto;}
>>   </style>
>> </head>
>> <body>
>>   <div class="box">
>>     <h1> 📝 Site 2 - Blog</h1>
>>     <p>Virtual Host: <strong>site2.lab</strong></p>
>>     <p>Server: Apache httpd di Docker</p>
>>   </div>
>> </body>
>> </html>
>> '@ | Set-Content apache/html-site2/index.html
PS C:\Users\ASUS\docker-lab\web-service> |
```

Command diatas digunakan untuk membuat file index.html pada folder apache/html-site2 menggunakan Set-Content. Isi file berupa halaman web sederhana untuk site2.lab yang menampilkan judul, informasi virtual host, serta styling CSS dasar.

3. Buat konfigurasi Apache Virtual Host

```

PS C:\Users\ASUS\docker-lab\web-service> @'
>> # =====
>> # Apache Virtual Host Configuration (Docker)
>> # =====
>>
>> # --- Site 1: site1.lab ---
>> <VirtualHost *:80>
>>     ServerName site1.lab
>>     ServerAlias www.site1.lab
>>     DocumentRoot /usr/local/apache2/htdocs/site1
>>
>>     <Directory /usr/local/apache2/htdocs/site1>
>>         AllowOverride None
>>         Require all granted
>>     </Directory>
>>
>>     # Per-site logging
>>     ErrorLog /var/log/apache2/site1-error.log
>>     CustomLog /var/log/apache2/site1-access.log combined
>> </VirtualHost>
>>
>> # --- Site 2: site2.lab ---
>> <VirtualHost *:80>
>>     ServerName site2.lab
>>     ServerAlias www.site2.lab
>>     DocumentRoot /usr/local/apache2/htdocs/site2
>>
>>     <Directory /usr/local/apache2/htdocs/site2>
>>         AllowOverride None
>>         Require all granted
>>     </Directory>
>>
>>     ErrorLog /var/log/apache2/site2-error.log
>>     CustomLog /var/log/apache2/site2-access.log combined
>> </VirtualHost>
>> '@ | Set-Content apache/sites/vhosts.conf
PS C:\Users\ASUS\docker-lab\web-service>

```

Command diatasdigunakan untuk membuat file konfigurasi Apache Virtual Host (vhosts.conf) menggunakan Set-Content. Isi konfigurasinya mengatur 2 website (site1.lab dan site2.lab) beserta DocumentRoot, hak akses directory, dan file log masing-masing.

4. Buat Dockerfile Apache

```

PS C:\Users\ASUS\docker-lab\web-service> @'
>> FROM httpd:2.4-alpine
>>
>> # Aktifkan module yang dibutuhkan
>> RUN sed -i \
>>     -e 's/#LoadModule vhost_alias_module/LoadModule vhost_alias_module/' \
>>     -e 's/#LoadModule rewrite_module/LoadModule rewrite_module/' \
>>     /usr/local/apache2/conf/httpd.conf
>>
>> # Include virtual host config
>> RUN echo "Include conf/extra/vhosts.conf" >> /usr/local/apache2/conf/httpd.conf
>>
>> # Buat direktori log
>> RUN mkdir -p /var/log/apache2
>>
>> # Copy virtual host config
>> COPY sites/vhosts.conf /usr/local/apache2/conf/extra/vhosts.conf
>>
>> # Copy site files
>> COPY html-site1/ /usr/local/apache2/htdocs/site1/
>> COPY html-site2/ /usr/local/apache2/htdocs/site2/
>>
>> EXPOSE 80
>> '@ | Set-Content apache/Dockerfile
PS C:\Users\ASUS\docker-lab\web-service>

```

Command diatas digunakan untuk membuat file Dockerfile untuk container Apache berbasis image httpd:2.4-alpine. Konfigurasinya mengaktifkan module Apache, menambahkan virtual host, membuat folder log, menyalin file konfigurasi serta file website site1 dan site2, lalu membuka port 80 agar web dapat diakses.

Langkah 2: Generate Self-Signed SSL Certificate

2. Generate SSL certificate untuk *.lab (wildcard)

[illegible]

Command diatas digunakan untuk membuat sertifikat SSL self-signed dan private key menggunakan OpenSSL. Sertifikat berlaku selama 365 hari dengan domain *.lab, lalu disimpan sebagai server.crt dan server.key pada folder certs agar dapat digunakan untuk HTTPS.

3. Verifikasi certificate

```
PS C:\Users\ASUS\docker-lab\web-service> "C:\Program Files\Git\usr\bin\openssl.exe" x509 -in certs/server.crt -noout -text | Select-Object -First 20
Certificate:
    Data:
        Version: 3 (0x2)
        Serial Number:
            53:74:4c:97:9f:ae:00:24:9d:d2:ce:85:73:64:c2:44:c8:45:65:f5
        Signature Algorithm: sha256WithRSAEncryption
        Issuer: C=ID, ST=Jawa Timur, L=Surabaya, O=PENS Lab, CN=*.*.lab
        Validity
            Not Before: May 10 02:13:15 2026 GMT
            Not After : May 10 02:13:15 2027 GMT
        Subject: C=ID, ST=Jawa Timur, L=Surabaya, O=PENS Lab, CN=*.*.lab
        Subject Public Key Info:
            Public Key Algorithm: rsaEncryption
            Public-Key: (2048 bit)
            Modulus:
                00:d7:b9:6f:cd:39:be:8b:cb:5f:cb:ec:81:97:35:
                b1:22:73:ad:ae:75:53:b0:dc:bd:97:5f:af:98:fa:
                18:4b:b4:6f:a0:0b:a7:26:c0:af:5b:c6:6f:2e:b4:
                fe:f3:9e:a2:36:9f:db:80:fe:66:b7:f3:eb:3b:35:
                db:f3:1e:e4:86:3d:6f:ea:dc:2d:df:27:59:18:a3:
    PS C:\Users\ASUS\docker-lab\web-service>
```

Command diatas digunakan untuk menampilkan detail isi sertifikat SSL server.crt menggunakan OpenSSL. Output menampilkan informasi seperti versi sertifikat, issuer, masa berlaku, algoritma enkripsi, dan public key dari sertifikat yang telah dibuat.

4. Melihat daftar file di dalam folder certs

```
PS C:\Users\ASUS\docker-lab\web-service> ls certs

Directory: C:\Users\ASUS\docker-lab\web-service\certs


Mode                LastWriteTime         Length Name
----                -
-a----             10/05/2026     09.13         1367 server.crt
-a----             10/05/2026     09.13         1704 server.key

PS C:\Users\ASUS\docker-lab\web-service> |
```

Command `ls certs` digunakan untuk menampilkan isi folder `certs`. Output menunjukkan bahwa file sertifikat SSL `server.crt` dan private key `server.key` berhasil dibuat.

Langkah 3: Konfigurasi Nginx sebagai Reverse Proxy + SSL Termination

```
PS C:\Users\ASUS\docker-lab\web-service> @'
>> # =====
>> # Nginx Reverse Proxy + SSL Configuration
>> # =====
>>
>> # --- Upstream definitions ---
>> upstream apache_backend {
>>     server apache-web:80;
>> }
>>
>> upstream flask_backend {
>>     server flask-app:5000;
>> }
>>
>> # --- HTTP: Redirect ke HTTPS ---
>> server {
>>     listen 80;
>>     server_name site1.lab site2.lab app.lab;
>>     return 301 https://$host$request_uri;
>> }
>>
>> # --- HTTPS: site1.lab → Apache ---
>> server {
>>     listen 443 ssl;
>>     server_name site1.lab;
>>
>>     ssl_certificate      /etc/nginx/certs/server.crt;
>>     ssl_certificate_key  /etc/nginx/certs/server.key;
>>     ssl_protocols        TLSv1.2 TLSv1.3;
>>
>>     location / {
>>         proxy_pass http://apache_backend;
>>         proxy_set_header Host $host;
>>         proxy_set_header X-Real-IP $remote_addr;
>>         proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
>>         proxy_set_header X-Forwarded-Proto $scheme;
>>     }
>> }
>>
>> '
```



```

>>     access_log /var/log/nginx/site1-access.log;
>>     error_log /var/log/nginx/site1-error.log;
>> }
>>
>> # --- HTTPS: site2.lab → Apache ---
>> server {
>>     listen 443 ssl;
>>     server_name site2.lab;
>>
>>     ssl_certificate /etc/nginx/certs/server.crt;
>>     ssl_certificate_key /etc/nginx/certs/server.key;
>>     ssl_protocols TLSv1.2 TLSv1.3;
>>
>>     location / {
>>         proxy_pass http://apache_backend;
>>         proxy_set_header Host $host;
>>         proxy_set_header X-Real-IP $remote_addr;
>>         proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
>>         proxy_set_header X-Forwarded-Proto $scheme;
>>     }
>>
>>     access_log /var/log/nginx/site2-access.log;
>>     error_log /var/log/nginx/site2-error.log;
>> }
>>
>> # --- HTTPS: app.lab → Flask ---
>> server {
>>     listen 443 ssl;
>>     server_name app.lab;
>>
>>     ssl_certificate /etc/nginx/certs/server.crt;
>>     ssl_certificate_key /etc/nginx/certs/server.key;
>>     ssl_protocols TLSv1.2 TLSv1.3;
>>
>>     location / {
>>         proxy_pass http://flask_backend;
>>         proxy_set_header Host $host;
>>         proxy_set_header X-Real-IP $remote_addr;
>>         proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
>>
>>         proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
>>         proxy_set_header X-Forwarded-Proto $scheme;
>>     }
>>
>>     access_log /var/log/nginx/app-access.log;
>>     error_log /var/log/nginx/app-error.log;
>> }
>>
>> # --- Default: catch-all ---
>> server {
>>     listen 80 default_server;
>>     listen 443 ssl default_server;
>>
>>     ssl_certificate /etc/nginx/certs/server.crt;
>>     ssl_certificate_key /etc/nginx/certs/server.key;
>>
>>     return 444;
>> }
>> '@ | Set-Content nginx/default.conf
PS C:\Users\ASUS\docker-lab\web-service> |

```

Command tersebut digunakan untuk membuat file konfigurasi default.conf pada Nginx. Konfigurasi ini mengatur Nginx sebagai reverse proxy dengan SSL,

mengarahkan site1.lab dan site2.lab ke Apache, serta app.lab ke Flask. Selain itu, HTTP otomatis diarahkan ke HTTPS dan request tidak dikenal akan ditolak dengan kode 444.

Langkah 4: Buat Flask Backend App

1. Buat File flask/requirements.txt

```
PS C:\Users\ASUS\docker-lab\web-service> @'  
>> flask==3.1.*  
>> psycopg2-binary==2.9.*  
>> '@ | Set-Content flask/requirements.txt
```

Set-Content flask/requirements.txt digunakan untuk membuat file requirements.txt yang berisi daftar dependency Python, yaitu Flask dan PostgreSQL driver (psycopg2-binary).

2. Buat File flask/app.py

```
PS C:\Users\ASUS\docker-lab\web-service> @'  
>> import os, socket, datetime  
>> from flask import Flask, jsonify, request  
>> import psycopg2  
>>  
>> app = Flask(__name__)  
>>  
>> def get_db():  
>>     return psycopg2.connect(  
>>         host=os.environ.get("DB_HOST", "db"),  
>>         dbname=os.environ.get("DB_NAME", "labdb"),  
>>         user=os.environ.get("DB_USER", "labuser"),  
>>         password=os.environ.get("DB_PASS", "labpass123"))  
>>  
>> @app.route("/")  
>> def index():  
>>     return jsonify({  
>>         "service": "Flask Backend API",  
>>         "hostname": socket.gethostname(),  
>>         "timestamp": datetime.datetime.now().isoformat(),  
>>         "client_ip": request.headers.get("X-Real-IP", request.remote_addr),  
>>         "proto": request.headers.get("X-Forwarded-Proto", "http")  
>>     })  
>>  
>> @app.route("/api/health")  
>> def health():  
>>     result = {"status": "ok", "database": "unknown"}  
>>     try:  
>>         conn = get_db()  
>>         cur = conn.cursor()  
>>         cur.execute("SELECT version();")  
>>         result["database"] = cur.fetchone()[0]  
>>         result["db_status"] = "connected"  
>>         cur.close(); conn.close()  
>>     except Exception as e:  
>>         result["db_status"] = f"error: {e}"
```

```

>>     result["db_status"] = f"error: {e}"
>>     return jsonify(result)
>>
>> @app.route("/api/visitors", methods=["POST"])
>> def add_visitor():
>>     try:
>>         conn = get_db(); cur = conn.cursor()
>>         cur.execute("""
>>             CREATE TABLE IF NOT EXISTS visitors (
>>                 id SERIAL PRIMARY KEY,
>>                 name VARCHAR(100),
>>                 visited_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
>>             )""")
>>         name = request.json.get("name", "anonymous")
>>         cur.execute("INSERT INTO visitors (name) VALUES (%s) RETURNING id, visited_at")
>>         row = cur.fetchone()
>>         conn.commit(); cur.close(); conn.close()
>>         return jsonify({"id": row[0], "name": name, "visited_at": str(row[1])}), 201
>>     except Exception as e:
>>         return jsonify({"error": str(e)}), 500
>>
>> @app.route("/api/visitors", methods=["GET"])
>> def get_visitors():
>>     try:
>>         conn = get_db(); cur = conn.cursor()
>>         cur.execute("SELECT id, name, visited_at FROM visitors ORDER BY id DESC LIMIT 2")
>>         rows = [{"id": r[0], "name": r[1], "visited_at": str(r[2])} for r in cur.fetchall()]
>>         cur.close(); conn.close()
>>         return jsonify(rows)
>>     except Exception as e:
>>         return jsonify({"error": str(e)}), 500
>>
>> if __name__ == "__main__":
>>     app.run(host="0.0.0.0", port=5000)
>> '@ | Set-Content flask/app.py

```

Set-Content flask/[app.py](#) digunakan untuk membuat file aplikasi Flask yang menyediakan beberapa endpoint API, seperti pengecekan health database dan penyimpanan data visitor ke PostgreSQL.

3. Buat File flask/requirements.txt

```

PS C:\Users\ASUS\docker-lab\web-service> '@'
>> FROM python:3.11-slim
>> WORKDIR /app
>> COPY requirements.txt .
>> RUN pip install --no-cache-dir -r requirements.txt
>> COPY app.py .
>> EXPOSE 5000
>> CMD ["python", "app.py"]
>> '@ | Set-Content flask/Dockerfile
PS C:\Users\ASUS\docker-lab\web-service>

```

Set-Content flask/Dockerfile digunakan untuk membuat Dockerfile Flask berbasis Python 3.11, menginstall dependency dari requirements.txt, menyalin file aplikasi, membuka port 5000, lalu menjalankan app.py.

Langkah 5: Buat Docker Compose

```
PS C:\Users\ASUS\docker-lab\web-service> @'
>> services:
>> # --- Nginx Reverse Proxy + SSL Termination ---
>> proxy:
>>   image: nginx:alpine
>>   container_name: nginx-proxy
>>   ports:
>>     - "8080:80"
>>     - "8443:443"
>>   volumes:
>>     - ./nginx/default.conf:/etc/nginx/conf.d/default.conf:ro
>>     - ./certs:/etc/nginx/certs:ro
>>     - nginx-logs:/var/log/nginx
>>   networks:
>>     - web-net
>>   depends_on:
>>     - apache-web
>>     - flask-app
>>   restart: unless-stopped
>>
>> # --- Apache Web Server (Virtual Hosts) ---
>> apache-web:
>>   build: ./apache
>>   container_name: apache-web
>>   volumes:
>>     - apache-logs:/var/log/apache2
>>   networks:
>>     - web-net
>>   restart: unless-stopped
>>
>> # --- Flask Backend API ---
>> flask-app:
>>   build: ./flask
>>   container_name: flask-app
>>   environment:
>>     - DB_HOST=db
```

```
>> - DB_NAME=labdb
>> - DB_USER=labuser
>> - DB_PASS=labpass123
>> networks:
>> - web-net
>> - db-net
>> depends_on:
>> db:
>>   condition: service_healthy
>> restart: unless-stopped
>>
>> # --- PostgreSQL Database ---
>> db:
>>   image: postgres:16-alpine
>>   container_name: postgres-db
>>   environment:
>>     POSTGRES_DB: labdb
>>     POSTGRES_USER: labuser
>>     POSTGRES_PASSWORD: labpass123
>>   volumes:
>>     - pg-data:/var/lib/postgresql/data
>>   networks:
>>     - db-net
>>   healthcheck:
>>     test: ["CMD-SHELL", "pg_isready -U labuser -d labdb"]
>>     interval: 5s
>>     timeout: 5s
>>     retries: 5
>>     restart: unless-stopped
>>
>> volumes:
>>   pg-data:
>>   nginx-logs:
>>   apache-logs:
>>
>> networks:
>>   web-net:
>>   db-net:
>> '@ | Set-Content docker-compose.yml
PS C:\Users\ASUS\docker-lab\web-service> |
```

Command diatas digunakan untuk membuat file docker-compose.yml yang mengatur seluruh layanan Docker dalam satu project. Konfigurasinya menjalankan Nginx sebagai reverse proxy + SSL, Apache sebagai web server, Flask sebagai backend API, dan PostgreSQL sebagai database, lengkap dengan network, volume, dependency, dan restart policy masing-masing container.

```
PS C:\Users\ASUS\docker-lab\web-service> docker compose ps
NAME                IMAGE                COMMAND                SERVICE    CREATED        STATUS                PORTS
apache-web          web-service-apache-web  "httpd-foreground"    apache-web  12 minutes ago  Up 11 minutes        80/tcp
flask-app           web-service-flask-app   "python app.py"        flask-app   12 minutes ago  Up 11 minutes        5000/tcp
nginx-proxy         nginx:alpine          "/docker-entrypoint...." proxy       12 minutes ago  Up 11 minutes        0.0.0.0:8080->80/tcp, [::]:8080->80/tcp, 0.0.0.0:8443->443/tcp, [::]:8443->443/tcp
postgres-db         postgres:16-alpine     "docker-entrypoint.s..." db          12 minutes ago  Up 11 minutes (healthy)  5432/tcp
PS C:\Users\ASUS\docker-lab\web-service> |
```

Langkah 6: Deploy dan Testing

1. Tambahkan DNS lokal

```
PS C:\WINDOWS\system32> Add-Content -Path C:\Windows\System32\drivers\etc\hosts -Value
"n127.0.0.1 site1.lab site2.lab app.lab" -Force
PS C:\WINDOWS\system32>
```

Command diatas digunakan untuk menambahkan domain lokal site1.lab, site2.lab, dan app.lab ke file hosts Windows. Konfigurasi ini membuat ketiga domain tersebut mengarah ke 127.0.0.1 (localhost) agar bisa diakses dari browser secara lokal.

2. Build dan jalankan

```
PS C:\User> docker compose up --build -d
#1 [internal] load local bake definitions
#1 reading from stdin 1.05kB done
#1 DONE 0.0s

#2 [flask-app internal] load build definition from Dockerfile
#2 transferring dockerfile: 30B 0.0s
#2 transferring dockerfile: 201B 0.0s done
#2 DONE 0.1s

#3 [apache-web internal] load build definition from Dockerfile
#3 transferring dockerfile: 677B 0.0s done
#3 DONE 0.1s

#4 [flask-app internal] load metadata for docker.io/library/python:3.11-slim
#4 ...

#5 [auth] library/httpd:pull token for registry-1.docker.io
#5 DONE 0.0s

#6 [auth] library/python:pull token for registry-1.docker.io
#6 DONE 0.0s

#7 [apache-web internal] load metadata for docker.io/library/httpd:2.4-alpine
#7 ...

#8 [flask-app internal] load metadata for docker.io/library/python:3.11-slim
#8 DONE 2.1s

#8 [flask-app internal] load .dockerignore
#8 transferring context: 2B done
#8 DONE 0.0s

#9 [flask-app internal] load build context
#9 transferring context: 2.48kB done
#9 DONE 0.1s

#10 [flask-app 1/5] FROM docker.io/library/python:3.11-slim@sha256:a5b427ace4900267d93db34138e512325c6fa6af84ad5e4ed5f3b36258cc4142
#10 resolve docker.io/library/python:3.11-slim@sha256:a5b427ace4900267d93db34138e512325c6fa6af84ad5e4ed5f3b36258cc4142 0.1s done

#26 exporting manifest sha256:6d569e03de737024alcfe669658c4b4a178ad5373c738ecf3bfbc815a0bdb87 0.0s done
#26 exporting config sha256:bad722364bf5b8e1ecd7e031dc5dd8b73f4e249300f10186c7c125cf56d3394 0.0s done
#26 exporting attestation manifest sha256:0604bef8f9a298324c8c3fac611f19888e574f5d3db8fe45b28e7945ebfa202 0.0s done
#26 exporting manifest list sha256:40bbf118fbc320e5c5c8142bf409f34d1eff10425c181074cc23a49ee381c1771 0.0s done
#26 naming to docker.io/library/web-service-apache-web:latest
#26 naming to docker.io/library/web-service-apache-web:latest done
#26 unpacking to docker.io/library/web-service-apache-web:latest
#26 unpacking to docker.io/library/web-service-apache-web:latest 0.2s done
#26 DONE 0.7s

#27 [apache-web] resolving provenance for metadata file
#27 DONE 0.0s

[+] up 11/11
✔Image web-service-apache-web Built
✔Image web-service-flask-app Built
✔Network web-service_db-net Created
✔Network web-service_web-net Created
✔Volume web-service_nginx-logs Created
✔Volume web-service_apache-logs Created
✔Volume web-service_pg-data Created
✔Container apache-web Started
✔Container postgres-db Healthy
✔Container flask-app Started
✔Container nginx-proxy Started

What's next:
Filter, search, and stream logs from all your Compose services
in one place with Docker Desktop's Logs view. docker-desktop://dashboard/logs
```

Command docker compose up --build -d digunakan untuk membangun ulang image Docker lalu menjalankan seluruh container berdasarkan konfigurasi di docker-compose.yml. Opsi --build memaksa Docker melakukan build ulang image, sedangkan -d menjalankan container di background (detached mode).

3. Test Virtual Host Apache via Nginx Proxy

Test HTTP → HTTPS redirect

```
PS C:\Users\ASUS\docker-lab\web-service> curl http://site1.lab:8080 -Method Head -MaximumRedirection 0 -ErrorAction SilentlyContinue -UseBasicParsing |
Select-Object StatusCode, @{N="RedirectTo"; E={$_.Headers.Location}}

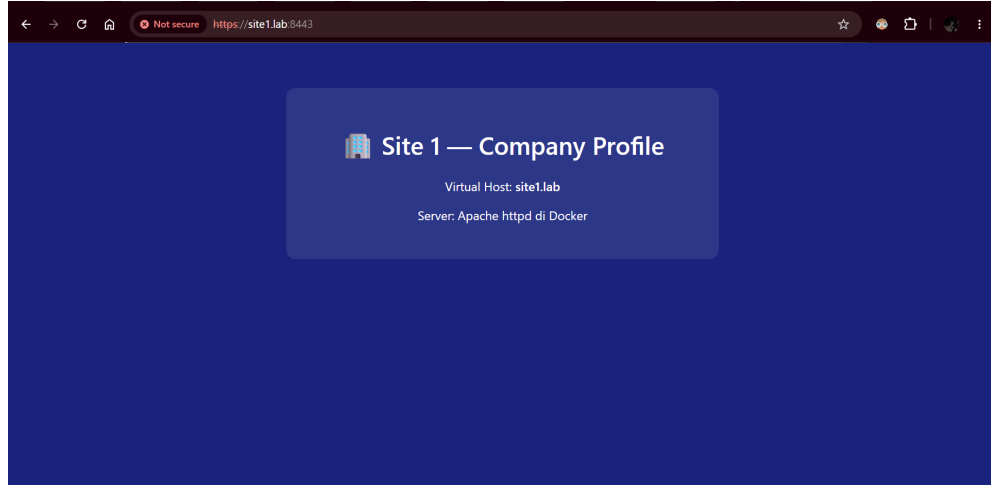
StatusCode RedirectTo
-----
301 https://site1.lab/

PS C:\Users\ASUS\docker-lab\web-service>
```

Command diatas digunakan untuk menguji apakah site1.lab melakukan redirect dari HTTP ke HTTPS. Hasil 301 menunjukkan redirect berhasil, sedangkan https://site1.lab/ pada kolom RedirectTo menunjukkan tujuan pengalihannya.

Test Site 1 (Apache via Nginx proxy, skip SSL verify)

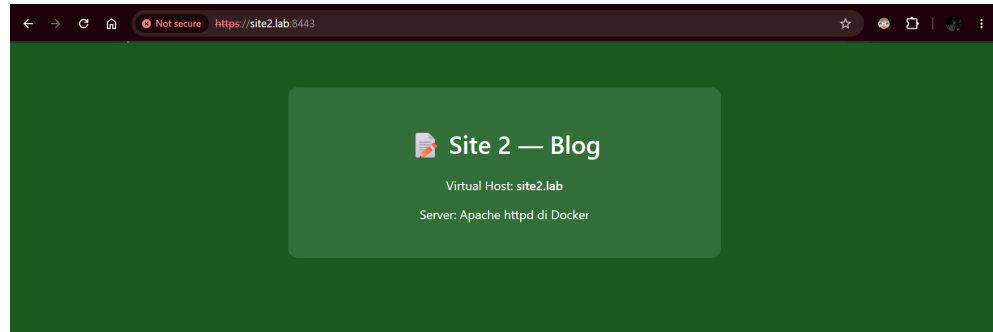
```
PS C:\Users\ASUS\docker-lab\web-service> docker exec nginx-proxy curl -I -H "Host: site1.lab" http://localhost
% Total % Received % Xferd Average Speed Time Time Time Current
Dload Upload Total Spent Left Speed
0 0 0 0 0 0 0 0 --:--:-- --:--:-- --:--:-- 0HTTP/1.1 301 Moved Permanently
Server: nginx/1.29.8
Date: Sun, 10 May 2026 04:06:00 GMT
Content-Type: text/html
Content-Length: 169
Connection: keep-alive
Location: https://site1.lab/
0 169 0 0 0 0 0 0 --:--:-- --:--:-- --:--:-- 0
PS C:\Users\ASUS\docker-lab\web-service>
```



Command `docker exec nginx-proxy curl -I -H "Host: site1.lab" http://localhost` digunakan untuk menguji konfigurasi virtual host dan redirect pada Nginx dari dalam container. Hasil 301 Moved Permanently menunjukkan request HTTP berhasil diarahkan ke HTTPS site1.lab. `https://site1.lab:8443` pada browser digunakan untuk mengakses website site1.lab melalui HTTPS pada port 8443. Halaman yang tampil menunjukkan konfigurasi Apache Virtual Host dan reverse proxy Nginx berhasil berjalan.

Test Site 2

```
PS C:\Users\ASUS\docker-lab\web-service> docker exec nginx-proxy curl -I -H "Host: site2.lab" http://localhost
% Total % Received % Xferd Average Speed Time Time Time Current
Dload Upload Total Spent Left Speed
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
Server: nginx/1.29.8
Date: Sun, 10 May 2026 04:07:27 GMT
Content-Type: text/html
Content-Length: 169
Connection: keep-alive
Location: https://site2.lab/
0 169 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
PS C:\Users\ASUS\docker-lab\web-service>
```



Command `docker exec nginx-proxy curl -I -H "Host: site2.lab" http://localhost` ini digunakan untuk menguji virtual host dan redirect HTTPS untuk domain `site2.lab` pada Nginx. Hasil 301 Moved Permanently menunjukkan request berhasil dialihkan ke `https://site2.lab/`. `https://site2.lab:8443` pada browser digunakan untuk mengakses website `site2.lab` melalui HTTPS pada port 443. Halaman yang tampil menunjukkan konfigurasi Apache Virtual Host untuk Site 2 berhasil berjalan melalui reverse proxy Nginx.

Test Flask API

```
PS C:\Users\ASUS\docker-lab\web-service> docker exec nginx-proxy curl -s http://flask-app:5000/
{"client_ip":"172.20.0.4","hostname":"af406ce3ecc","proto":"http","service":"Flask Backend API","timestamp":"2026-05-10T03:58:26.886570"}
PS C:\Users\ASUS\docker-lab\web-service> docker exec nginx-proxy curl -s http://flask-app:5000/api/health
{"database":"PostgreSQL 16.13 on x86_64-pc-linux-musl, compiled by gcc (Alpine 15.2.0) 15.2.0, 64-bit","db_status":"connected","status":"ok"}
PS C:\Users\ASUS\docker-lab\web-service>
```

Command `docker exec nginx-proxy curl -s http://flask-app:5000/` digunakan untuk menguji koneksi dari container nginx-proxy ke service Flask. Output JSON menampilkan informasi backend seperti hostname, timestamp, IP client, dan protocol. Sedangkan Command `docker exec nginx-proxy curl -s http://flask-app:5000/api/health` digunakan untuk mengecek status health API Flask dan koneksi database PostgreSQL. Output menunjukkan database berhasil terhubung dengan status "connected" dan service dalam kondisi "ok".

4. Test API CRUD

Tambah visitor


```

PS C:\Users\ASUS\docker-lab\web-service> $body = @{ name = "Mahasiswa PENS" } | ConvertTo-Json
PS C:\Users\ASUS\docker-lab\web-service> [Net.ServicePointManager]::SecurityProtocol = "Tls12"; [Net.ServicePointManager]::ServerCertificateValidationCallback = {$true}
PS C:\Users\ASUS\docker-lab\web-service> Invoke-RestMethod -Method Post -Uri "https://app.lab:8443/api/visitors" -Body $body -ContentType "application/json"

id name          visited_at
-- name          -
1 Mahasiswa PENS 2026-05-10 04:17:23.200450

PS C:\Users\ASUS\docker-lab\web-service> |

```

Command \$body = @{ name = "Mahasiswa PENS" } | ConvertTo-Json ini digunakan untuk membuat data JSON berisi nama visitor yang akan dikirim ke API. [Net.ServicePointManager]::SecurityProtocol = "Tls12" dan ServerCertificateValidationCallback = {\$true} digunakan untuk mengaktifkan TLS 1.2 dan mengabaikan validasi sertifikat SSL self-signed agar request HTTPS dapat dijalankan. Sedangkan Command Invoke-RestMethod -Method Post ... digunakan untuk mengirim request POST ke endpoint <https://app.lab:8443/api/visitors>. Output menunjukkan data visitor berhasil ditambahkan ke database PostgreSQL melalui Flask API.

Lihat daftar visitor

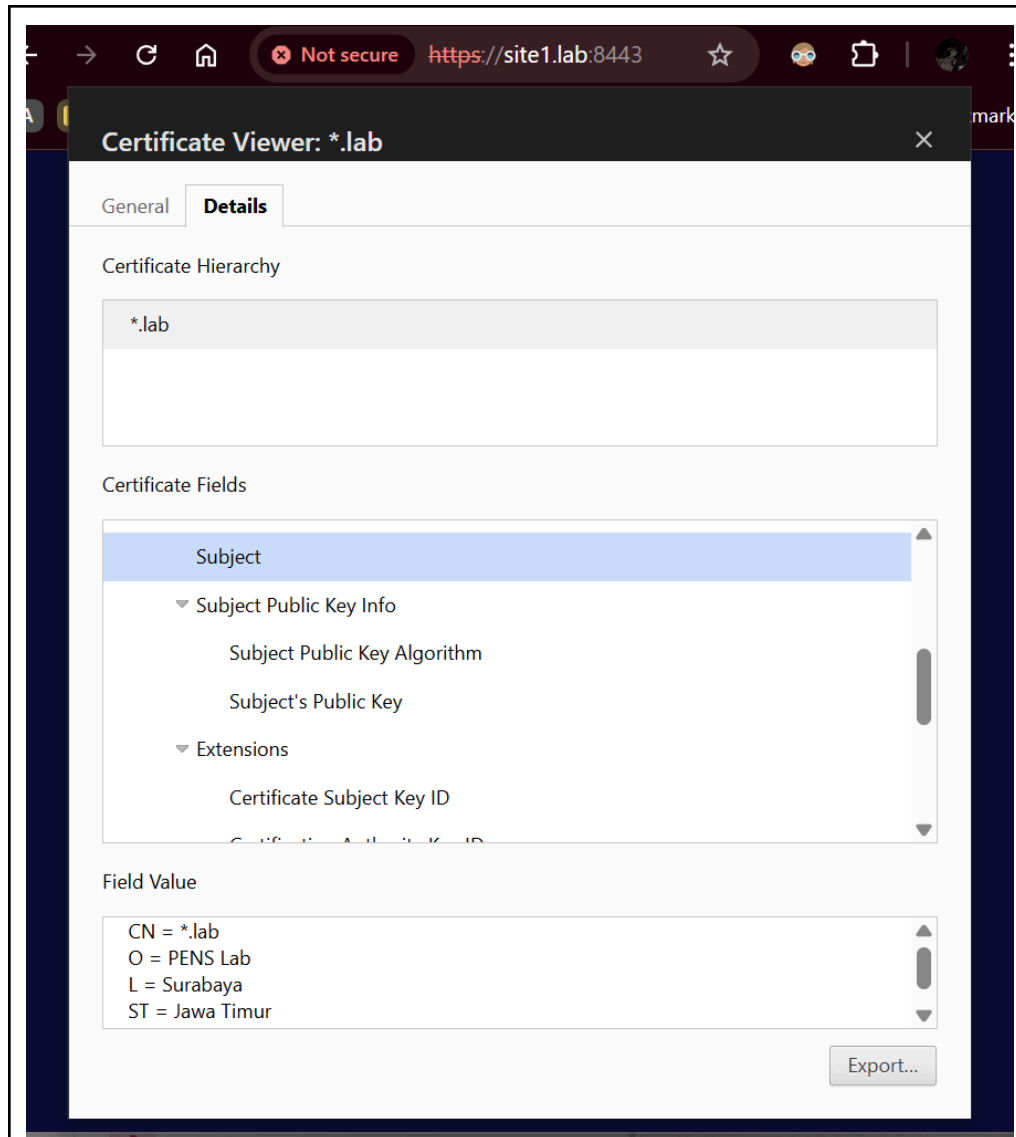
```

PS C:\Users\ASUS\docker-lab\web-service> docker exec nginx-proxy curl -k -L -H "Host: app.lab" http://localhost/api/visitors
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total     Spent    Left     Speed
100 169 100   169    0    0 62662    0 --:--:-- --:--:-- --:--:-- 84500
100  77 100    77    0    0  1454    0 --:--:-- --:--:-- --:--:--  1454
[{"id":1,"name":"Mahasiswa PENS","visited_at":"2026-05-10 04:17:23.200450"}]
PS C:\Users\ASUS\docker-lab\web-service> |

```

Command tersebut digunakan untuk mengakses endpoint app.lab/api/visitors melalui HTTPS dari dalam container Nginx. Opsi -k mengabaikan validasi SSL self-signed, sedangkan -L mengikuti redirect HTTP ke HTTPS. Output JSON menunjukkan data visitor berhasil diambil dari database PostgreSQL melalui Flask API.

5. Cek SSL Certificate



Buka <https://site1.lab:8443> di Chrome/Edge. Klik "Not Secure" (di kiri URL). Klik "Certificate is not valid". Adalah cara untuk memverifikasi apakah sertifikat yang terpasang di Nginx benar-benar sesuai dengan yang dibuat (memastikan domain, pembuat, dan masa berlakunya tepat).

6. Analisis Log

Log Nginx

```
PS C:\Users\ASUS\docker-lab\web-service> docker exec nginx-proxy cat /var/log/nginx/site1-access.log
172.20.0.1 - - [10/May/2026:03:42:27 +0000] "GET / HTTP/1.1" 200 550 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36"
172.20.0.1 - - [10/May/2026:03:42:27 +0000] "GET /favicon.ico HTTP/1.1" 404 236 "https://site1.lab:8443/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36"
172.20.0.1 - - [10/May/2026:03:49:14 +0000] "GET / HTTP/1.1" 200 550 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36"
172.20.0.1 - - [10/May/2026:03:52:53 +0000] "GET / HTTP/1.1" 200 550 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36"
172.20.0.1 - - [10/May/2026:03:52:55 +0000] "GET / HTTP/1.1" 200 550 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36"
172.20.0.1 - - [10/May/2026:03:52:55 +0000] "GET / HTTP/1.1" 200 550 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36"
172.20.0.1 - - [10/May/2026:03:52:56 +0000] "GET / HTTP/1.1" 200 550 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36"
172.20.0.1 - - [10/May/2026:03:53:36 +0000] "GET / HTTP/1.1" 200 550 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36"
172.20.0.1 - - [10/May/2026:03:53:45 +0000] "GET / HTTP/1.1" 200 550 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36"
172.20.0.1 - - [10/May/2026:04:07:04 +0000] "GET / HTTP/1.1" 200 550 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36"
172.20.0.1 - - [10/May/2026:04:07:05 +0000] "GET / HTTP/1.1" 200 550 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36"
172.20.0.1 - - [10/May/2026:04:07:05 +0000] "GET / HTTP/1.1" 200 550 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36"
172.20.0.1 - - [10/May/2026:04:07:06 +0000] "GET / HTTP/1.1" 200 550 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36"
172.20.0.1 - - [10/May/2026:04:07:10 +0000] "GET / HTTP/1.1" 400 657 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36"
172.20.0.1 - - [10/May/2026:04:07:10 +0000] "GET /favicon.ico HTTP/1.1" 400 657 "http://site1.lab:8443/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36"
172.20.0.1 - - [10/May/2026:04:18:15 +0000] "POST /api/visitors HTTP/1.1" 201 75 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36"
172.20.0.1 - - [10/May/2026:04:18:15 +0000] "GET /api/visitors HTTP/1.1" 200 77 "-" "curl/8.17.0"
172.20.0.1 - - [10/May/2026:04:18:37 +0000] "GET /api/visitors HTTP/1.1" 200 77 "-" "curl/8.17.0"

PS C:\Users\ASUS\docker-lab\web-service> docker exec nginx-proxy cat /var/log/nginx/app-access.log
172.20.0.1 - - [10/May/2026:03:49:52 +0000] "GET /api/health HTTP/1.1" 200 142 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36"
172.20.0.1 - - [10/May/2026:03:49:52 +0000] "GET /favicon.ico HTTP/1.1" 404 287 "https://app.lab:8443/api/health" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36"
172.0.0.1 - - [10/May/2026:04:18:05 +0000] "POST /api/visitors HTTP/1.1" 500 100 "-" "curl/8.17.0"
172.20.0.1 - - [10/May/2026:04:17:23 +0000] "POST /api/visitors HTTP/1.1" 201 75 "-" "Mozilla/5.0 (Windows NT 10.0; id-ID) WindowsPowerShell/5.1.26100.8115"
172.20.0.1 - - [10/May/2026:04:18:15 +0000] "GET /api/visitors HTTP/1.1" 200 77 "-" "Mozilla/5.0 (Windows NT 10.0; id-ID) WindowsPowerShell/5.1.26100.8115"
172.20.0.1 - - [10/May/2026:04:18:37 +0000] "GET /api/visitors HTTP/1.1" 200 77 "-" "curl/8.17.0"
```

Command `docker exec nginx-proxy cat /var/log/nginx/site1-access.log` digunakan untuk melihat isi access log Nginx untuk site1.lab. Log menampilkan riwayat request client seperti alamat IP, method HTTP, status response, dan browser yang digunakan. Sedangkan, Command `docker exec nginx-proxy cat /var/log/nginx/app-access.log` digunakan untuk melihat access log Nginx pada service app.lab. Log menunjukkan request API seperti `/api/health` dan `/api/visitors` yang berhasil diproses melalui reverse proxy Nginx.

Log Apache

```
PS C:\Users\ASUS\docker-lab\web-service> docker exec apache-web cat /var/log/apache2/site1-access.log
172.20.0.4 - - [10/May/2026:03:42:27 +0000] "GET / HTTP/1.1" 200 550 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36"
172.20.0.4 - - [10/May/2026:03:42:27 +0000] "GET /favicon.ico HTTP/1.1" 404 236 "https://site1.lab:8443/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36"
172.20.0.4 - - [10/May/2026:03:49:14 +0000] "GET / HTTP/1.1" 200 550 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36"
172.20.0.4 - - [10/May/2026:03:52:53 +0000] "GET / HTTP/1.1" 200 550 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36"
172.20.0.4 - - [10/May/2026:03:52:55 +0000] "GET / HTTP/1.1" 200 550 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36"
172.20.0.4 - - [10/May/2026:03:52:55 +0000] "GET / HTTP/1.1" 200 550 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36"
172.20.0.4 - - [10/May/2026:03:52:56 +0000] "GET / HTTP/1.1" 200 550 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36"
172.20.0.4 - - [10/May/2026:03:53:36 +0000] "GET / HTTP/1.1" 200 550 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36"
172.20.0.4 - - [10/May/2026:03:53:45 +0000] "GET / HTTP/1.1" 200 550 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36"
172.20.0.4 - - [10/May/2026:04:07:04 +0000] "GET / HTTP/1.1" 200 550 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36"
172.20.0.4 - - [10/May/2026:04:07:04 +0000] "GET / HTTP/1.1" 200 550 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36"
172.20.0.4 - - [10/May/2026:04:07:05 +0000] "GET / HTTP/1.1" 200 550 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36"
172.20.0.4 - - [10/May/2026:04:07:05 +0000] "GET / HTTP/1.1" 200 550 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36"
```

Command `docker exec apache-web cat /var/log/apache2/site1-access.log` digunakan untuk melihat isi access log Apache pada virtual host site1.lab. Log menampilkan detail request yang diteruskan dari Nginx ke Apache, seperti IP container proxy, method HTTP, status response, dan informasi browser client.

Atau akses via volume

```
PS C:\Users\ASUS\docker-lab\web-service> docker run --rm -v web-service_nginx-logs:/logs alpine ls /logs
access.log
app-access.log
app-error.log
error.log
site1-access.log
site1-error.log
site2-access.log
site2-error.log
PS C:\Users\ASUS\docker-lab\web-service> |
```

Command diatas digunakan untuk menjalankan container sementara (--rm) guna melihat isi volume Docker web-service_nginx-logs. Output menunjukkan file-file log Nginx berhasil tersimpan, seperti access log dan error log untuk site1, site2, serta app.lab.

E. PERTANYAAN

Pre-Lab

1. Apa keuntungan menjalankan web server di container dibandingkan langsung di host?
 - Isolasi Dependensi: Container memungkinkan setiap web server memiliki versi pustaka (library) dan dependensi yang berbeda tanpa terjadi konflik satu sama lain atau dengan sistem host.
 - Reprodusibilitas: Menjamin bahwa environment pengembangan, pengujian, dan produksi identik, sehingga meminimalisir masalah "it works on my machine".
 - Efisiensi dan Skalabilitas: Container memiliki overhead yang jauh lebih rendah dibanding Virtual Machine (VM) karena berbagi kernel host, namun tetap memberikan manajemen sumber daya (CPU/RAM) yang terukur.
2. Jelaskan perbedaan document root Apache (/usr/local/apache2/htdocs/) vs Nginx (/usr/share/nginx/html/).

Secara fungsional keduanya adalah direktori tempat server mencari file untuk disajikan ke klien. Perbedaannya terletak pada standar konvensi image resminya:

 - Apache (/usr/local/apache2/htdocs/): Jalur default pada image Docker httpd yang berbasis pada struktur instalasi sumber (source installation) Apache.
 - Nginx (/usr/share/nginx/html/): Jalur default yang mengikuti standar hirarki sistem file Linux (FHS) pada distribusi seperti Ubuntu atau Alpine yang digunakan oleh image Docker nginx.
3. Apa itu SSL Termination dan mengapa dilakukan di reverse proxy?

SSL Termination adalah proses enkripsi dan dekripsi trafik TLS/SSL yang dilakukan di lapisan Reverse Proxy (Nginx), sehingga trafik yang diteruskan ke server backend adalah trafik HTTP biasa. Alasannya; mengurangi beban komputasi pada server aplikasi (backend) untuk proses kriptografi, memudahkan manajemen sertifikat (terpusat di satu titik), dan memungkinkan Load Balancer untuk memeriksa konten trafik (seperti header HTTP) sebelum diteruskan.

4. Apa perbedaan name-based dan IP-based virtual hosting?
 - Name-based Virtual Hosting: Menggunakan header Host pada request HTTP untuk membedakan antar situs. Memungkinkan satu alamat IP melayani banyak nama domain (misal: site1.lab dan site2.lab).
 - IP-based Virtual Hosting: Setiap situs web dikaitkan dengan alamat IP yang berbeda pada satu server fisik. Server menentukan situs mana yang akan disajikan berdasarkan alamat IP tujuan koneksi.
5. Mengapa self-signed certificate menghasilkan warning di browser?

Browser memiliki daftar Trusted Root Certification Authorities (CA) yang sah secara global.

 - Self-signed certificate dibuat dan ditandatangani oleh entitas itu sendiri, bukan oleh CA otoritas yang dikenal.
 - Karena tidak ada pihak ketiga terpercaya yang memvalidasi identitas pemilik sertifikat, browser memberikan peringatan keamanan kepada pengguna bahwa keaslian identitas server tidak dapat diverifikasi secara otomatis.

Post-Lab

1. Bandingkan response header dari Apache vs Nginx. Header apa yang menunjukkan software web server?

Header yang menunjukkan software web server adalah Server.

 - Nginx Proxy: Akan menampilkan Server: nginx/1.29.x.
 - Apache Backend: Secara default menampilkan Server: Apache/2.4.x (Unix).
 - Analisis: Saat user mengakses melalui Proxy, browser hanya akan melihat header Server: nginx. Ini karena Nginx bertindak sebagai "wajah" terdepan yang menutupi identitas asli server di belakangnya (Security by Obscurity).
2. Jika Nginx proxy down, apakah Apache masih bisa diakses langsung? Bagaimana cara testnya?

Secara desain, jika Nginx down, user dari internet/host tidak bisa mengakses Apache melalui domain site1.lab karena port 80/443 yang terbuka ke publik ada di container Nginx.

- Apakah masih bisa diakses? Secara internal (dalam network Docker) bisa, namun dari luar (host) hanya bisa jika port Apache (81/82) dipetakan (mapped) ke host di docker-compose.yml.
- Cara Test:
 - Matikan Nginx: `docker stop nginx-proxy`.
 - Coba akses `https://site1.lab:8443` -> Pasti gagal (Connection Refused).
 - Lakukan `docker exec -it flask-app curl http://apache-web` -> Masih bisa diakses secara internal karena network antar-backend masih aktif.

3. Tunjukkan bahwa X-Real-IP header diteruskan dengan benar dari Nginx ke Flask.

Untuk memastikan server backend mengetahui alamat IP asli dari klien (bukan IP internal Nginx), digunakan mekanisme Header Forwarding.

- Bukti Implementasi: Pada konfigurasi Nginx, instruksi `proxy_set_header X-Real-IP $remote_addr;` digunakan.
- Verifikasi: Hal ini dapat dibuktikan dengan memeriksa log pada container Flask (`docker logs flask-app`). Di sana tercatat IP klien asli (misal: 172.20.0.1) pada objek request header, menunjukkan bahwa informasi identitas pengirim telah diteruskan secara akurat melalui proxy

4. Jelaskan mengapa Flask app perlu terhubung ke dua network (web-net dan db-net)

Aplikasi Flask dihubungkan ke dua network yang berbeda untuk menerapkan prinsip Isolasi Jaringan dan Keamanan Berlapis:

- web-net (Frontend Network): Digunakan khusus untuk komunikasi antara Nginx Proxy dan Flask App agar permintaan dari luar dapat diterima.
- db-net (Backend Network): Digunakan khusus untuk jalur komunikasi privat antara Flask App dan PostgreSQL.
- Tujuan: Strategi ini mencegah Nginx Proxy memiliki akses langsung ke database. Dengan demikian, jika lapisan proxy berhasil ditembus, penyerang tidak memiliki jalur langsung ke server database karena perbedaan segmen jaringan.

5. Apa yang terjadi jika file `server.key` atau `server.crt` dihapus saat container running?
- Kondisi Running: Jika file dihapus saat container sedang berjalan, layanan HTTPS tidak akan langsung terganggu karena Nginx telah memuat (load) konten sertifikat tersebut ke dalam memori (RAM) saat proses start-up.
 - Kondisi Restart/Reload: Gangguan fatal akan terjadi saat Nginx melakukan restart atau reload konfigurasi. Nginx akan gagal melakukan booting dan memunculkan error `'BIO_new_file... no such file or directory'`. Hal ini dikarenakan Nginx memerlukan verifikasi fisik file kunci tersebut pada disk untuk menginisialisasi protokol SSL/TLS.

Modul 4

Database Service di Docker — PostgreSQL



A. TUJUAN PEMBELAJARAN

Setelah praktikum ini, mahasiswa mampu:

1. Men-deploy PostgreSQL sebagai container Docker dengan konfigurasi production-grade
2. Melakukan inisialisasi database otomatis menggunakan init script
3. Mengelola data persistent PostgreSQL dengan Docker Volume
4. Melakukan backup dan restore database di container (pg_dump / pg_restore)
5. Mengkonfigurasi PostgreSQL: authentication (pg_hba.conf), tuning (postgresql.conf)
6. Menggunakan pgAdmin4 sebagai GUI database management dalam container
7. Membuat skema database, tabel, index, dan menjalankan query SQL
8. Menerapkan health check dan monitoring dasar PostgreSQL

B. DASAR TEORI

2.1 PostgreSQL Overview

PostgreSQL adalah Relational Database Management System (RDBMS) open-source yang dikenal dengan keandalan, fitur lengkap, dan compliance terhadap standar SQL. PostgreSQL mendukung ACID transactions, JSON/JSONB, full-text search, partitioning, dan extensibility melalui extension.

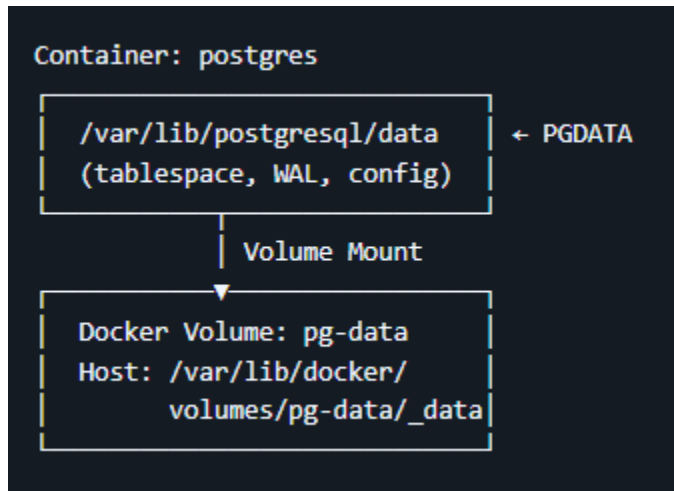
2.2 PostgreSQL di Docker

Image resmi postgres di Docker Hub menyediakan mekanisme konfigurasi melalui environment variable dan init script:

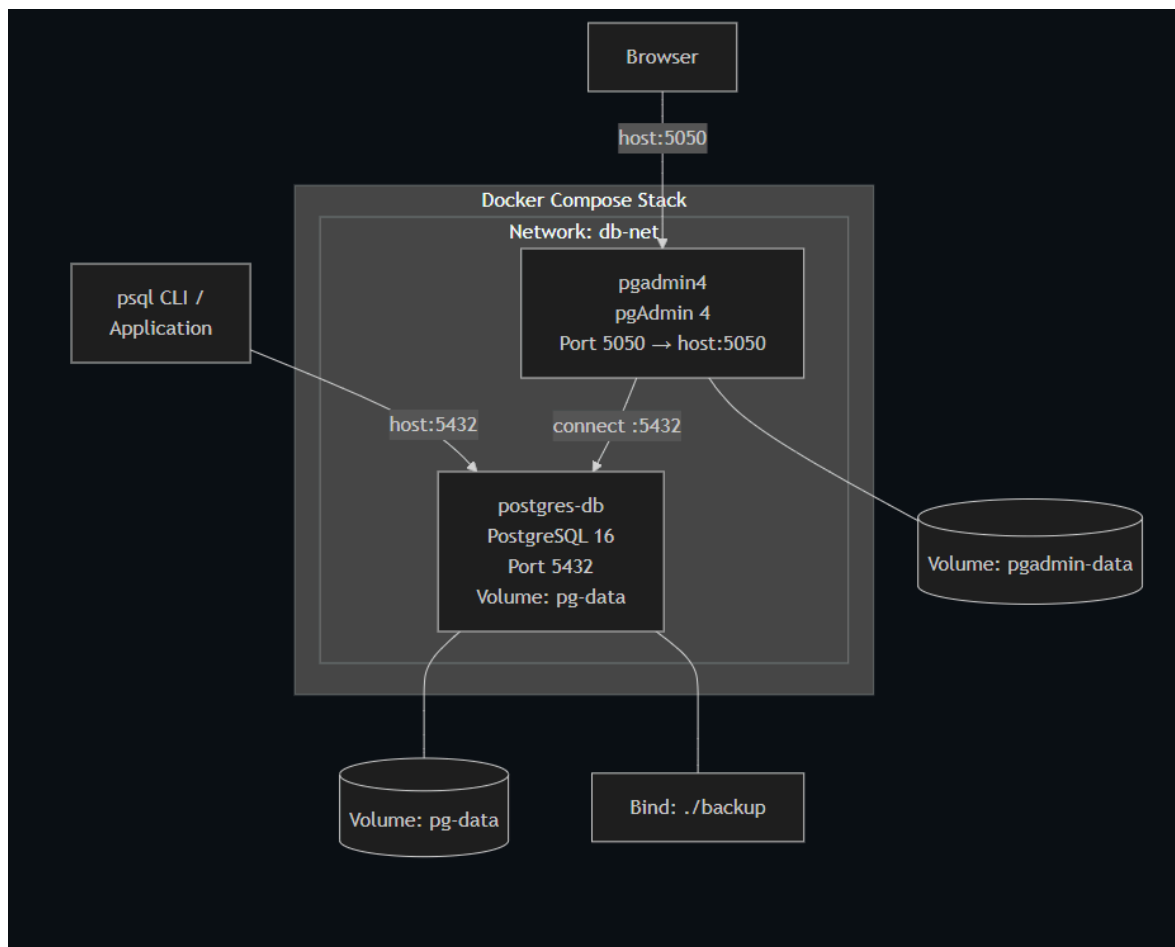
Environment Variable	Fungsi
POSTGRES_DB	Nama database yang dibuat saat pertama kali start
POSTGRES_USER	Username superuser (default: postgres)
POSTGRES_PASSWORD	Password superuser (wajib)
POSTGRES_INITDB_ARGS	Argumen tambahan untuk initdb
PGDATA	Lokasi data directory (default: /var/lib/postgresql/data)

Init Script: File .sql atau .sh yang ditempatkan di /docker-entrypoint-initdb.d/ akan dieksekusi secara otomatis hanya saat pertama kali container dibuat (volume kosong).

2.3 Data Persistence



C. TOPOLOGI LAB



D. LANGKAH PRAKTIKUM

Langkah 0: Persiapan Project

```
dito@LOG-JUWADI:~$ mkdir -p ~/docker-lab/postgresql/{init,config,backup}
cd ~/docker-lab/postgresql
dito@LOG-JUWADI:~/docker-lab/postgresql$
```

Perintah `mkdir -p ~/docker-lab/postgresql/{init,config,backup}` digunakan untuk membuat struktur folder project PostgreSQL berbasis Docker. Folder `init` digunakan untuk menyimpan script SQL inialisasi database yang otomatis dijalankan saat container pertama kali dibuat. Folder `config` digunakan untuk menyimpan konfigurasi custom PostgreSQL. Folder `backup` digunakan untuk menyimpan file hasil backup database. Opsi `-p` memastikan seluruh folder parent dibuat otomatis jika belum ada. Setelah itu, perintah `cd ~/docker-lab/postgresql` digunakan untuk masuk ke direktori project agar semua file konfigurasi dan compose dibuat di lokasi yang benar.

Langkah 1: Deploy PostgreSQL dengan Docker Compose

1. Buat init script (dijalankan saat pertama kali)

```
dito@LOG-JUWADI:~$ mkdir -p ~/docker-lab/postgresql/{init,config,backup}
cd ~/docker-lab/postgresql
dito@LOG-JUWADI:~/docker-lab/postgresql$ cat > init/01-create-schema.sql << 'EOF'
>
-- -- =====
-- Init Script: Database Schema untuk Lab PENS
-- Dijalankan otomatis saat container pertama kali start
-- =====
--
-- Buat database tambahan
CREATE DATABASE inventory_db;
--
-- Gunakan database utama (labdb sudah dibuat via env)
\c labdb
--
-- Buat schema
CREATE SCHEMA IF NOT EXISTS app;
--
-- Tabel: Mahasiswa
CREATE TABLE app.mahasiswa (
    id SERIAL PRIMARY KEY,
    nrp VARCHAR(15) UNIQUE NOT NULL,
    nama VARCHAR(100) NOT NULL,
    kelas CHAR(1) CHECK (kelas IN ('A', 'B', 'C', 'D')),
    kelompok INTEGER CHECK (kelompok BETWEEN 1 AND 10),
    email VARCHAR(100),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
--
-- Tabel: Mata Kuliah
CREATE TABLE app.matakuliah (
    id SERIAL PRIMARY KEY,
    kode VARCHAR(10) UNIQUE NOT NULL,
    nama VARCHAR(100) NOT NULL,
    sks INTEGER CHECK (sks BETWEEN 1 AND 6)
);
--
-- Tabel: Nilai (relasi many-to-many)
CREATE TABLE app.nilai (
    id SERIAL PRIMARY KEY,
    mahasiswa_id INTEGER REFERENCES app.mahasiswa(id) ON DELETE CASCADE,
    matakuliah_id INTEGER REFERENCES app.matakuliah(id) ON DELETE CASCADE,
    nilai_angka NUMERIC(5,2) CHECK (nilai_angka BETWEEN 0 AND 100),
    grade CHAR(2),
    semester VARCHAR(10),
    UNIQUE(mahasiswa_id, matakuliah_id, semester)
);
--
-- Tabel: Log Aktivitas (untuk Modul 5 logging)
CREATE TABLE app.activity_log (
    id BIGSERIAL PRIMARY KEY,
```

```

timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
level VARCHAR(10) DEFAULT 'INFO',
source VARCHAR(50),
message TEXT,
metadata JSONB
);

-- Index untuk performa query
CREATE INDEX idx_mahasiswa_kelas ON app.mahasiswa(kelas);
CREATE INDEX idx_mahasiswa_nrp ON app.mahasiswa(nrp);
CREATE INDEX idx_nilai_semester ON app.nilai(semester);
CREATE INDEX idx_activity_log_timestamp ON app.activity_log(timestamp);
CREATE INDEX idx_activity_log_level ON app.activity_log(level);
CREATE INDEX idx_activity_log_metadata ON app.activity_log USING GIN(metadata);

-- Insert sample data
INSERT INTO app.matakuliah (kode, nama, sks) VALUES
('JAR01', 'Administrasi Jaringan', 3),
('SBD01', 'Sistem Basis Data', 3),
('SO01', 'Sistem Operasi', 2),
('WEB01', 'Pemrograman Web', 3);

INSERT INTO app.mahasiswa (nrp, nama, kelas, kelompok, email) VALUES
('3122600001', 'Ahmad Fauzi', 'A', 1, 'ahmad@student.pens.ac.id'),
('3122600002', 'Budi Santoso', 'A', 1, 'budi@student.pens.ac.id'),
('3122600003', 'Citra Dewi', 'B', 2, 'citra@student.pens.ac.id'),
('3122600004', 'Dian Pratama', 'B', 2, 'dian@student.pens.ac.id'),
('3122600005', 'Eka Putra', 'C', 3, 'eka@student.pens.ac.id');

INSERT INTO app.nilai (mahasiswa_id, matakuliah_id, nilai_angka, grade, semester) VALUES
(1, 1, 85.50, 'A', '2025-1'),
(1, 2, 78.00, 'B+', '2025-1'),
(2, 1, 92.00, 'A', '2025-1'),
(3, 1, 70.25, 'B', '2025-1'),
(4, 3, 88.75, 'A', '2025-1');

-- Buat read-only user untuk aplikasi
CREATE USER app_reader WITH PASSWORD 'reader123';
GRANT USAGE ON SCHEMA app TO app_reader;
GRANT SELECT ON ALL TABLES IN SCHEMA app TO app_reader;
ALTER DEFAULT PRIVILEGES IN SCHEMA app GRANT SELECT ON TABLES TO app_reader;

RAISE NOTICE 'Database initialization completed successfully!';
> EOF

```

Perintah `cat > init/01-create-schema.sql << 'EOF'` digunakan untuk membuat file SQL bernama `01-create-schema.sql` di folder `init`. File ini akan otomatis dijalankan oleh image PostgreSQL saat container pertama kali melakukan inisialisasi database. Script tersebut membuat database tambahan `inventory_db`, lalu berpindah ke database utama `labdb` menggunakan `\c labdb`. Setelah itu dibuat schema `app` untuk memisahkan tabel aplikasi dari schema default PostgreSQL. Script kemudian membuat tabel mahasiswa, matakuliah, nilai, dan activity_log lengkap dengan primary key, foreign key, constraint validasi data, dan relasi antar tabel. Index juga dibuat untuk meningkatkan performa query pencarian dan filtering. Selanjutnya dimasukkan sample data agar database langsung memiliki isi awal untuk pengujian. Di bagian akhir dibuat user `app_reader` dengan hak akses read-only sehingga aplikasi tertentu hanya dapat membaca data tanpa melakukan perubahan. Statement `RAISE NOTICE` digunakan untuk menampilkan pesan bahwa proses inisialisasi selesai dengan sukses.

2. Buat custom PostgreSQL config

```
ditto@LOG-JUWADI:~/docker-lab/postgresql$ cat > config/custom-postgresql.conf << 'EOF'
> # =====
> # Custom PostgreSQL Configuration untuk Lab
> # =====
>
> # Connection
listen_addresses = '*'
max_connections = 50
>
> # Memory (sesuaikan untuk container dengan RAM terbatas)
shared_buffers = 128MB
work_mem = 4MB
maintenance_work_mem = 64MB
effective_cache_size = 256MB
>
> # WAL & Checkpoint
wal_level = replica
max_wal_size = 256MB
min_wal_size = 64MB
>
> # Logging
logging_collector = on
log_directory = '/var/log/postgresql'
log_filename = 'postgresql-%Y-%m-%d.log'
log_statement = 'mod'
log_min_duration_statement = 1000
log_connections = on
log_disconnections = on
log_line_prefix = '%t [%p] %u@%d '
>
> # Locale & Timezone
timezone = 'Asia/Jakarta'
log_timezone = 'Asia/Jakarta'
> EOF
```

Perintah `cat > config/custom-postgresql.conf << 'EOF'` digunakan untuk membuat file konfigurasi PostgreSQL custom. Konfigurasi `listen_addresses = '*'` membuat PostgreSQL menerima koneksi dari semua alamat jaringan. `max_connections = 50` membatasi jumlah koneksi aktif agar resource server tetap stabil. Parameter memory seperti `shared_buffers`, `work_mem`, `maintenance_work_mem`, dan `effective_cache_size` digunakan untuk mengatur penggunaan RAM PostgreSQL agar tetap ringan pada container Docker dengan resource terbatas. Konfigurasi WAL dan checkpoint mengatur mekanisme write-ahead logging agar proses recovery dan replikasi berjalan baik. Bagian logging mengaktifkan pencatatan query, koneksi, dan query lambat sehingga memudahkan monitoring dan troubleshooting. Pengaturan timezone memastikan seluruh timestamp database menggunakan zona waktu Asia/Jakarta.

3. Buat Docker Compose

```

dito@LOG-JUWADI:~/docker-lab/postgresql$ cat > docker-compose.yml << 'EOF'
>
> services:
# --- PostgreSQL 16 ---
db:
  image: postgres:16-alpine
  container_name: postgres-db
  environment:
    POSTGRES_DB: labdb
    POSTGRES_USER: labuser
    POSTGRES_PASSWORD: labpass123
    TZ: Asia/Jakarta
  ports:
    - "5432:5432"
  volumes:
    - pg-data:/var/lib/postgresql/data
    - ./init:/docker-entrypoint-initdb.d:ro
    - ./config/custom-postgresql.conf:/etc/postgresql/custom.conf:ro
    - ./backup:/backup
    - pg-logs:/var/log/postgresql
  command: >
    postgres
    -c config_file=/etc/postgresql/custom.conf
    -c hba_file=/var/lib/postgresql/data/pg_hba.conf
  networks:
    - db-net
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U labuser -d labdb"]
    interval: 10s
    timeout: 5s
    retries: 5
    restart: unless-stopped

# --- pgAdmin 4 (GUI) ---
pgadmin:
  image: dpage/pgadmin4:latest
  container_name: pgadmin4
  environment:
    PGADMIN_DEFAULT_EMAIL: admin@pens.ac.id
    PGADMIN_DEFAULT_PASSWORD: admin123
    PGADMIN_LISTEN_PORT: 5050
  ports:
    - "5050:5050"
  volumes:
    - pgadmin-data:/var/lib/pgadmin
  networks:
    - db-net
  depends_on:
    db:
      condition: service_healthy
  restart: unless-stopped

```

Perintah `cat > docker-compose.yml << 'EOF'` digunakan untuk membuat file Docker Compose yang mendefinisikan layanan PostgreSQL dan pgAdmin. Service db menggunakan image `postgres:16-alpine` karena ringan dan stabil. Environment variable digunakan untuk menentukan nama database, username, password, dan timezone. Mapping port `5432:5432` memungkinkan host mengakses PostgreSQL container melalui localhost. Volume `pg-data` digunakan untuk menyimpan data database secara persisten agar tidak hilang saat container dihapus. Folder `init` dimount agar script SQL otomatis dijalankan. File konfigurasi `custom` juga dimount ke dalam

container. Folder backup dan volume pg-logs digunakan untuk penyimpanan backup dan log. Perintah command memaksa PostgreSQL menggunakan file konfigurasi custom. Healthcheck digunakan untuk mengecek apakah database siap menerima koneksi. Restart unless-stopped memastikan container otomatis aktif kembali setelah reboot. Service pgadmin digunakan sebagai GUI administrasi database berbasis web. pgAdmin dihubungkan ke network yang sama agar dapat mengakses service db menggunakan hostname db.

4. Deploy

```

dito@LOG-JUWADI:~/docker-lab/postgresql$ docker compose up -d
[+] up 23/23
  ✓Image d... Pulled 77.1s
  ✓Network... Created 0.2s
  ✓Volume ... Created 0.1s
  ✓Volume ... Created 0.0s
  ✓Volume ... Created 0.1s

dito@LOG-JUWADI:~/docker-lab/postgresql$ docker compose ps
NAME                IMAGE              COMMAND                  SERVICE    CREATED
pgadmin4            dpape/pgadmin4:latest  "/entrypoint.sh"        pgadmin    About a minute
ago Up About a minute  0.0.0.0:5050->5050/tcp, [::]:5050->5050/tcp
postgres-lab        postgres:16-alpine    "docker-entrypoint.s..." db          About a minute
ago Up About a minute (healthy)  0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp

dito@LOG-JUWADI:~/docker-lab/postgresql$ docker compose logs db | tail -20
postgres-lab | 2026-05-10 14:59:46 WIB [29] @ LOG:  invalid record length at 0/1D695A8:
expected at least 24, got 0
postgres-lab | 2026-05-10 14:59:46 WIB [29] @ LOG:  redo done at 0/1D69540 system usage
: CPU: user: 0.00 s, system: 0.01 s, elapsed: 0.03 s
postgres-lab | 2026-05-10 14:59:46 WIB [27] @ LOG:  checkpoint starting: end-of-recover
y immediate wait
postgres-lab | 2026-05-10 14:59:46 WIB [27] @ LOG:  checkpoint complete: wrote 1905 buf
fers (11.6%); 0 WAL file(s) added, 0 removed, 0 recycled; write=0.048 s, sync=0.115 s, t
otal=0.171 s; sync files=627, longest=0.003 s, average=0.001 s; distance=8667 kB, estima
te=8667 kB; lsn=0/1D695A8, redo lsn=0/1D695A8
postgres-lab | 2026-05-10 14:59:46 WIB [1] @ LOG:  database system is ready to accept c
onnections
postgres-lab | 2026-05-10 14:59:56 WIB [39] [unknown]@[unknown] LOG:  connection receiv
ed: host=[local]
postgres-lab | 2026-05-10 14:59:56 WIB [39] labuser@labdb LOG:  connection authorized:
user=labuser database=labdb application_name=pg_isready
postgres-lab | 2026-05-10 14:59:56 WIB [39] labuser@labdb LOG:  disconnection: session
time: 0:00:00.012 user=labuser database=labdb host=[local]
postgres-lab | 2026-05-10 15:00:06 WIB [46] [unknown]@[unknown] LOG:  connection receiv
ed: host=[local]
postgres-lab | 2026-05-10 15:00:06 WIB [46] labuser@labdb LOG:  connection authorized:
user=labuser database=labdb application_name=pg_isready
postgres-lab | 2026-05-10 15:00:06 WIB [46] labuser@labdb LOG:  disconnection: session
time: 0:00:00.002 user=labuser database=labdb host=[local]
postgres-lab | 2026-05-10 15:00:16 WIB [53] [unknown]@[unknown] LOG:  connection receiv
ed: host=[local]
postgres-lab | 2026-05-10 15:00:16 WIB [53] labuser@labdb LOG:  connection authorized:
user=labuser database=labdb application_name=pg_isready
postgres-lab | 2026-05-10 15:00:16 WIB [53] labuser@labdb LOG:  disconnection: session
time: 0:00:00.002 user=labuser database=labdb host=[local]
postgres-lab | 2026-05-10 15:00:26 WIB [60] [unknown]@[unknown] LOG:  connection receiv
ed: host=[local]
postgres-lab | 2026-05-10 15:00:26 WIB [60] labuser@labdb LOG:  connection authorized:
user=labuser database=labdb application_name=pg_isready
postgres-lab | 2026-05-10 15:00:26 WIB [60] labuser@labdb LOG:  disconnection: session
time: 0:00:00.002 user=labuser database=labdb host=[local]
postgres-lab | 2026-05-10 15:00:36 WIB [67] [unknown]@[unknown] LOG:  connection receiv
ed: host=[local]
postgres-lab | 2026-05-10 15:00:36 WIB [67] labuser@labdb LOG:  connection authorized:
user=labuser database=labdb application_name=pg_isready
postgres-lab | 2026-05-10 15:00:36 WIB [67] labuser@labdb LOG:  disconnection: session
time: 0:00:00.002 user=labuser database=labdb host=[local]

```

Perintah docker compose up -d digunakan untuk menjalankan seluruh service pada background mode. Docker akan otomatis mendownload image jika belum tersedia. Setelah container aktif, docker compose ps digunakan untuk melihat status seluruh container apakah sudah running atau belum. Perintah docker compose logs db | tail -20 digunakan untuk melihat 20 log terakhir dari container PostgreSQL guna memastikan proses inisialisasi database berhasil dan healthcheck berjalan normal.

Langkah 2: Koneksi dan Verifikasi Database

1. Koneksi via psql dari host

```
ditto@LOG-JUWADI:~/docker-lab/postgresql$ sudo apt install -y postgresql-client
[sudo: authenticate] Password:
Installing:
  postgresql-client

Installing dependencies:
  libpq5 postgresql-client-18 postgresql-client-common

Suggested packages:
  libpq-oauth postgresql-18 postgresql-doc-18

Summary:
  Upgrading: 0, Installing: 4, Removing: 0, Not Upgrading: 0
  Download size: 1619 kB
  Space needed: 5488 kB / 1024 GB available

ditto@LOG-JUWADI:~/docker-lab/postgresql$ psql -h localhost -U labuser -d labdb
Password for user labuser:
psql (18.3 (Ubuntu 18.3-1), server 16.13)
Type "help" for help.

labdb=# \l
labdb=# \dn
          List of schemas
  Name  | Owner
-----+-----
 app    | labuser
 public | pg_database_owner
(2 rows)

labdb=# \dt app.*
          List of tables
 Schema | Name      | Type  | Owner
-----+-----+-----+-----
 app    | activity_log | table | labuser
 app    | mahasiswa  | table | labuser
 app    | matakuliah  | table | labuser
 app    | nilai      | table | labuser
(4 rows)

labdb=# \d+ app.mahasiswa
```

Perintah sudo apt install -y postgresql-client digunakan untuk menginstall client PostgreSQL pada host Linux agar dapat menjalankan command psql tanpa harus masuk container. Setelah client tersedia, perintah psql -h localhost -U labuser -d labdb digunakan untuk terkoneksi ke database PostgreSQL yang berjalan di Docker melalui port localhost. Di dalam shell psql, command \l menampilkan daftar database, \dn menampilkan schema, \dt app.* menampilkan seluruh tabel pada schema app, dan \d+ app.mahasiswa

menampilkan struktur detail tabel mahasiswa. Query SELECT digunakan untuk membaca isi data tabel. Command \q digunakan untuk keluar dari shell PostgreSQL.

2. Koneksi via docker exec

```
ditto@LOG-JUWADI:~/docker-lab/postgresql$ docker exec -it postgres-db psql -U labuser -d labdb
psql (16.13)
Type "help" for help.

labdb=#
```

```
labdb=# SELECT m.nrp, m.nama, mk.nama AS matakuliah, n.nilai_angka, n.grade
labdb=# FROM app.nilai n
labdb=# JOIN app.mahasiswa m ON n.mahasiswa_id = m.id
labdb=# JOIN app.matakuliah mk ON n.matakuliah_id = mk.id
labdb=# ORDER BY m.nrp, mk.nama;
```

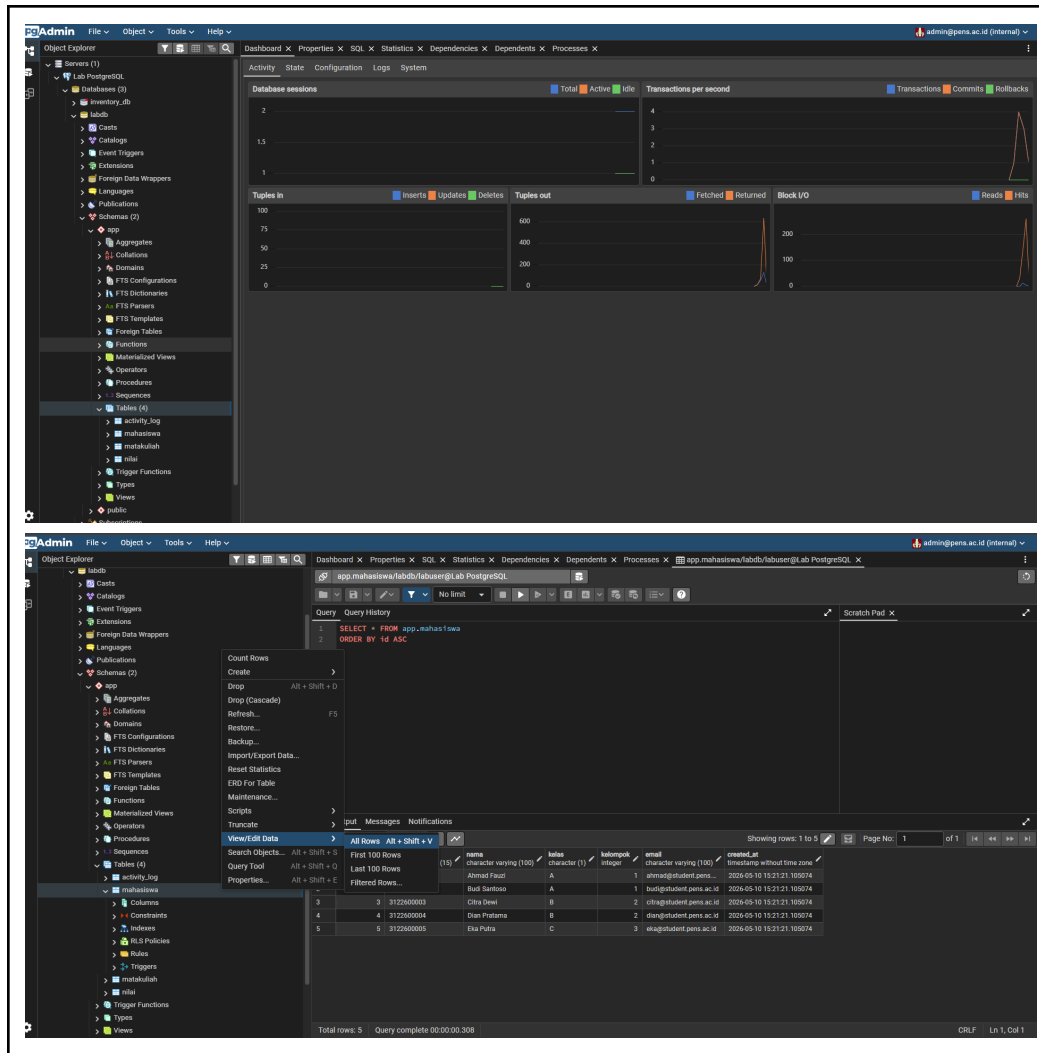
nrp	nama	matakuliah	nilai_angka	grade
3122600001	Ahmad Fauzi	Administrasi Jaringan	85.50	A
3122600001	Ahmad Fauzi	Sistem Basis Data	78.00	B+
3122600002	Budi Santoso	Administrasi Jaringan	92.00	A
3122600003	Citra Dewi	Administrasi Jaringan	70.25	B
3122600004	Dian Pratama	Sistem Operasi	88.75	A

(5 rows)

Perintah docker exec -it postgres-db psql -U labuser -d labdb digunakan untuk membuka shell psql langsung dari dalam container PostgreSQL. Opsi -it memberikan mode interaktif terminal. Query JOIN yang dijalankan menghubungkan tabel nilai, mahasiswa, dan matakuliah untuk menghasilkan data gabungan berupa NRP mahasiswa, nama mahasiswa, nama mata kuliah, nilai angka, dan grade. Query ini menunjukkan implementasi relasi many-to-many antar mahasiswa dan mata kuliah melalui tabel nilai.

3. Koneksi via pgAdmin4





Akses browser ke <http://localhost:5050> digunakan untuk membuka antarmuka pgAdmin4. Login dilakukan menggunakan email dan password yang telah ditentukan di Docker Compose. Saat menambahkan server baru, hostname menggunakan db karena pgAdmin dan PostgreSQL berada dalam network Docker yang sama. Setelah koneksi berhasil, pengguna dapat melakukan navigasi melalui struktur database, schema, dan tabel menggunakan GUI. Fitur View/Edit Data memungkinkan data tabel dilihat dan diedit langsung tanpa menggunakan query SQL manual.

Langkah 3: Operasi CRUD SQL

```

dito@LOG-JUWADI:~/docker-lab/postgresql$ docker exec -i postgres-lab psql -U labuser -d labdb << 'SQLEOF'
> -- === CREATE ===
INSERT INTO app.mahasiswa (nrp, nama, kelas, kelompok, email)
VALUES ('3122600010', 'Fajar Rizki', 'D', 5, 'fajar@student.pens.ac.id');

-- === READ ===
SELECT * FROM app.mahasiswa WHERE kelas = 'A';

SELECT mk.nama, AVG(n.nilai_angka)::NUMERIC(5,2) AS rata_rata, COUNT(*) AS jumlah
FROM app.nilai n
JOIN app.matakuliah mk ON n.matakuliah_id = mk.id
GROUP BY mk.nama
ORDER BY rata_rata DESC;

-- === UPDATE ===
UPDATE app.mahasiswa
SET email = 'fajar.rizki@student.pens.ac.id'
WHERE nrp = '3122600010';

-- === DELETE ===
DELETE FROM app.mahasiswa
WHERE nrp = '3122600010';

-- === JSONB query ===
INSERT INTO app.activity_log (level, source, message, metadata)
VALUES (
    'INFO',
    'web-app',
    'User login',
    '{"user": "admin", "ip": "192.168.1.10"}'
);

SELECT *
FROM app.activity_log
WHERE metadata->'user' = 'admin';
> SQLEOF

INSERT 0 1
 id | nrp      | nama      | kelas | kelompok | email                  | created_at
-----+-----+-----+-----+-----+-----+-----
  1 | 312260001 | Ahmad Fauzi | A     |          | ahmad@student.pens.ac.id | 2026-05-10 15:21:21.105074
  2 | 312260002 | Budi Santoso | A     |          | budi@student.pens.ac.id  | 2026-05-10 15:21:21.105074
(2 rows)

      nama      | rata_rata | jumlah
-----+-----+-----
 Sistem Operasi |      88.75 |      1
 Administrasi Jaringan |      82.58 |      3
 Sistem Basis Data |      78.00 |      1
(3 rows)

UPDATE 1
DELETE 1
INSERT 0 1
 id | timestamp          | level | source | message | metadata
-----+-----+-----+-----+-----+-----
  1 | 2026-05-10 15:33:06.13358 | INFO  | web-app | User login | {"ip": "192.168.1.10", "user": "admin"}
(1 row)

```

Perintah docker exec -it postgres-db psql -U labuser -d labdb << 'SQLEOF' digunakan untuk menjalankan sekumpulan query SQL langsung dari terminal host ke dalam PostgreSQL container. Bagian CREATE melakukan INSERT data mahasiswa baru. READ digunakan untuk membaca data mahasiswa kelas tertentu dan menghitung rata-rata nilai tiap mata kuliah menggunakan agregasi

AVG dan COUNT. UPDATE digunakan untuk mengubah email mahasiswa berdasarkan NRP. DELETE digunakan untuk menghapus data mahasiswa tertentu. Bagian JSONB menunjukkan penggunaan tipe data semi-structured PostgreSQL dengan menyimpan metadata login berbentuk JSON dan melakukan query berdasarkan isi field JSON menggunakan operator ->>.

Langkah 4: Backup dan Restore

1. Backup database (pg_dump)

```
ditto@LOG-JUWADI:~/docker-lab/postgresql$ mkdir -p ~/docker-lab/postgresql/backup
ditto@LOG-JUWADI:~/docker-lab/postgresql$ mkdir -p backup
ditto@LOG-JUWADI:~/docker-lab/postgresql$ docker exec postgres-lab pg_dump \
-U labuser \
-d labdb \
-Fc \
-f /backup/labdb_backup.dump
ditto@LOG-JUWADI:~/docker-lab/postgresql$ docker exec postgres-lab pg_dump \
-U labuser \
-d labdb \
-f /backup/labdb_backup.sql
ditto@LOG-JUWADI:~/docker-lab/postgresql$ docker exec postgres-lab pg_dump \
-U labuser \
-d labdb \
-n app \
-Fc \
-f /backup/labdb_schema_app.dump
ditto@LOG-JUWADI:~/docker-lab/postgresql$ ls -lah backup/
total 52K
drwxr-xr-x 2 ditto ditto 4.0K May 10 15:42 .
drwxr-xr-x 5 ditto ditto 4.0K May 10 14:55 ..
-rw-r--r-- 1 root root 14K May 10 15:42 labdb_backup.dump
-rw-r--r-- 1 root root 11K May 10 15:42 labdb_backup.sql
-rw-r--r-- 1 root root 14K May 10 15:42 labdb_schema_app.dump
ditto@LOG-JUWADI:~/docker-lab/postgresql$
```

Perintah pg_dump digunakan untuk membuat backup database PostgreSQL. Opsi -Fc menghasilkan backup custom format yang terkompres dan fleksibel untuk restore. Backup plain SQL menghasilkan file query SQL biasa yang dapat dibaca manual. Backup schema app saja digunakan jika hanya ingin membackup struktur dan data aplikasi tertentu. Seluruh file backup disimpan di folder /backup yang telah dimount ke host. Perintah ls -la backup/ digunakan untuk memastikan file backup berhasil dibuat.

2. Restore database

```
ditto@LOG-JUWADI:~/docker-lab/postgresql$ docker exec postgres-lab psql \
-U labuser \
-d postgres \
-c "CREATE DATABASE labdb_restore;"
CREATE DATABASE
ditto@LOG-JUWADI:~/docker-lab/postgresql$ docker exec postgres-lab pg_restore \
-U labuser \
-d labdb_restore \
/backup/labdb_backup.dump
ditto@LOG-JUWADI:~/docker-lab/postgresql$ docker exec postgres-lab psql \
-U labuser \
-d labdb_restore \
-c "SELECT * FROM app.mahasiswa;"
 id | nrp      | nama      | kelas | kelompok | email                      | created_at
-----+-----+-----+-----+-----+-----+-----
  1 | 3122600001 | Ahmad Fauzi | A     | 1         | ahmad@student.pens.ac.id | 2026-05-10 15:21:21.105074
  2 | 3122600002 | Budi Santoso | A     | 1         | budi@student.pens.ac.id  | 2026-05-10 15:21:21.105074
  3 | 3122600003 | Citra Dewi  | B     | 2         | citra@student.pens.ac.id | 2026-05-10 15:21:21.105074
  4 | 3122600004 | Dian Pratama | B     | 2         | dian@student.pens.ac.id  | 2026-05-10 15:21:21.105074
  5 | 3122600005 | Eka Putra   | C     | 3         | eka@student.pens.ac.id   | 2026-05-10 15:21:21.105074
(5 rows)
```

Perintah `CREATE DATABASE labdb_restore` digunakan untuk membuat database baru sebagai target restore. `pg_restore` kemudian digunakan untuk memulihkan isi database dari file backup custom format ke database tersebut. Setelah restore selesai, query `SELECT` dijalankan untuk memastikan data mahasiswa berhasil dipulihkan dengan benar.

3. Backup otomatis dengan cron di container

```
ditto@LOG-JUWADI:~/docker-lab/postgresql$ cat > backup/auto-backup.sh << 'BASH'
> #!/bin/sh

TIMESTAMP=$(date +%Y%m%d_%H%M%S)
BACKUP_FILE="/backup/labdb_${TIMESTAMP}.dump"

pg_dump -U labuser -d labdb -Fc -f "$BACKUP_FILE"

echo "[$(date)] Backup created: $BACKUP_FILE"

# Hapus backup lebih dari 7 hari
find /backup -name "labdb_*.dump" -mtime +7 -delete
> BASH
ditto@LOG-JUWADI:~/docker-lab/postgresql$ chmod +x backup/auto-backup.sh
ditto@LOG-JUWADI:~/docker-lab/postgresql$ docker exec postgres-lab /backup/auto-backup.sh
[Sun May 10 15:44:42 WIB 2026] Backup created: /backup/labdb_20260510_154442.dump
ditto@LOG-JUWADI:~/docker-lab/postgresql$ ls -lah backup/
total 72K
drwxr-xr-x 2 dito dito 4.0K May 10 15:44 .
drwxr-xr-x 5 dito dito 4.0K May 10 14:55 ..
-rwxr-xr-x 1 dito dito 274 May 10 15:44 auto-backup.sh
-rw-r--r-- 1 root root 14K May 10 15:44 labdb_20260510_154442.dump
-rw-r--r-- 1 root root 14K May 10 15:42 labdb_backup.dump
-rw-r--r-- 1 root root 11K May 10 15:42 labdb_backup.sql
-rw-r--r-- 1 root root 14K May 10 15:42 labdb_schema_app.dump
```

Perintah `cat > backup/auto-backup.sh << 'BASH'` digunakan untuk membuat script shell otomatisasi backup. Script membuat nama file backup berdasarkan timestamp agar setiap backup unik. `pg_dump` dijalankan otomatis untuk membuat file dump database. Command `find` digunakan untuk menghapus backup yang berumur lebih dari 7 hari agar storage tidak penuh. `chmod +x`

memberikan permission executable pada script. Script kemudian diuji secara manual menggunakan docker exec untuk memastikan backup otomatis berjalan sesuai harapan.

Langkah 5: Monitoring PostgreSQL

1. Statistik database

```
ditto@LOG-JUNADI: /docker-lab/postgresql$ docker exec -i postgres-lab psql -U labuser -d labdb << 'SQLEOF'
> -- Ukuran database
SELECT
    pg_database.datname,
    pg_size_pretty(pg_database_size(pg_database.datname)) AS size
FROM pg_database
ORDER BY pg_database_size(pg_database.datname) DESC;

-- Ukuran tabel
SELECT
    schemaname || '.' || tablename AS table_full,
    pg_size_pretty(
        pg_total_relation_size(schemaname || '.' || tablename)
    ) AS total_size
FROM pg_tables
WHERE schemaname = 'app'
ORDER BY pg_total_relation_size(schemaname || '.' || tablename) DESC;

-- Koneksi aktif
SELECT
    pid,
    username,
    datname,
    client_addr,
    state,
    query_start,
    query
FROM pg_stat_activity
WHERE datname = 'labdb';

-- Statistik tabel
SELECT
    relname,
    seq_scan,
    seq_tup_read,
    idx_scan,
    idx_tup_fetch,
    n_tup_ins,
    n_tup_upd,
    n_tup_del
FROM pg_stat_user_tables
WHERE schemaname = 'app';
> SQLEOF
```

datname	size
labdb_restore	8007 kB
labdb	7959 kB
template1	7575 kB
postgres	7519 kB
template0	7361 kB
inventory_db	7361 kB
(6 rows)	

table_full	total_size
app.activity_log	88 kB
app.mahasiswa	72 kB
app.nilai	56 kB
app.matakuliah	40 kB
(4 rows)	

Perintah monitoring dijalankan menggunakan psql di dalam container. Query ukuran database menggunakan `pg_database_size` untuk mengetahui kapasitas penyimpanan tiap database. Query ukuran tabel menggunakan `pg_total_relation_size` untuk melihat konsumsi storage setiap tabel. Query `pg_stat_activity` digunakan untuk memonitor koneksi aktif, query yang sedang berjalan, dan status session database. Query `pg_stat_user_tables` digunakan untuk melihat statistik penggunaan tabel seperti jumlah sequential scan, index scan, insert, update, dan delete sehingga dapat dianalisis performa query dan aktivitas database.

2. Cek PostgreSQL log

```
initdbLOG-JUNK01: /docker-lab/postgresql$ docker logs postgres-lab | tail -30
sh: locale: not found
2026-05-10 15:21:19.333 WIB [35] WARNING: no usable system locales were found
initdb: warning: enabling "trust" authentication for local connections
initdb: hint: You can change this by editing pg_hba.conf or using the option -A, or --auth-local and --auth-host, the next time you run initdb.
2026-05-10 15:21:21 WIB [1] @ LOG: starting PostgreSQL 16.13 on x86_64-pc-linux-musl, compiled by gcc (Alpine 15.2.0) 15.2.0, 64-bit
2026-05-10 15:21:21 WIB [1] @ LOG: listening on IPv4 address "0.0.0.0", port 5432
2026-05-10 15:21:21 WIB [1] @ LOG: listening on IPv6 address ":::", port 5432
2026-05-10 15:21:21 WIB [1] @ LOG: listening on Unix socket "/var/run/postgresql/.s.PGSQL.5432"
2026-05-10 15:21:21 WIB [60] @ LOG: database system was shut down at 2026-05-10 15:21:21 WIB
2026-05-10 15:21:21 WIB [1] @ LOG: database system is ready to accept connections
2026-05-10 15:21:27 WIB [70] [unknown]@[unknown] LOG: connection received: host=[local]
2026-05-10 15:21:27 WIB [70] labuser@labdb LOG: connection authorized: user=labuser database=labdb application_name=pg_isready
2026-05-10 15:21:27 WIB [70] labuser@labdb LOG: disconnection: session time: 0:00:00.000 user=labuser database=labdb host=[local]
2026-05-10 15:21:37 WIB [77] [unknown]@[unknown] LOG: connection received: host=[local]
2026-05-10 15:21:37 WIB [77] labuser@labdb LOG: connection authorized: user=labuser database=labdb application_name=pg_isready
2026-05-10 15:21:37 WIB [77] labuser@labdb LOG: disconnection: session time: 0:00:00.001 user=labuser database=labdb host=[local]
2026-05-10 15:21:48 WIB [85] [unknown]@[unknown] LOG: connection received: host=[local]
2026-05-10 15:21:48 WIB [85] labuser@labdb LOG: connection authorized: user=labuser database=labdb application_name=pg_isready
2026-05-10 15:21:48 WIB [85] labuser@labdb LOG: disconnection: session time: 0:00:00.003 user=labuser database=labdb host=[local]
2026-05-10 15:21:58 WIB [92] [unknown]@[unknown] LOG: connection received: host=[local]
2026-05-10 15:21:58 WIB [92] labuser@labdb LOG: connection authorized: user=labuser database=labdb application_name=pg_isready
2026-05-10 15:21:58 WIB [92] labuser@labdb LOG: disconnection: session time: 0:00:00.002 user=labuser database=labdb host=[local]
2026-05-10 15:22:08 WIB [100] [unknown]@[unknown] LOG: connection received: host=[local]
2026-05-10 15:22:08 WIB [100] labuser@labdb LOG: connection authorized: user=labuser database=labdb application_name=pg_isready
2026-05-10 15:22:08 WIB [100] labuser@labdb LOG: disconnection: session time: 0:00:00.001 user=labuser database=labdb host=[local]
2026-05-10 15:22:18 WIB [107] [unknown]@[unknown] LOG: connection received: host=[local]
2026-05-10 15:22:18 WIB [107] labuser@labdb LOG: connection authorized: user=labuser database=labdb application_name=pg_isready
2026-05-10 15:22:18 WIB [107] labuser@labdb LOG: disconnection: session time: 0:00:00.001 user=labuser database=labdb host=[local]
2026-05-10 15:22:28 WIB [115] [unknown]@[unknown] LOG: connection received: host=[local]
2026-05-10 15:22:28 WIB [115] labuser@labdb LOG: connection authorized: user=labuser database=labdb application_name=pg_isready
2026-05-10 15:22:28 WIB [115] labuser@labdb LOG: disconnection: session time: 0:00:00.002 user=labuser database=labdb host=[local]

initdbLOG-JUNK01: /docker-lab/postgresql$ docker logs -f postgres-lab
The files belonging to this database system will be owned by user "postgres".
This user must also own the server process.

The database cluster will be initialized with locale "en_US.utf8".
The default database encoding has accordingly been set to "UTF8".
The default text search configuration will be set to "english".

Data page checksums are disabled.

fixing permissions on existing directory /var/lib/postgresql/data ... ok
creating subdirectories ... ok
selecting dynamic shared memory implementation ... posix
selecting default max_connections ... 100
selecting default shared_buffers ... 128MB
selecting default time zone ... Asia/Jakarta
creating configuration files ... ok
running bootstrap script ... ok
sh: locale: not found
2026-05-10 15:21:19.333 WIB [35] WARNING: no usable system locales were found
performing post-bootstrap initialization ... ok
initdb: warning: enabling "trust" authentication for local connections
initdb: hint: You can change this by editing pg_hba.conf or using the option -A, or --auth-local and --auth-host, the next time you run initdb.
syncing data to disk ... ok

Success. You can now start the database server using:

    pg_ctl -D /var/lib/postgresql/data -l logfile start
```

Perintah `docker exec postgres-db ls /var/log/postgresql/` digunakan untuk melihat file log yang tersedia dalam container PostgreSQL. File log disimpan berdasarkan tanggal sesuai konfigurasi `log_filename`. Perintah `cat` digunakan untuk membaca isi log terbaru dan `tail -30` membatasi tampilan hanya 30 baris terakhir. Log ini berguna untuk menganalisis error, koneksi database, query lambat, serta aktivitas pengguna pada PostgreSQL.

E. PERTANYAAN

Pre-Lab

1. Apa fungsi file/folder `/docker-entrypoint-initdb.d/` di image PostgreSQL?
Folder `/docker-entrypoint-initdb.d/` digunakan oleh image resmi PostgreSQL untuk menjalankan file inisialisasi otomatis saat database pertama kali dibuat. Semua file `.sql`, `.sql.gz`, atau `.sh` di folder tersebut akan dieksekusi secara otomatis ketika volume database masih kosong. Pada praktikum ini, file `01-create-schema.sql` dijalankan untuk membuat schema, tabel, index, sample data, dan user database. Jika database sudah pernah dibuat sebelumnya, script di folder ini tidak akan dijalankan lagi.
2. Mengapa `POSTGRES_PASSWORD` wajib diset? Apa risikonya jika tidak ada password?
`POSTGRES_PASSWORD` digunakan untuk menentukan password user administrator PostgreSQL saat container pertama kali dibuat. Password ini penting untuk keamanan akses database. Jika password tidak diset, database dapat diakses tanpa autentikasi yang aman, terutama jika port PostgreSQL terbuka ke jaringan luar. Risiko utamanya adalah unauthorized access, pencurian data, modifikasi database tanpa izin, dan potensi penghapusan data oleh pihak lain.
3. Jelaskan perbedaan antara `pg_dump` format custom (`-Fc`) dan format SQL plain text.
Format custom (`-Fc`) menghasilkan file backup biner terkompres yang lebih kecil dan fleksibel untuk proses restore menggunakan `pg_restore`. Format ini mendukung restore parsial, parallel restore, dan selective restore object tertentu. Sementara SQL plain text menghasilkan file `.sql` berisi query SQL biasa yang dapat dibaca manusia dan dijalankan langsung menggunakan `psql`. File plain text biasanya lebih besar karena tidak dikompresi.
4. Apa itu `shared_buffers` dan mengapa perlu disesuaikan untuk container?
`shared_buffers` adalah parameter memory PostgreSQL yang menentukan jumlah RAM yang digunakan PostgreSQL untuk cache data internal. Semakin besar nilainya, semakin banyak data yang dapat disimpan di memory sehingga query menjadi lebih cepat. Pada container Docker, resource RAM biasanya terbatas sehingga nilai `shared_buffers` perlu disesuaikan agar PostgreSQL tidak menggunakan memory berlebihan yang dapat menyebabkan container lambat atau crash.
5. Mengapa data PostgreSQL harus disimpan di Docker Volume, bukan di container layer?
Data PostgreSQL harus disimpan di Docker Volume karena container layer bersifat ephemeral atau sementara. Jika container dihapus, data di dalam container layer ikut hilang. Docker Volume menyimpan data secara persisten di

host sehingga database tetap aman walaupun container dihapus, restart, atau image diperbarui. Volume juga lebih stabil dan lebih optimal untuk performa database dibanding penyimpanan langsung di filesystem container.

Post-Lab

1. Jalankan `docker compose down` lalu `docker compose up -d`. Apakah data mahasiswa masih ada? Buktikan.

Ya, data mahasiswa masih ada karena perintah `docker compose down` hanya menghentikan dan menghapus container tanpa menghapus volume database. Data tetap tersimpan di volume `pg-data`. Pembuktian dapat dilakukan dengan menjalankan query:

```
SELECT * FROM app.mahasiswa;
```

Hasil query tetap menampilkan seluruh data mahasiswa yang sebelumnya sudah dibuat.

2. Jalankan `docker compose down -v` lalu `docker compose up -d`. Apa yang terjadi? Apakah `init script` dijalankan ulang?

Perintah `docker compose down -v` menghapus container sekaligus seluruh Docker Volume yang terhubung, termasuk volume database PostgreSQL. Akibatnya seluruh data database hilang. Saat `docker compose up -d` dijalankan kembali, PostgreSQL mendeteksi bahwa database masih kosong sehingga proses inisialisasi dimulai ulang dan script pada `/docker-entrypoint-initdb.d/` dijalankan kembali. Hal ini terlihat pada log container yang menunjukkan proses `CREATE TABLE`, `INSERT`, dan `CREATE ROLE`.

3. Bandingkan ukuran file backup format custom vs SQL. Mana yang lebih kecil dan mengapa?

Backup format custom (`.dump`) biasanya lebih kecil dibanding backup SQL plain text (`.sql`) karena format custom menggunakan kompresi internal PostgreSQL. File SQL plain text menyimpan seluruh query SQL dalam bentuk teks biasa sehingga ukuran file lebih besar. Selain lebih kecil, format custom juga lebih efisien untuk restore database besar.

4. Buat query yang menampilkan mahasiswa yang belum memiliki nilai di semester apapun.

```
SELECT m.id, m.nrp, m.nama  
FROM app.mahasiswa m  
LEFT JOIN app.nilai n
```

```
ON m.id = n.mahasiswa_id  
WHERE n.id IS NULL;
```

Query tersebut menggunakan LEFT JOIN untuk mencari mahasiswa yang tidak memiliki pasangan data pada tabel nilai.

5. Jelaskan peran user app_reader yang dibuat di init script. Apa bedanya dengan labuser?

User app_reader dibuat sebagai user read-only yang hanya memiliki hak akses membaca data pada schema app. User ini tidak dapat melakukan INSERT, UPDATE, DELETE, atau perubahan struktur database. Sementara labuser adalah user utama database yang memiliki hak akses penuh untuk membuat tabel, memodifikasi data, melakukan backup, restore, dan administrasi database lainnya. Pemisahan role seperti ini penting untuk keamanan aplikasi agar tidak semua user memiliki hak akses penuh terhadap database.

Modul 5

Logging Service Docker dengan PostgreSQL

A. TUJUAN PEMBELAJARAN

Setelah praktikum ini, mahasiswa mampu:

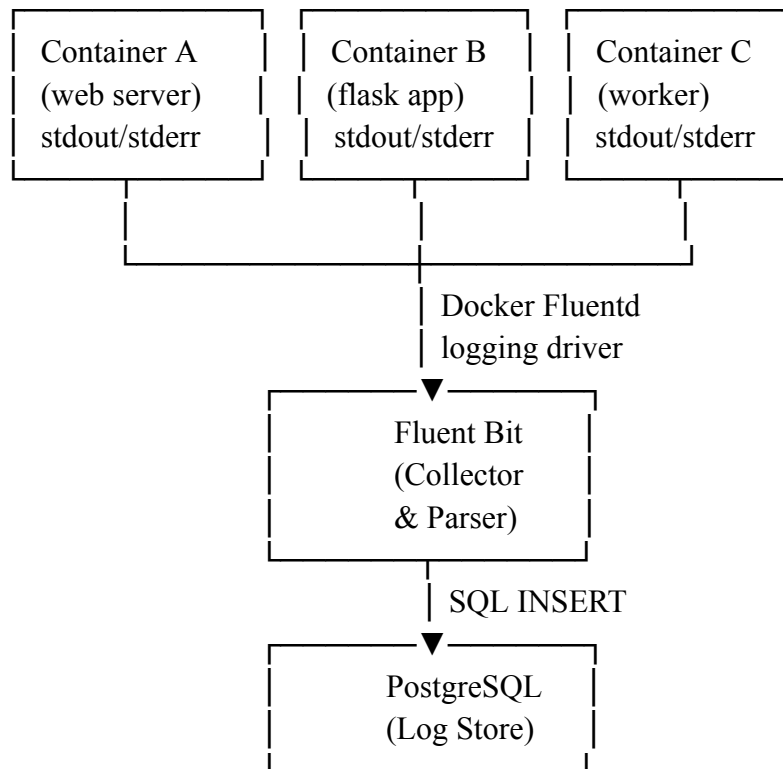
1. Memahami arsitektur centralized logging di lingkungan container
2. Men-deploy Fluent Bit sebagai log collector dan forwarder dalam Docker
3. Mengkonfigurasi Docker logging driver untuk mengirim log ke Fluent Bit
4. Menyimpan log dari semua container ke PostgreSQL secara terpusat
5. Membuat Python log generator untuk mensimulasikan log aplikasi
6. Melakukan query dan analisis log menggunakan SQL di PostgreSQL
7. Mengimplementasikan log rotation dan retention policy
8. Mengorkestrasi seluruh stack logging dengan Docker Compose

B. DASAR TEORI

1. Pentingnya Centralized Logging

Di lingkungan container, service bisa berjumlah banyak dan bersifat ephemeral (dibuat dan dihapus secara dinamis). Log tersebar di masing-masing container menyulitkan debugging dan audit. Centralized logging mengumpulkan semua log ke satu tempat, memudahkan pencarian, analisis, dan alerting.

2. Arsitektur Logging



3. Fluent Bit vs Fluentd

Aspek	Fluent Bit	Fluentd
Bahasa	C	Ruby + C
Memory	~1 MB	~40 MB
Plugin	Built-in essentials	1000+ plugin ecosystem
Use Case	Edge/sidecar collector	Aggregator/forwarder
Docker Image	fluent/fluent-bit	fluent/fluentd

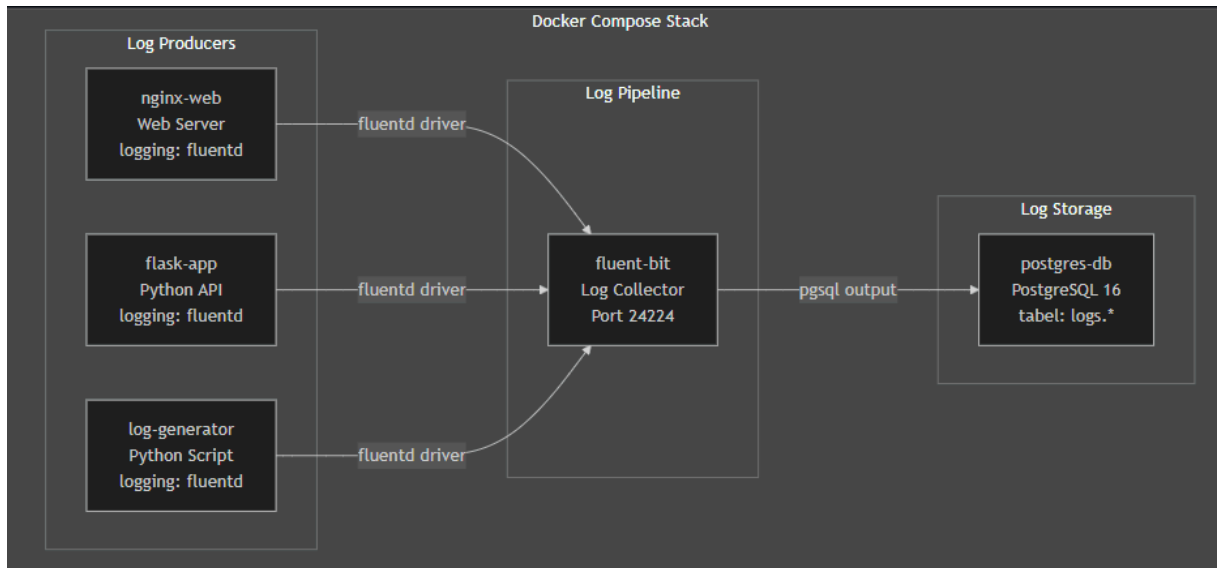
Praktikum ini menggunakan Fluent Bit karena ringan dan cocok untuk skenario lab.

4. Docker Logging Driver

Docker mendukung beberapa logging driver. Default-nya adalah json-file yang menyimpan log ke file JSON di host. Untuk centralized logging, kita bisa menggunakan driver fluentd yang mengirim log langsung ke Fluent Bit/Fluentd.

Driver	Deskripsi
json-file	Default, simpan ke file JSON
syslog	Kirim ke syslog daemon
fluentd	Kirim ke Fluentd/Fluent Bit
journald	Kirim ke systemd journal
none	Buang semua log

C. TOPOLOGI LAB



D. PRAKTIKUM

Langkah 0: Persiapan Project

1. Persiapan Project

```
elfandi@elfandi-Latitude-7414:~$ mkdir -p ~/docker-lab/logging/{fluent-bit,app,generator,init}
elfandi@elfandi-Latitude-7414:~$ cd ~/docker-lab/logging
```

Langkah ini digunakan untuk pembuatan struktur folder project menggunakan mkdir dan memisahkan setiap komponen sistem logging seperti database initialization, Fluent Bit, aplikasi, dan log generator. Hal ini bertujuan agar project lebih terorganisir dan mudah dikelola dalam lingkungan Docker.

Langkah 1: Buat Database Schema untuk Logging

1. Buat Database Schema untuk Logging

```

elfandi@elfandi-Latitude-7414:~/docker-lab/logging$ cat > init/01-logging-schema.sql << 'EOF'
-- =====
-- Schema untuk Centralized Logging
-- =====

CREATE SCHEMA IF NOT EXISTS logs;

-- Tabel utama: menyimpan semua log dari container
CREATE TABLE logs.container_logs (
    id BIGSERIAL PRIMARY KEY,
    received_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    timestamp TIMESTAMP,
    container_name VARCHAR(100),
    container_id VARCHAR(64),
    source VARCHAR(10),          -- stdout / stderr
    log_level VARCHAR(10),
    message TEXT,
    raw_log TEXT,
    metadata JSONB DEFAULT '{}'
);

-- Tabel: summary per container per jam (untuk dashboard)
CREATE TABLE logs.hourly_summary (
    id SERIAL PRIMARY KEY,
    hour TIMESTAMP NOT NULL,
    container_name VARCHAR(100),
    total_logs INTEGER DEFAULT 0,
    error_count INTEGER DEFAULT 0,
    warn_count INTEGER DEFAULT 0,
    info_count INTEGER DEFAULT 0,
    UNIQUE(hour, container_name)
);

-- Index untuk performa query
CREATE INDEX idx_logs_timestamp ON logs.container_logs(timestamp);
CREATE INDEX idx_logs_container ON logs.container_logs(container_name);
CREATE INDEX idx_logs_level ON logs.container_logs(log_level);
CREATE INDEX idx_logs_received ON logs.container_logs(received_at);
CREATE INDEX idx_logs_metadata ON logs.container_logs USING GIN(metadata);
CREATE INDEX idx_summary_hour ON logs.hourly_summary(hour);

-- Fungsi: auto-cleanup log > 30 hari
CREATE OR REPLACE FUNCTION logs.cleanup_old_logs()
RETURNS INTEGER AS $$
DECLARE
    deleted_count INTEGER;
BEGIN
    DELETE FROM logs.container_logs
    WHERE received_at < NOW() - INTERVAL '30 days';
    GET DIAGNOSTICS deleted_count = ROW_COUNT;
    RETURN deleted_count;
END;
$$ LANGUAGE plpgsql;

-- View: log terbaru dengan format readable
EOFSE NOTICE 'Logging schema created successfully!';ime.

```

Pada langkah ini dibuat schema PostgreSQL khusus untuk menyimpan log dari container. Dibuat tabel utama container_logs untuk menyimpan semua log, serta tabel hourly_summary untuk analisis agregasi log. Selain itu ditambahkan index untuk optimasi query, function cleanup otomatis log lama, serta view untuk mempermudah akses data log secara ringkas.

Langkah 2: Konfigurasi Fluent Bit

1. Konfigurasi Fluent Bit


```

elfandi@elfandi-Latitude-7414:~/docker-lab/logging$ cat > fluent-bit/fluent-bit.conf << 'EOF'
[SERVICE]
    Flush      5
    Daemon     Off
    Log_Level  info
    Parsers_File parsers.conf

[INPUT]
    Name        forward
    Listen      0.0.0.0
    Port        24224
    Tag         docker.*

[FILTER]
    Name        parser
    Match       docker.*
    Key_Name    log
    Parser      docker_json
    Reserve_Data On

[FILTER]
    Name        modify
    Match       docker.*
    Rename      log message
    Rename      container_name source_container

[OUTPUT]
    Name        pgsqldb
    Match       docker.*
    Host        postgres-db
    Port        5432
    User        labuser
    Password    labpass123
    Database    labdb
    Table       container_logs
    Schema      logs
    Timestamp_Key timestamp
    Async       false
    min_pool_size 1
    max_pool_size 4

[OUTPUT]
    Name        stdout
    Match       docker.*
    Format       json_lines
EOF
cat > fluent-bit/parsers.conf << 'EOF'
[PARSER]
    Name        docker_json
    Format       json
    Time_Key     time
    Time_Format  %Y-%m-%dT%H:%M:%S.%L
    Time_Keep    On
EOF
Time Format  %Y-%m-%d %H:%M:%S.%L[^\ ]+ (?<level>[A-Z]+) (?<message>.*)$

```

Fluent Bit dikonfigurasi sebagai log collector yang menerima log dari Docker melalui input forward. Log kemudian diparse menggunakan parser JSON dan dimodifikasi sebelum dikirim ke PostgreSQL. Output utama adalah database postgres-db, sedangkan output kedua digunakan untuk debugging melalui stdout dalam format JSON.

Langkah 3: Buat Log Generator (Simulasi Multi-Level Log)

1. Buat Log Generator (Simulasi Multi-Level Log)

```
elfandi@elfandi-Latitude-7414:~/docker-lab/logging$ cat > generator/generator.py << 'PYEOF'
"""
Log Generator – mensimulasikan log dari aplikasi production.
Menghasilkan log dengan berbagai level: DEBUG, INFO, WARN, ERROR, CRITICAL.
"""
import json, time, random, socket, datetime, sys, os

HOSTNAME = socket.gethostname()
LOG_INTERVAL = float(os.environ.get("LOG_INTERVAL", "2"))

# Simulasi event dengan bobot probabilitas
EVENTS = [
    {"level": "INFO", "weight": 50, "messages": [
        "User login successful",
        "Page /dashboard loaded in {ms}ms",
        "API request GET /api/users completed",
        "Session created for user_{uid}",
        "Cache hit for key: product_{pid}",
        "Health check passed",
        "Background job completed: email_send"
    ]},
    {"level": "DEBUG", "weight": 20, "messages": [
        "Database query executed in {ms}ms",
        "Redis connection pool: {pool} active",
        "Request headers: content-type=application/json",
        "Middleware chain completed in {ms}ms"
    ]},
    {"level": "WARN", "weight": 15, "messages": [
        "Slow query detected: {ms}ms (threshold: 1000ms)",
        "Memory usage at {mem}% – approaching limit",
        "Rate limit approaching for IP 192.168.{ip}.{host}",
        "Deprecated API endpoint called: /api/v1/legacy",
        "Certificate expires in {days} days"
    ]},
    {"level": "ERROR", "weight": 10, "messages": [
        "Failed to connect to database: timeout after 5s",
        "NullPointerException in UserService.getProfile()",
        "HTTP 500 Internal Server Error on /api/checkout",
        "Disk write failed: /var/log/app.log – Permission denied",
        "Payment gateway returned error code {code}"
    ]},
    {"level": "CRITICAL", "weight": 5, "messages": [
        "Database connection pool exhausted – all {pool} connections in use",
        "Out of memory: container killed by OOM",
        "SSL certificate EXPIRED – HTTPS unavailable",
        "Data corruption detected in table: orders"
    ]}
]

def weighted_choice():
    total = sum(e["weight"] for e in EVENTS)
    r = random.uniform(0, total)
    cumulative = 0
    for event in EVENTS:
        cumulative += event["weight"]
        if r < cumulative:
            return event

EOF ["python", "-u", "generator.py"]ndom.uniform(-0.5, 0.5))terval={LOG_INTERVAL}s"
```

Pada langkah ini dibuat aplikasi Python yang berfungsi sebagai simulasi sistem produksi dengan menghasilkan log secara otomatis. Log memiliki berbagai level seperti INFO, DEBUG, WARN, ERROR, dan CRITICAL dengan pesan acak untuk mensimulasikan kondisi sistem nyata. Output log dikirim ke stdout dalam format JSON agar dapat ditangkap oleh Docker logging driver.

Langkah 4: Buat Flask App dengan Structured Logging

1. Buat Flask App dengan Structured Logging

```
elfandi@elfandi-Latitude-7414:~/docker-lab/logging$ cat > app/requirements.txt << 'EOF'
flask==3.1.*
psycpg2-binary==2.9.*
EOF

cat > app/app.py << 'PYEOF'
import os, json, socket, datetime, logging, sys
from flask import Flask, jsonify, request
import psycpg2

app = Flask(__name__)

# Structured JSON logging ke stdout
class JSONFormatter(logging.Formatter):
    def format(self, record):
        return json.dumps({
            "timestamp": datetime.datetime.now().isoformat(),
            "level": record.levelname,
            "hostname": socket.gethostname(),
            "service": "flask-app",
            "message": record.getMessage(),
            "module": record.module
        })

handler = logging.StreamHandler(sys.stdout)
handler.setFormatter(JSONFormatter())
app.logger.handlers = [handler]
app.logger.setLevel(logging.INFO)

DB = dict(host=os.environ.get("DB_HOST", "postgres-db"),
          dbname=os.environ.get("DB_NAME", "labdb"),
          user=os.environ.get("DB_USER", "labuser"),
          password=os.environ.get("DB_PASS", "labpass123"))

@app.route("/")
def index():
    app.logger.info(f"Index accessed from {request.remote_addr}")
    return jsonify({"service": "flask-app", "status": "running"})

@app.route("/api/logs/stats")
def log_stats():
    """Query statistik log dari PostgreSQL"""
    try:
        conn = psycpg2.connect(**DB); cur = conn.cursor()
        cur.execute("""
            SELECT log_level, COUNT(*) as count
            FROM logs.container_logs
            WHERE received_at > NOW() - INTERVAL '1 hour'
            GROUP BY log_level ORDER BY count DESC
        """)
        stats = [{"level": r[0], "count": r[1]} for r in cur.fetchall()]
        cur.execute("SELECT COUNT(*) FROM logs.container_logs")
        total = cur.fetchone()[0]
        cur.close(); conn.close()
        return jsonify({"total_logs": total, "last_hour": stats})
    except Exception as e:
        app.logger.error(f"Error in log_stats: {e}")
        return jsonify({"error": str(e)}), 500

EOF ["python", "-u", "app.py"] -r requirements.txt or r in cur.fetchall()] FROM logs.container
```

Aplikasi Flask dibuat untuk menyediakan API serta menghasilkan structured logging dalam format JSON. Aplikasi ini juga terhubung ke PostgreSQL untuk mengambil statistik log dan melakukan pencarian berdasarkan keyword atau level log. Semua aktivitas aplikasi dicatat sebagai log

terstruktur untuk dikirim ke Fluent Bit.

Langkah 5: Docker Compose untuk Stack Logging

1. Docker Compose untuk Stack Logging

```
elfandi@elfandi-Latitude-7414:~/docker-lab/logging$ cat > docker-compose.yml << 'EOF'
services:
  # === Log Collector: Fluent Bit ===
  fluent-bit:
    image: fluent/fluent-bit:latest
    container_name: fluent-bit
    ports:
      - "24224:24224"
      - "24224:24224/udp"
    volumes:
      - ./fluent-bit/fluent-bit.conf:/fluent-bit/etc/fluent-bit.conf:ro
      - ./fluent-bit/parsers.conf:/fluent-bit/etc/parsers.conf:ro
    networks:
      - logging-net
    depends_on:
      postgres-db:
        condition: service_healthy
    restart: unless-stopped

  # === Log Storage: PostgreSQL ===
  postgres-db:
    image: postgres:16-alpine
    container_name: postgres-db
    environment:
      POSTGRES_DB: labdb
      POSTGRES_USER: labuser
      POSTGRES_PASSWORD: labpass123
      TZ: Asia/Jakarta
    ports:
      - "5432:5432"
    volumes:
      - pg-data:/var/lib/postgresql/data
      - ./init:/docker-entrypoint-initdb.d:ro
    networks:
      - logging-net
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U labuser -d labdb"]
      interval: 10s
      timeout: 5s
      retries: 5
    restart: unless-stopped

  # === Log Producer: Nginx ===
  nginx-web:
    image: nginx:alpine
    container_name: nginx-web
    ports:
      - "8080:80"
    networks:
      - logging-net
    logging:
      driver: fluentd
      options:
        fluentd-address: "localhost:24224"
        fluentd-async: "true"
EOF
```

Pada langkah ini seluruh service digabungkan menggunakan Docker Compose, terdiri dari Fluent Bit sebagai log collector, PostgreSQL sebagai

database, serta tiga producer log yaitu Nginx, Flask App, dan Log Generator. Semua service terhubung dalam satu network logging-net sehingga dapat saling berkomunikasi, dan Fluent Bit bertugas mengirim semua log ke database secara terpusat.

Langkah 6: Deploy dan Verifikasi

1. Build dan jalankan

```
ditto@LOG-JUWADI: ~/docker-lab/logging$ docker compose up --build -d
[+] Building 1.8s (21/21) FINISHED
=> [internal] load local bake definitions 0.0s
=> => reading from stdin 1.00kB 0.0s
=> [log-generator internal] load build definition from Dockerfile 0.0s
=> => transferring dockerfile: 131B 0.0s
=> [flask-app internal] load build definition from Dockerfile 0.0s
=> => transferring dockerfile: 206B 0.0s
=> [log-generator internal] load metadata for docker.io/library/python:3.11-alpine 1.1s
=> [flask-app internal] load metadata for docker.io/library/python:3.11-slim 1.1s
=> [log-generator internal] load .dockerignore 0.0s
=> => transferring context: 2B 0.0s
=> [flask-app internal] load .dockerignore 0.0s
=> => transferring context: 2B 0.0s
=> [log-generator 1/3] FROM docker.io/library/python:3.11-alpine@sha256:8b5bfdb1fd2d 0.0s
=> => resolve docker.io/library/python:3.11-alpine@sha256:8b5bfdb1fd2d78aa94e21c4d61 0.0s
=> [log-generator internal] load build context 0.0s
=> => transferring context: 34B 0.0s
=> [flask-app 1/5] FROM docker.io/library/python:3.11-slim@sha256:a5b427ace4900267d9 0.0s
=> => resolve docker.io/library/python:3.11-slim@sha256:a5b427ace4900267d93db34138e5 0.0s
=> [flask-app internal] load build context 0.0s
=> => transferring context: 63B 0.0s
=> CACHED [log-generator 2/3] WORKDIR /app 0.0s
=> CACHED [log-generator 3/3] COPY generator.py . 0.0s
=> [log-generator] exporting to image 0.2s
=> => exporting layers 0.0s
=> => exporting manifest sha256:39d915d5a4ecef4e121c4a9767fb80623513cdb6a3c5a75d1e84 0.0s
=> => exporting config sha256:f185305e0af1210cd60dc2d3446956d274ac4fa501a02d10dcdb4 0.0s
=> => exporting attestation manifest sha256:ffab07f3608a3f86ee46def7ed9b7a345b785528 0.0s
=> => exporting manifest list sha256:eb8db4a265ca2edfc5bde1259fd8faee2f014251b2bdf4 0.0s
=> => naming to docker.io/library/logging-log-generator:latest 0.0s
=> => unpacking to docker.io/library/logging-log-generator:latest 0.0s
=> CACHED [flask-app 2/5] WORKDIR /app 0.0s
=> CACHED [flask-app 3/5] COPY requirements.txt . 0.0s
=> CACHED [flask-app 4/5] RUN pip install --no-cache-dir -r requirements.txt 0.0s
=> CACHED [flask-app 5/5] COPY app.py . 0.0s
=> [flask-app] exporting to image 0.2s
=> => exporting layers 0.0s
=> => exporting manifest sha256:99d76a6db1c12b49a73ce57684dee76af1e99fbd3fb0f63e5a8f 0.0s
=> => exporting config sha256:5d4b0789a9650c2fb0c2086cb85857ad9a23e174c8fdb92a09b979 0.0s
=> => exporting attestation manifest sha256:bfc9e41d4fe4b6bb7b7d3bb0cdcc269dc2d9a1c1 0.0s
=> => exporting manifest list sha256:1d5da5bceab4602673b2b5babeac8302afd067f911cca9d 0.0s
=> => naming to docker.io/library/logging-flask-app:latest 0.0s
=> => unpacking to docker.io/library/logging-flask-app:latest 0.0s
=> [log-generator] resolving provenance for metadata file 0.0s
=> [flask-app] resolving provenance for metadata file 0.0s

[+] up 8/8
✔Image logging-flask-app Built 2.0s
✔Image logging-log-generator Built 2.0s
✔Network logging_logging-net Created 0.1s
✔Container postgres-db Healthy 11.4s
✔Container fluent-bit Started 11.5s
✔Container log-generator Started 11.6s
✔Container flask-app Started 11.7s
✔Container nginx-web Started 11.7s
ditto@LOG-JUWADI: ~/docker-lab/logging$ docker compose ps

```

NAME	STATUS	IMAGE	PORTS	COMMAND	SERVICE	CREATED
flask-app	Up 26 seconds	logging-flask-app	0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp	"python -u app.py"	flask-app	38 secon
ds ago	Up 26 seconds	fluent/fluent-bit:latest	0.0.0.0:24224->24224/tcp, 0.0.0.0:24224->24224/udp, [::]:24224->24224/tcp, [::]:24224->24224/udp	"/fluent-bit/bin/flu..."	fluent-bit	38 secon
log-generator	Up 26 seconds	logging-log-generator	0.0.0.0:5433->5432/tcp, [::]:5433->5432/tcp	"python -u generator..."	log-generator	38 secon
ds ago	Up 26 seconds	nginx:alpine	0.0.0.0:8081->80/tcp, [::]:8081->80/tcp	"/docker-entrypoint..."	nginx-web	38 secon
postgres-db	Up 37 seconds (healthy)	postgres:16-alpine	0.0.0.0:5433->5432/tcp, [::]:5433->5432/tcp	"docker-entrypoint.s..."	postgres-db	38 secon

```
ditto@LOG-JUWADI: ~/docker-lab/logging$ sleep 30
```

Perintah ini membangun seluruh image custom lalu menjalankan semua container secara background. docker compose ps digunakan untuk memastikan seluruh service berjalan normal. Delay sleep 30 memberi waktu agar Fluent Bit mulai menerima log dan PostgreSQL mulai menyimpan data log dari seluruh container.

2. Generate traffic untuk Nginx

```
dito@LOG-JUWADI:~/docker-lab/logging$ for i in $(seq 1 20); do curl -s http://localhost:8081  
> /dev/null; done  
dito@LOG-JUWADI:~/docker-lab/logging$ for i in $(seq 1 10); do curl -s http://localhost:5001  
/ > /dev/null; done  
dito@LOG-JUWADI:~/docker-lab/logging$ docker compose logs fluent-bit  
fluent-bit | Fluent Bit v5.0.5  
fluent-bit | * Copyright (C) 2015-2026 The Fluent Bit Authors  
fluent-bit | * Fluent Bit is a CNCF graduated project under the Fluent organization  
fluent-bit | * https://fluentbit.io  
fluent-bit |  
fluent-bit |  
fluent-bit |  
fluent-bit |  
fluent-bit |  
fluent-bit |  
fluent-bit |  
fluent-bit |  
fluent-bit |  
fluent-bit |  
fluent-bit | [2026/05/10 10:14:23.042] [ info] [fluent bit] version=5.0.5, commit=05121015a  
7, pid=1  
fluent-bit | [2026/05/10 10:14:23.042] [ info] [storage] ver=1.5.4, type=memory, sync=norma  
l, checksum=off, max_chunks_up=128  
fluent-bit | [2026/05/10 10:14:23.042] [ info] [simd ] SSE2  
fluent-bit | [2026/05/10 10:14:23.042] [ info] [cmetrics] version=2.1.3  
fluent-bit | [2026/05/10 10:14:23.042] [ info] [ctraces ] version=0.7.1  
fluent-bit | [2026/05/10 10:14:23.042] [ info] [input:forward:forward.0] initializing  
fluent-bit | [2026/05/10 10:14:23.042] [ info] [input:forward:forward.0] storage_strategy='  
memory' (memory only)  
fluent-bit | [2026/05/10 10:14:23.042] [ info] [input:forward:forward.0] listening on 0.0.0  
.0:24224  
fluent-bit | [2026/05/10 10:14:23.042] [ info] [output:pgsql:pgsql.0] Opening connection: #  
0  
fluent-bit | [2026/05/10 10:14:23.055] [ info] [output:pgsql:pgsql.0] switching postgresql  
connection to non-blocking mode  
fluent-bit | [2026/05/10 10:14:23.055] [ info] [output:pgsql:pgsql.0] host=postgres-db port  
=5432 dbname=labdb OK  
fluent-bit | [2026/05/10 10:14:23.055] [ info] [output:pgsql:pgsql.0] we check that the tab  
le "container_logs" exists, if not we create it  
fluent-bit | NOTICE: relation "container_logs" already exists, skipping  
fluent-bit | [2026/05/10 10:14:23.057] [ info] [sp] stream processor started  
fluent-bit | [2026/05/10 10:14:23.057] [ info] [engine] Shutdown Grace Period=5, Shutdown I  
nput Grace Period=2  
fluent-bit | [2026/05/10 10:14:23.057] [ info] [output:stdout:stdout.1] worker #0 started  
fluent-bit | {"date":1778408063.0,"source_container":"/nginx-web","source":"stdout","messag
```

Loop ini mengakses Nginx sebanyak 20 kali untuk menghasilkan access log. Setiap request akan dicatat Docker logging driver lalu diteruskan ke Fluent Bit dan PostgreSQL. Teknik ini digunakan untuk mensimulasikan traffic web nyata.

Loop yang mengakses Flask API beberapa kali agar aplikasi menghasilkan structured JSON log. Log tersebut kemudian diproses oleh Fluent Bit dan dimasukkan ke database PostgreSQL.

3. Query log di PostgreSQL

```

dito@LOG-JUWADI:~/docker-lab/logging$ docker exec -i postgres-db psql -U labuser -d labdb <<
'SQLEOF'
> -- Total log yang masuk
SELECT COUNT(*) AS total_logs FROM logs.container_logs;

-- Log terbaru (view)
SELECT * FROM logs.recent_logs LIMIT 10;

-- Distribusi log per container
SELECT container_name, COUNT(*) AS total
FROM logs.container_logs
GROUP BY container_name ORDER BY total DESC;

-- Distribusi per level
SELECT log_level, COUNT(*) AS total
FROM logs.container_logs
GROUP BY log_level ORDER BY total DESC;

-- Error summary (view)
SELECT * FROM logs.error_summary;

-- Cari log ERROR dalam 1 jam terakhir
SELECT timestamp, container_name, message
FROM logs.container_logs
WHERE log_level = 'ERROR'
AND received_at > NOW() - INTERVAL '1 hour'
ORDER BY timestamp DESC LIMIT 10;

-- Log rate per menit (5 menit terakhir)
SELECT
    date_trunc('minute', received_at) AS minute,
    COUNT(*) AS logs_per_minute
FROM logs.container_logs
WHERE received_at > NOW() - INTERVAL '5 minutes'
GROUP BY minute ORDER BY minute;
> SQLEOF

```

```

total_logs
-----
          0
(1 row)

 id | time | container | level | message_preview
----+-----+-----+-----+-----
(0 rows)

 container_name | total
-----+-----
(0 rows)

 log_level | total
-----+-----
(0 rows)

 container_name | log_level | count | last_seen
-----+-----+-----+-----
(0 rows)

 timestamp | container_name | message
-----+-----+-----
(0 rows)

 minute | logs_per_minute
-----+-----
(0 rows)

```

Langkah ini digunakan untuk memverifikasi bahwa centralized logging berjalan normal. Query COUNT(*) menghitung total log yang berhasil masuk. Query distribusi container dan level digunakan untuk analisis volume log. View recent_logs menampilkan log terbaru sedangkan error_summary menampilkan ringkasan log level tinggi seperti ERROR dan CRITICAL. Query rate per menit digunakan untuk memonitor traffic logging secara realtime.

4. Gunakan API untuk query log

```

dito@LOG-JUWADI:~/docker-lab/logging$ curl http://localhost:5000/api/logs/stats | python3 -m
json.tool
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total     Spent    Left     Speed
100      32 100      32    0      0    143      0
{
  "last_hour": [],
  "total_logs": 0
}
dito@LOG-JUWADI:~/docker-lab/logging$ curl "http://localhost:5000/api/logs/search?q=error&li
mit=5" | python3 -m json.tool
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total     Spent    Left     Speed
100       3 100       3    0      0     43      0
[]
dito@LOG-JUWADI:~/docker-lab/logging$ curl "http://localhost:5000/api/logs/search?level=CRIT
ICAL&limit=5" | python3 -m json.tool
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total     Spent    Left     Speed
100       3 100       3    0      0    143      0
[]

```

Endpoint /api/logs/stats digunakan untuk melihat statistik log satu jam terakhir dalam format JSON. Endpoint /api/logs/search memungkinkan pencarian log berdasarkan keyword tertentu atau level log tertentu seperti CRITICAL. API ini berguna untuk simulasi dashboard monitoring atau integrasi dengan frontend monitoring system.

5. Log retention cleanup

```

dito@LOG-JUWADI:~/docker-lab/logging$ docker exec postgres-db psql -U labuser -d labdb -c \
"SELECT logs.cleanup_old_logs() AS deleted_rows;"
deleted_rows
-----
          0
(1 row)

```

Langkah ini menjalankan function cleanup untuk menghapus log yang lebih lama dari 30 hari. Maintenance seperti ini penting dalam centralized logging karena volume log dapat bertambah sangat cepat. Cleanup membantu menjaga ukuran database tetap stabil dan performa query tetap optimal.

E. PERTANYAAN

Pre-Lab

1. Mengapa centralized logging penting di lingkungan container?

Centralized logging penting karena container bersifat ephemeral. Saat container restart, crash, atau dihapus, log lokal dapat hilang. Dengan centralized logging, semua log dikirim ke satu tempat terpusat seperti PostgreSQL, Elasticsearch, atau Loki sehingga monitoring, debugging, auditing, dan troubleshooting menjadi lebih mudah. Administrator tidak perlu masuk ke setiap container untuk membaca log karena semua data sudah terkumpul dalam satu sistem.

2. Apa perbedaan antara Docker logging driver json-file dan fluentd?
Driver json-file menyimpan log container ke file JSON lokal di host Docker. Ini adalah default logging driver Docker dan cocok untuk penggunaan sederhana. Namun log tersebar di host dan sulit dianalisis secara terpusat. Driver fluentd mengirim log langsung ke Fluentd atau Fluent Bit melalui network socket sehingga log dapat diproses, difilter, dan diteruskan ke database atau sistem monitoring lain secara real-time. fluentd lebih cocok untuk production dan multi-container environment.
3. Jelaskan keuntungan menyimpan log di database (PostgreSQL) vs file text.
PostgreSQL memungkinkan log di-query menggunakan SQL sehingga pencarian dan analisis jauh lebih mudah dibanding membaca file text manual. Database juga mendukung indexing, aggregation, filtering, dan retention policy. Selain itu data log dapat diintegrasikan dengan dashboard atau API aplikasi. File text lebih sederhana dan ringan, tetapi sulit digunakan untuk analisis kompleks dan pencarian data dalam jumlah besar.
4. Apa itu structured logging dan mengapa lebih baik daripada plain text log?
Structured logging adalah format log berbentuk data terstruktur seperti JSON yang memiliki field jelas seperti timestamp, level, service, dan message. Structured logging memudahkan parsing otomatis, filtering, indexing, dan analytics. Plain text log hanya berupa teks biasa sehingga sulit diproses oleh sistem monitoring modern. Structured log lebih cocok untuk microservices dan container environment karena dapat dibaca manusia maupun mesin.
5. Mengapa Fluent Bit lebih cocok untuk sidecar/edge collection dibanding Fluentd?
Fluent Bit lebih ringan, hemat memori, dan memiliki performa tinggi dibanding Fluentd. Fluent Bit ditulis menggunakan bahasa C sehingga penggunaan resource kecil dan cocok dijalankan sebagai sidecar container atau edge collector pada server dengan resource terbatas. Fluentd lebih fleksibel dan memiliki plugin lebih banyak, tetapi konsumsi memori lebih besar karena berbasis Ruby.

Post-Lab

1. Berapa total log yang masuk ke PostgreSQL setelah 5 menit? Tunjukkan distribusi per container dan per level.

sleep 300 ← untuk menunggu 5 menit

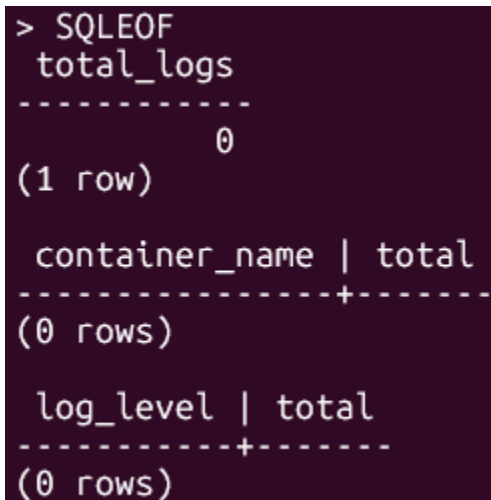
docker exec -i postgres-db psql -U labuser -d labdb << 'SQLEOF'

-- Total log setelah sistem berjalan
SELECT COUNT(*) AS total_logs
FROM logs.container_logs;

-- Distribusi log per container
SELECT
container_name,
COUNT(*) AS total
FROM logs.container_logs
GROUP BY container_name
ORDER BY total DESC;

-- Distribusi log per level
SELECT
log_level,
COUNT(*) AS total
FROM logs.container_logs
GROUP BY log_level
ORDER BY total DESC;

SQLEOF

A terminal window with a dark background and light green text. It shows the output of three SQL queries. The first query returns a single row with 'total_logs' as 0. The second and third queries return 0 rows.

```
> SQLEOF
total_logs
-----
          0
(1 row)

container_name | total
-----+-----
(0 rows)

log_level | total
-----+-----
(0 rows)
```

Penjelasan hasil:

- total_logs menunjukkan jumlah seluruh log yang berhasil masuk ke PostgreSQL.
- distribusi per container menunjukkan container mana yang menghasilkan log terbanyak.
- distribusi per level menunjukkan jumlah log INFO, WARN, ERROR, dan CRITICAL.

2. Tulis query SQL yang menampilkan log rate per menit selama 10 menit terakhir.

```
SELECT
    date_trunc('minute', received_at) AS minute,
    COUNT(*) AS logs_per_minute
FROM logs.container_logs
WHERE received_at > NOW() - INTERVAL '10 minutes'
GROUP BY minute
ORDER BY minute;
```

3. Apa yang terjadi jika container fluent-bit di-stop? Apakah container lain juga stop? Apakah log hilang?

Jika fluent-bit dihentikan, container lain seperti nginx-web, flask-app, dan log-generator tetap berjalan karena mereka independen setelah startup selesai. Namun log baru tidak dapat dikirim ke PostgreSQL karena collector tidak aktif. Bergantung pada konfigurasi logging driver Docker, sebagian log dapat tertahan sementara di buffer atau hilang jika buffer penuh. PostgreSQL tetap menyimpan log lama yang sudah masuk sebelum Fluent Bit berhenti.

4. Jelaskan alur sebuah log entry dari log-generator stdout sampai masuk ke tabel container_logs.

log-generator menghasilkan structured log JSON dan mencetaknya ke stdout. Docker logging driver fluentd menangkap stdout tersebut lalu mengirimkannya ke Fluent Bit pada port 24224. Fluent Bit menerima log melalui input forward, kemudian parser membaca JSON log dan filter memodifikasi field tertentu. Setelah diproses, Fluent Bit meneruskan log ke output PostgreSQL menggunakan plugin pgsqldb. Data akhirnya disimpan ke tabel logs.container_logs di database labdb.

5. Modifikasi LOG_INTERVAL menjadi 0.5 detik. Berapa log rate per menit yang dihasilkan?

Ubah bagian service log-generator pada docker-compose.yml menjadi seperti berikut:

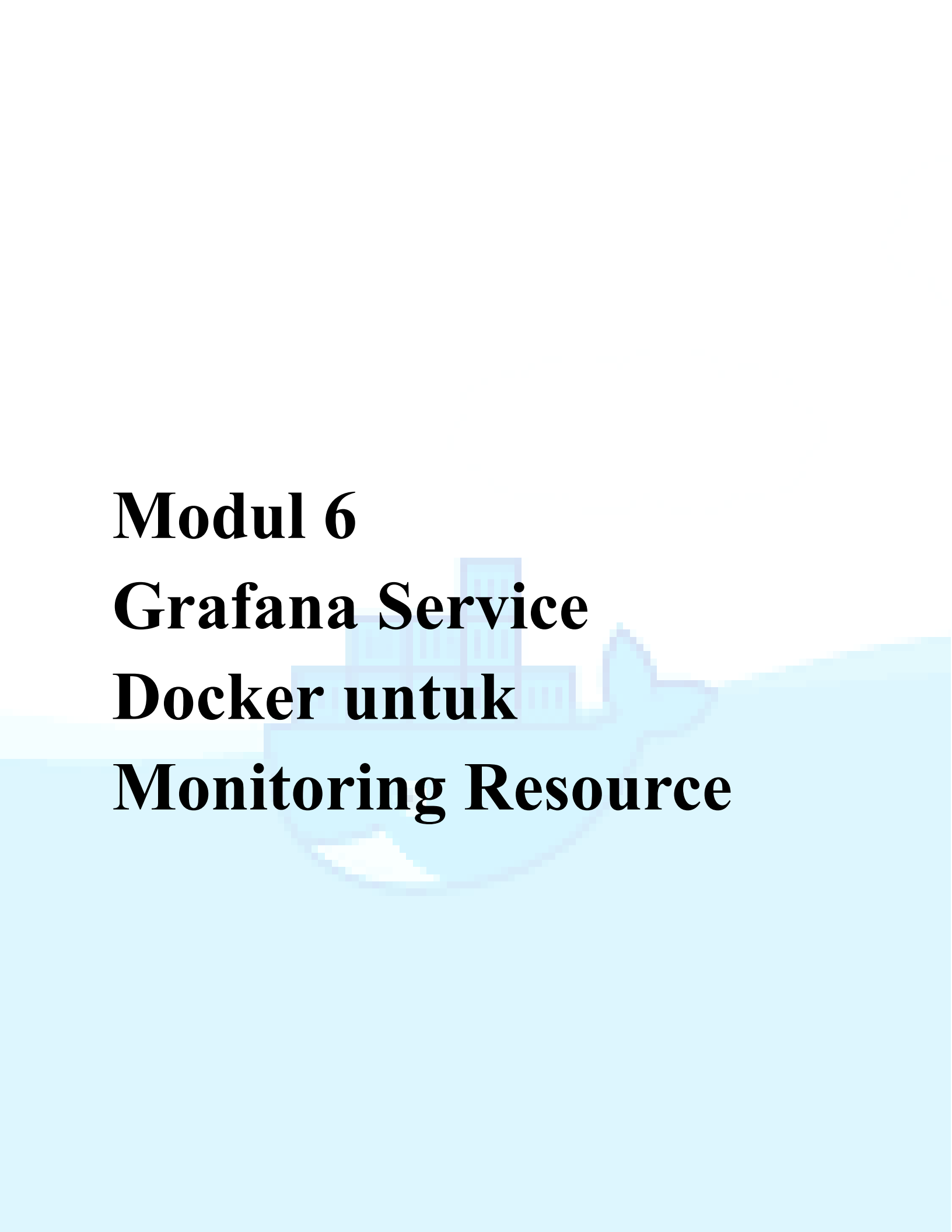
```
# === Log Producer: Generator ===
log-generator:
  build: ./generator
  container_name: log-generator
  environment:
    - LOG_INTERVAL=0.5
  networks:
    - logging-net
  logging:
    driver: fluentd
    options:
      fluentd-address: "localhost:24224"
      fluentd-async: "true"
      tag: "docker.generator"
  depends_on:
    - fluent-bit
  restart: unless-stopped
```

Penjelasan:

- LOG_INTERVAL=0.5 berarti generator membuat log setiap 0.5 detik.
- Semakin kecil interval, semakin tinggi log rate.
- Dengan interval 0.5 detik, estimasi log rate:

$$60 : 0.5 = 120$$

sekitar 120 log per menit.



Modul 6

Grafana Service

Docker untuk

Monitoring Resource

A. TUJUAN PEMBELAJARAN

Setelah praktikum ini, mahasiswa mampu:

1. Memahami arsitektur monitoring berbasis Prometheus + Grafana
2. Men-deploy Prometheus sebagai metrics collector dan time-series database di Docker
3. Men-deploy Node Exporter untuk mengumpulkan metrik host (CPU, RAM, disk, network)
4. Men-deploy cAdvisor untuk mengumpulkan metrik per-container Docker
5. Men-deploy Grafana sebagai dashboard visualization platform di Docker
6. Mengkonfigurasi Prometheus data source di Grafana
7. Membuat dan mengkustomisasi dashboard Grafana: panel, query PromQL, alert threshold
8. Mengintegrasikan log dari PostgreSQL (Modul 5) ke dashboard Grafana
9. Mengorkestrasi seluruh monitoring stack dengan Docker Compose

B. DASAR TEORI

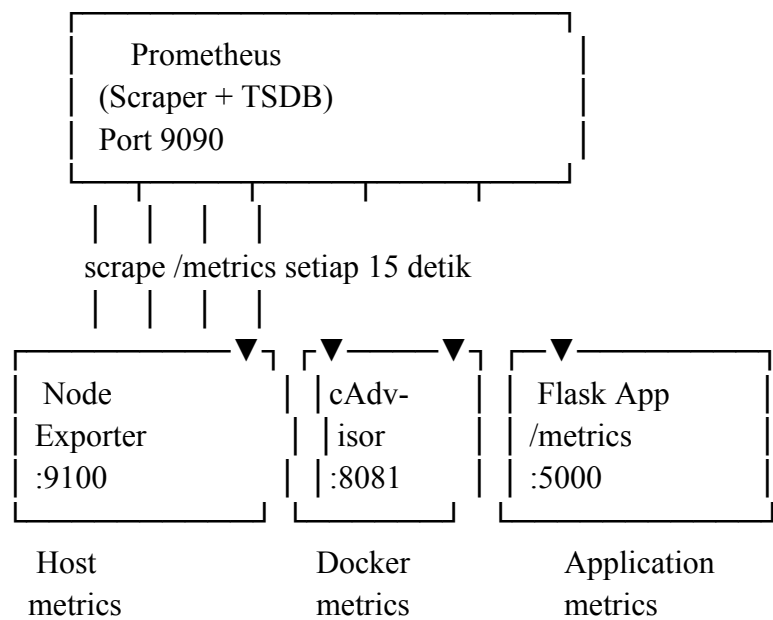
1. Monitoring vs Logging

Aspek	Monitoring (Metrics)	Logging
Data	Numerik time-series (CPU%, memory, request/s)	Teks terstruktur/tidak terstruktur (event, error)
Tujuan	Trend, alerting, capacity planning	Debugging, audit trail, forensik
Volume	Rendah–sedang (angka agregat)	Tinggi (setiap event dicatat)
Query	PromQL, rata-rata, percentile	Full-text search, filter
Retention	Minggu–bulan (downsampled)	Hari–minggu (raw)
Tool	Prometheus, Grafana, Datadog	ELK, Fluent Bit, Loki

Monitoring dan logging saling melengkapi. Monitoring memberi tahu ada masalah (CPU spike, container restart), logging menjelaskan mengapa (error message, stack trace).

2. Arsitektur Prometheus

Prometheus menggunakan model pull-based: Prometheus secara aktif melakukan HTTP GET ke endpoint /metrics dari setiap target pada interval tertentu (scrape). Target meng-expose metrik dalam format text Prometheus.



3. Komponen Monitoring Stack

Komponen	Fungsi	Port	Image
Prometheus	Scrape metrik, simpan time-series, query PromQL	9090	prom/prometheus
Node Exporter	Expose metrik host: CPU, memory, disk, network	9100	prom/node-exporter
cAdvisor	Expose metrik per-container Docker: CPU, mem, I/O	8081	gcr.io/cadvisor/cadvisor

Grafana	Visualisasi dashboard, alerting, data source management	3000	grafana/grafana
---------	---	------	-----------------

4. PromQL (Prometheus Query Language)

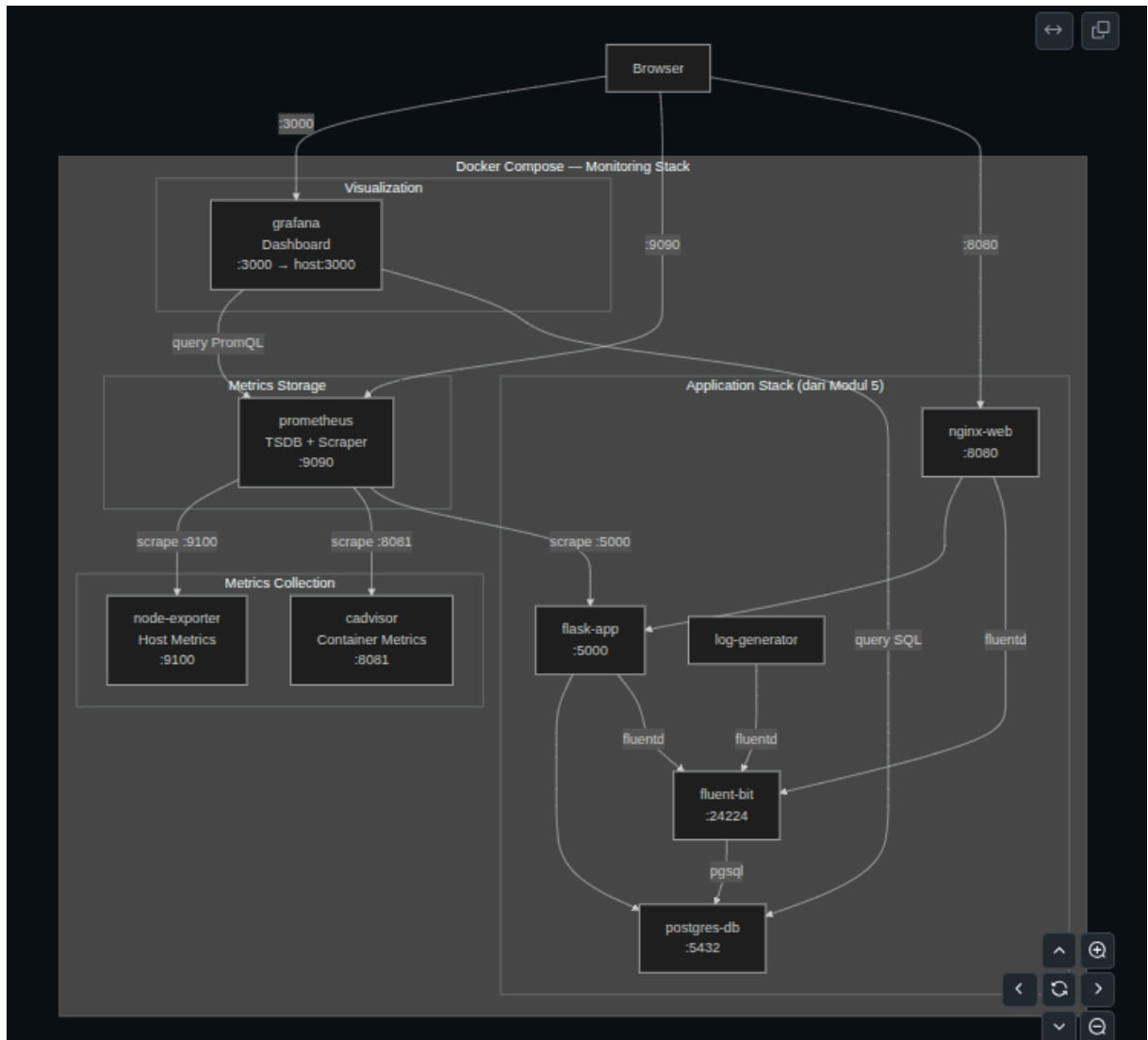
PromQL digunakan untuk query metrik di Prometheus dan Grafana. Beberapa contoh:

Query PromQL	Deskripsi
node_cpu_seconds_total	Total detik CPU per mode (idle, user, system)
rate(node_cpu_seconds_total{mode="idle"}[5m])	Rate idle CPU dalam 5 menit terakhir
100 - (avg(rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100)	CPU usage %
node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes * 100	Memory available %
rate(container_cpu_usage_seconds_total[5m])	CPU usage per container
container_memory_usage_bytes	Memory usage per container

5. Grafana Data Source & Dashboard

Grafana tidak menyimpan data sendiri. Grafana terhubung ke data source (Prometheus, PostgreSQL, Elasticsearch, dll.) dan menampilkan data dalam bentuk panel-panel di dashboard. Dashboard bisa dibuat manual atau diimpor dari Grafana.com menggunakan Dashboard ID.

C. TOPOLOGI LAB



D. PRAKTIKUM

Langkah 0: Persiapan Project

1. Persiapan Project

```
elfandi@elfandi-Latitude-7414:~$ mkdir -p ~/docker-lab/monitoring/{prometheus,grafana/{provisioning/
datasources,provisioning/dashboards,dashboards},app,generator,fluent-bit,init}
elfandi@elfandi-Latitude-7414:~$ cd ~/docker-lab/monitoring
elfandi@elfandi-Latitude-7414:~/docker-lab/monitoring$
```

Perintah ini bertujuan untuk membuat seluruh struktur direktori proyek monitoring berbasis Docker secara sekaligus, termasuk folder untuk konfigurasi Prometheus, Grafana, Fluent-bit, dan aplikasi pendukung

lainnya. Penggunaan parameter `-p` dan teknik brace expansion (`{}`) memungkinkan pembuatan beberapa sub-direktori berlapis secara efisien dalam satu baris perintah. Setelah folder berhasil dibuat, perintah `cd` digunakan untuk berpindah ke dalam direktori utama proyek tersebut agar siap melakukan konfigurasi lebih lanjut.

Langkah 1: Konfigurasi Prometheus

1. Buat konfigurasi scrape

```
elfandi@elfandi-Latitude-7414:~/docker-lab/monitoring$ cat > prometheus/prometheus.yml << 'EOF'
# =====
# Prometheus Configuration
# =====
global:
  scrape_interval: 15s      # Scrape setiap 15 detik
  evaluation_interval: 15s  # Evaluasi rules setiap 15 detik
  scrape_timeout: 10s

# =====
# Alerting Rules
# =====
rule_files:
  - "alert_rules.yml"

# =====
# Scrape Targets
# =====
scrape_configs:

# --- Prometheus self-monitoring ---
EOF
elfandi@elfandi-Latitude-7414:~/docker-lab/monitoring$
```

File `prometheus.yml` ini dikonfigurasi untuk mengatur sistem pemantauan utama (Prometheus), dengan menentukan interval pengambilan data (scrape) dan evaluasi metrik setiap 15 detik. Selain itu, konfigurasi ini berfungsi untuk memuat file aturan peringatan (`alert_rules.yml`) dan mendefinisikan target layanan yang akan dipantau secara berkala.

2. Buat alert rules

```

elrand@getrand-Latitude-7414:~/docker-lab/monitoring$ cat > prometheus/alert_rules.yml << 'EOF'
groups:
- name: host_alerts
  rules:
    # CPU usage > 80% selama 2 menit
    - alert: HighCpuUsage
      expr: 100 - (avg(rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100) > 80
      for: 2m
      labels:
        severity: warning
      annotations:
        summary: "CPU usage tinggi ({{ $value | printf \"%.1f\" }}%)"
        description: "CPU usage di atas 80% selama lebih dari 2 menit."

    # Memory available < 20%
    - alert: LowMemoryAvailable
      expr: (node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes * 100) < 20
      for: 2m
      labels:
        severity: warning
      annotations:
        summary: "Memory tersedia rendah ({{ $value | printf \"%.1f\" }}%)"

    # Disk usage > 85%
    - alert: HighDiskUsage
      expr: 100 - (node_filesystem_avail_bytes{mountpoint="/"} / node_filesystem_size_bytes{mountpoint="/"} * 100) > 85
      for: 5m
      labels:
        severity: critical
      annotations:
        summary: "Disk usage tinggi ({{ $value | printf \"%.1f\" }}%)"

- name: container_alerts
  rules:
    # Container restart > 3 kali dalam 15 menit
    - alert: ContainerRestartFrequent
      expr: increase(container_restart_count[15m]) > 3
      for: 1m
      labels:
        severity: critical
      annotations:
        summary: "Container {{ $labels.name }} restart berulang"

    # Container memory > 256MB
    - alert: ContainerHighMemory
      expr: container_memory_usage_bytes{name!=""} > 268435456
      for: 5m
      labels:
        severity: warning
      annotations:
        summary: "Container {{ $labels.name }} memory > 256MB"
EOF

```

File alert_rules.yml ini bertujuan untuk mendefinisikan aturan peringatan (alerting rules) yang memantau ambang batas performa sistem pada host dan kontainer secara otomatis. Konfigurasi ini mengatur agar Prometheus memicu notifikasi jika terjadi kondisi kritis, seperti penggunaan CPU tinggi (>80%), sisa memori rendah (<20%), kepenuhan disk (>85%), serta aktivitas kontainer yang tidak stabil.

Langkah 2: Konfigurasi Grafana (Provisioning)

1. Provisioning data source

```
elfandigelfandi-Latitude-7414:~/docker-lab/monitoring$ cat > grafana/provisioning/datasources/datasources.yml << 'EOF'
apiVersion: 1

datasources:
  # --- Prometheus (metrics) ---
  - name: Prometheus
    type: prometheus
    access: proxy
    url: http://prometheus:9090
    isDefault: true
    editable: true
    jsonData:
      timeInterval: "15s"

  # --- PostgreSQL (logs dari Modul 5) ---
  - name: PostgreSQL-Logs
    type: postgres
    access: proxy
    url: postgres-db:5432
    database: labdb
    user: labuser
    secureJsonData:
      password: labpass123
    jsonData:
      sslmode: disable
      maxOpenConns: 5
      postgresVersion: 1600
      timescaledb: false
    editable: true
EOF
```

File `datasources.yml` ini bertujuan untuk mendaftarkan sumber data (data sources) secara otomatis ke dalam Grafana agar metrik dan log dapat divisualisasikan. Konfigurasi ini menghubungkan Grafana dengan Prometheus untuk menarik data metrik sistem, serta menghubungkan database PostgreSQL (labdb) untuk memantau log yang tersimpan di dalamnya.

2. Provisioning dashboard

```
elfandigelfandi-Latitude-7414:~/docker-lab/monitoring$ cat > grafana/provisioning/dashboards/dashboards.yml << 'EOF'
apiVersion: 1

providers:
  - name: "Lab PENS Dashboards"
    orgId: 1
    folder: "Lab PENS"
    type: file
    disableDeletion: false
    editable: true
    updateIntervalSeconds: 30
    options:
      path: /var/lib/grafana/dashboards
      foldersFromFilesStructure: false
EOF
```

File `dashboards.yml` ini berfungsi untuk mengatur penyediaan (provisioning) dashboard secara otomatis di Grafana dari file lokal yang tersimpan di dalam sistem. Konfigurasi ini mendefinisikan provider bernama "Lab PENS

Dashboards" untuk memuat file-file dashboard dari path /var/lib/grafana/dashboards dan mengelompokkannya ke dalam folder bernama "Lab PENS".

3. Buat Dashboard JSON: Docker Host Overview

```
elfandi@elfandi-Latitude-7414:~/docker-lab/monitoring$ cat > grafana/dashboards/docker-host-overview.json << 'JSONEOF'
{
  "annotations": { "list": [] },
  "editable": true,
  "fiscalYearStartMonth": 0,
  "graphTooltip": 1,
  "links": [],
  "panels": [
    {
      "title": "CPU Usage %",
      "type": "gauge",
      "gridPos": { "h": 6, "w": 6, "x": 0, "y": 0 },
      "datasource": { "type": "prometheus", "uid": "" },
      "fieldConfig": {
        "defaults": {
          "thresholds": {
            "mode": "absolute",
            "steps": [
              { "color": "green", "value": null },
              { "color": "yellow", "value": 60 },
              { "color": "red", "value": 85 }
            ]
          },
          "unit": "percent", "min": 0, "max": 100
        },
        "overrides": []
      },
      "targets": [
        {
          "expr": "100 - (avg(rate(node_cpu_seconds_total{node=\"idle\"}[5m])) * 100)",
          "legendFormat": "CPU Usage"
        }
      ]
    },
    {
      "title": "Memory Usage %",
      "type": "gauge",
      "gridPos": { "h": 6, "w": 6, "x": 6, "y": 0 },
      "datasource": { "type": "prometheus", "uid": "" },
      "fieldConfig": {
        "defaults": {
          "thresholds": {
            "mode": "absolute",
            "steps": [
              { "color": "green", "value": null },
              { "color": "yellow", "value": 70 },
              { "color": "red", "value": 90 }
            ]
          },
          "unit": "percent", "min": 0, "max": 100
        },
        "overrides": []
      },
      "targets": [
        {
          "expr": "penc-host-overview", "now": "tes_total[5m]", "legendFormat": "Write {{ device }}"
        }
      ]
    }
  ]
}
```

File docker-host-overview.json ini merupakan templat konfigurasi berbasis JSON yang bertujuan untuk menyusun tata letak (layout) visualisasi grafik pada dashboard Grafana. Di dalamnya ditentukan pengaturan komponen visual (panels) berupa indikator gauge serta kueri data ke Prometheus untuk memantau metrik penggunaan CPU dan memori secara real-time.

4. Buat Dashboard JSON: Container Metrics

```

alfandigelfandi-Latitude-7414:~/docker-lab/monitoring$ cat > grafana/dashboards/container-metrics.json << 'JSONEOF'
{
  "annotations": { "list": [ ] },
  "editable": true,
  "panels": [
    {
      "title": "Running Containers",
      "type": "stat",
      "gridPos": { "h": 4, "w": 6, "x": 0, "y": 0 },
      "datasource": { "type": "prometheus", "uid": "" },
      "fieldConfig": { "defaults": { "color": { "mode": "thresholds" }, "thresholds": { "steps": [ { "color": "green", "value":
": null } ] } }, "overrides": [ ] },
      "targets": [ { "expr": "count(container_last_seen{name!=\"\"})", "legendFormat": "Containers" } ]
    },
    {
      "title": "Total Container CPU Usage",
      "type": "stat",
      "gridPos": { "h": 4, "w": 6, "x": 6, "y": 0 },
      "datasource": { "type": "prometheus", "uid": "" },
      "fieldConfig": { "defaults": { "unit": "percent", "thresholds": { "steps": [ { "color": "green", "value": null }, { "co
lor": "red", "value": 80 } ] } }, "overrides": [ ] },
      "targets": [ { "expr": "sum(rate(container_cpu_usage_seconds_total{name!=\"\"}[5m])) * 100", "legendFormat": "Total CPU
" } ]
    },
    {
      "title": "Total Container Memory",
      "type": "stat",
      "gridPos": { "h": 4, "w": 6, "x": 12, "y": 0 },
      "datasource": { "type": "prometheus", "uid": "" },
      "fieldConfig": { "defaults": { "unit": "bytes", "thresholds": { "steps": [ { "color": "green", "value": null } ] } }, "o
verrides": [ ] },
      "targets": [ { "expr": "sum(container_memory_usage_bytes{name!=\"\"})", "legendFormat": "Total Memory" } ]
    },
    {
      "title": "Prometheus Alerts Active",
      "type": "stat",
      "gridPos": { "h": 4, "w": 6, "x": 18, "y": 0 },
      "datasource": { "type": "prometheus", "uid": "" },
      "fieldConfig": { "defaults": { "thresholds": { "steps": [ { "color": "green", "value": null }, { "color": "red", "value
": 1 } ] } }, "overrides": [ ] },
      "targets": [ { "expr": "count(ALERTS{alertstate=\"firing\"}) OR vector(0)", "legendFormat": "Firing" } ]
    },
    {
      "title": "CPU Usage per Container",
      "type": "timeseries",
      "gridPos": { "h": 8, "w": 12, "x": 0, "y": 4 },
      "datasource": { "type": "prometheus", "uid": "" },
      "fieldConfig": { "defaults": { "unit": "percent" }, "overrides": [ ] },
      "targets": [ { "expr": "rate(container_cpu_usage_seconds_total{name!=\"\"}[5m]) * 100", "legendFormat": "{{ name }}" } ]
    },
    {
      "title": "Memory Usage per Container",
      "type": "timeseries",
      "gridPos": { "h": 8, "w": 12, "x": 0, "y": 12 },
      "datasource": { "type": "prometheus", "uid": "" },
      "fieldConfig": { "defaults": { "unit": "bytes" }, "overrides": [ ] },
      "targets": [ { "expr": "rate(container_memory_usage_bytes_total{name!=\"\"}[5m])", "legendFormat": "{{ name }}" } ]
    }
  ]
}
JSONEOF: "pens-container-metrics": "now" }, network_transmit_bytes_total{name!=\"\"}[5m]", "legendFormat": "{{ name }}" } ]

```

File container-metrics.json ini merupakan templat konfigurasi berbasis JSON yang bertujuan untuk membangun dashboard Grafana khusus untuk memantau performa kontainer Docker. Di dalamnya diatur tata letak visualisasi untuk menampilkan data statistik jumlah kontainer yang aktif, total penggunaan CPU dan memori, jumlah peringatan (alerts) yang sedang aktif, serta grafik tren penggunaan sumber daya per kontainer secara real-time.

5. Buat Dashboard JSON: Log Analytics (PostgreSQL)

```

alFandigelfand@Latitude-7414: /docker/lab/monitoring$ cat > grafana/dashboards/log-analytics.json << 'JSONEOF'
{
  "annotations": { "list": [] },
  "editable": true,
  "panels": [
    {
      "title": "Total Logs (All Time)",
      "type": "stat",
      "gridPos": { "h": 4, "w": 6, "x": 0, "y": 0 },
      "datasource": { "type": "postgres", "uid": "" },
      "fieldConfig": { "defaults": { "thresholds": { "steps": [{ "color": "blue", "value": null } ] }, "overrides": [ ] },
      "targets": [
        {
          "rawSql": "SELECT COUNT(*) AS total FROM logs.container_logs;",
          "format": "table"
        }
      ]
    },
    {
      "title": "Errors Last Hour",
      "type": "stat",
      "gridPos": { "h": 4, "w": 6, "x": 6, "y": 0 },
      "datasource": { "type": "postgres", "uid": "" },
      "fieldConfig": { "defaults": { "thresholds": { "steps": [
        { "color": "green", "value": null },
        { "color": "red", "value": 10 }
      ] } }, "overrides": [ ] },
      "targets": [
        {
          "rawSql": "SELECT COUNT(*) AS errors FROM logs.container_logs WHERE log_level IN ('ERROR','CRITICAL') AND received_at > NOW() - INTERVAL '1 hour';",
          "format": "table"
        }
      ]
    },
    {
      "title": "Log Volume per Minute",
      "type": "timeseries",
      "gridPos": { "h": 8, "w": 24, "x": 0, "y": 4 },
      "datasource": { "type": "postgres", "uid": "" },
      "targets": [
        {
          "rawSql": "SELECT date_trunc('minute', received_at) AS time, COUNT(*) AS count FROM logs.container_logs WHERE $__timeFilter(received_at) GROUP BY time ORDER BY time;",
          "format": "time_series"
        }
      ]
    },
    {
      "title": "Log Level Distribution",
      "type": "piechart",
      "gridPos": { "h": 8, "w": 8, "x": 0, "y": 12 },
      "datasource": { "type": "postgres", "uid": "" },
      "targets": [
        {
          "rawSql": "SELECT log_level AS metric, COUNT(*) AS value FROM logs.container_logs WHERE received_at > NOW() - INTERVAL '1 hour' GROUP BY log_level ORDER BY value DESC;",
          "format": "table"
        }
      ]
    },
    {
      "title": "Logs per Container",
      "rawSql": "pens-log-analytics"stgresQL", " " }, ( 'ERROR','CRITICAL') ORDER BY timestamp DESC LIMIT 20;" ,message, 120) AS message
      "type": "table",
      "gridPos": { "h": 8, "w": 24, "x": 0, "y": 20 },
      "datasource": { "type": "postgres", "uid": "" },
      "targets": [
        {
          "rawSql": "SELECT container_id AS metric, COUNT(*) AS value FROM logs.container_logs WHERE received_at > NOW() - INTERVAL '1 hour' GROUP BY container_id ORDER BY value DESC LIMIT 20;"
        }
      ]
    }
  ]
}
JSONEOF

```

File log-analytics.json ini merupakan templat konfigurasi berbasis JSON yang bertujuan untuk membuat dashboard analisis log di Grafana dengan mengambil data langsung dari database PostgreSQL. Konfigurasi ini menyusun panel visualisasi untuk menampilkan total akumulasi log, jumlah error dalam satu jam terakhir, volume log per menit, hingga distribusi tingkatan log (log level) menggunakan kueri SQL.

Langkah 3: Buat Flask App dengan Prometheus Metrics

```

elfandi@elfandi-Latitude-7414:~/docker-lab/monitoring$ cat > app/requirements.txt << 'EOF'
flask==3.1.*
psycopg2-binary==2.9.*
prometheus-client==0.21.*
EOF

cat > app/app.py << 'PYEOF'
"""Flask app dengan Prometheus metrics endpoint dan structured logging."""
import os, json, socket, datetime, logging, sys, time
from flask import Flask, jsonify, request, Response
import psycopg2
from prometheus_client import Counter, Histogram, Gauge, generate_latest, CONTENT_TYPE_LATEST

app = Flask(__name__)

# --- Prometheus Metrics ---
REQUEST_COUNT = Counter(
    "flask_http_requests_total",
    "Total HTTP requests",
    ["method", "endpoint", "status"]
)
REQUEST_LATENCY = Histogram(
    "flask_http_request_duration_seconds",
    "HTTP request latency",
    ["method", "endpoint"],
    buckets=[0.01, 0.05, 0.1, 0.25, 0.5, 1.0, 2.5, 5.0]
)
DB_CONNECTIONS = Gauge(
    "flask_db_connections_active",
    "Active database connections"
)
LOG_COUNT = Gauge(
    "flask_log_total_count",
    "Total logs in PostgreSQL"
)

# --- Structured JSON Logging ---
class JSONFormatter(logging.Formatter):
    def format(self, record):
        return json.dumps({
            "timestamp": datetime.datetime.now().isoformat(),
            "level": record.levelname,
            "hostname": socket.gethostname(),
            "service": "flask-app",
            "message": record.getMessage()
        })

handler = logging.StreamHandler(sys.stdout)
handler.setFormatter(JSONFormatter())
app.logger.handlers = [handler]
app.logger.setLevel(logging.INFO)

DB = dict(host=os.environ.get("DB_HOST", "postgres-db"),
EOF ["python", "-u", "app.py"] -r requirements.txt_hour": stats}}hall()]}gsnected"}][dbname"],))

```

Perintah ini bertujuan untuk menyiapkan dependensi dan kode utama aplikasi backend berbasis Flask agar terintegrasi dengan sistem pemantauan. File requirements.txt digunakan untuk memasang pustaka utama seperti Flask, driver PostgreSQL (psycopg2), dan klien Prometheus. Sementara itu, file app.py dikonfigurasi untuk mengekspos metrik khusus aplikasi (seperti jumlah request, latensi, dan koneksi database) serta mengaktifkan format pencatatan log terstruktur berbasis JSON (structured logging).

Langkah 4: Siapkan Fluent Bit, Log Generator, dan Init Script


```

elfandi@elfandi-Latitude-7414:~/docker-lab/monitoring$ # --- Fluent Bit ---
cat > fluent-bit/fluent-bit.conf << 'EOF'
[SERVICE]
    Flush      5
    Daemon     Off
    Log_Level   info
    Parsers_File parsers.conf

[INPUT]
    Name        forward
    Listen      0.0.0.0
    Port        24224
    Tag          docker.*

[FILTER]
    Name        modify
    Match        docker.*
    Rename       log message

[OUTPUT]
    Name        pgsqldb
    Match        docker.*
    Host        postgres-db
    Port        5432
    User        labuser
    Password    labpass123
    Database     labdb
    Table        container_logs
    Schema       logs
    Timestamp_Key timestamp
    Async        false
    min_pool_size 1
    max_pool_size 4

[OUTPUT]
    Name        stdout
    Match        docker.*
    Format        json_lines
EOF

cat > fluent-bit/parsers.conf << 'EOF'
[PARSER]
    Name        docker_json
    Format        json
    Time_Key     time
    Time_Format  %Y-%m-%dT%H:%M:%S.%L
    Time_Keep    On
EOF

# --- Log Generator ---
cat > generator/generator.py << 'PYEOF'
import json, time, random, socket, datetime, os

HOSTNAME = socket.gethostname()
EOFUP BY container_name, log_level ORDER BY count DESC;WARN', 'CRITICAL')st_seenssage_preview

```

File fluent-bit.conf dan parsers.conf dikonfigurasi untuk mengatur Fluent Bit sebagai agen pengumpul dan penerus log (log forwarder) dari kontainer Docker. Konfigurasi ini berfungsi menerima log melalui protokol forward, menyaring dan memformatnya menjadi struktur JSON yang sesuai, lalu mengirimkannya ke dua tujuan (outputs): disimpan ke database PostgreSQL (labdb) dan ditampilkan ke stdout. Sementara itu, file generator.py disiapkan untuk bertindak sebagai log generator otomatis guna menguji keandalan alur pengumpulan log tersebut.

Langkah 5: Docker Compose — Full Monitoring Stack

```

elrand@elrand-Latitude-7414:~/docker-lab/monitoring$ cat > docker-compose.yml << 'EOF'
services:

# =====
# MONITORING LAYER
# =====

# --- Prometheus (Metrics TSDB) ---
prometheus:
  image: prom/prometheus:latest
  container_name: prometheus
  ports:
    - "9090:9090"
  volumes:
    - ./prometheus/prometheus.yml:/etc/prometheus/prometheus.yml:ro
    - ./prometheus/alert_rules.yml:/etc/prometheus/alert_rules.yml:ro
    - prom-data:/prometheus
  command:
    - "--config.file=/etc/prometheus/prometheus.yml"
    - "--storage.tsdb.path=/prometheus"
    - "--storage.tsdb.retention.time=7d"
    - "--web.enable-lifecycle"
  networks:
    - monitoring-net
  restart: unless-stopped

# --- Node Exporter (Host Metrics) ---
node-exporter:
  image: prom/node-exporter:latest
  container_name: node-exporter
  ports:
    - "9100:9100"
  volumes:
    - /proc:/host/proc:ro
    - /sys:/host/sys:ro
    - /:/rootfs:ro
  command:
    - "--path.procfs=/host/proc"
    - "--path.sysfs=/host/sys"
    - "--path.rootfs=/rootfs"
    - "--collector.filesystem.mount-points-exclude=^/(sys|proc|dev|host|etc)($|/)"
  networks:
    - monitoring-net
  restart: unless-stopped

# --- cAdvisor (Container Metrics) ---
cadvisor:
  image: gcr.io/cadvisor/cadvisor:latest
  container_name: cadvisor
  ports:
    - "8081:8080"
  volumes:
    - /:/rootfs:ro
    - /var/run:/var/run:ro
    - /sys:/sys:ro
EOF

```

File `docker-compose.yml` ini bertujuan untuk mendefinisikan dan mengonfigurasi arsitektur layanan pada lapisan pemantauan (monitoring layer) di dalam lingkungan Docker. Konfigurasi ini mendeskripsikan spesifikasi kontainer untuk Prometheus (sebagai database metrik), Node Exporter (pengumpul metrik infrastruktur host), dan cAdvisor (pengumpul metrik performa kontainer) agar dapat berjalan bersama dalam satu jaringan terisolasi (monitoring-net).

Langkah 6: Deploy dan Verifikasi

1. Build dan jalankan

```
app docker-compose.yml fluent-bit generator grafana init prometheus
elfandi@elfandi-Latitude-7414:~/docker-lab/monitoring$ sudo nano docker-compose.yml
elfandi@elfandi-Latitude-7414:~/docker-lab/monitoring$ docker compose up --build -d
[+] Building 1.6s (21/21) FINISHED
=> [internal] load local bake definitions
=> => reading from stdin 1.04kB
=> [log-generator internal] load build definition from Dockerfile
=> => transferring dockerfile: 131B
=> [flask-app internal] load build definition from Dockerfile
=> => transferring dockerfile: 206B
=> [flask-app internal] load metadata for docker.io/library/python:3.11-slim
=> [log-generator internal] load metadata for docker.io/library/python:3.11-alpine
=> [flask-app internal] load .dockerignore
=> => transferring context: 2B
=> [flask-app 1/5] FROM docker.io/library/python:3.11-slim@sha256:9a7765b36773a370614
=> => resolve docker.io/library/python:3.11-slim@sha256:9a7765b36773a37061455b332f18e
=> [flask-app internal] load build context
=> => transferring context: 63B
=> [log-generator internal] load .dockerignore
=> => transferring context: 2B
=> [log-generator 1/3] FROM docker.io/library/python:3.11-alpine@sha256:8b5bfdb1fd2d7
=> => resolve docker.io/library/python:3.11-alpine@sha256:8b5bfdb1fd2d78aa94e21c4d61b
=> [log-generator internal] load build context
=> => transferring context: 34B
=> CACHED [flask-app 2/5] WORKDIR /app
=> CACHED [flask-app 3/5] COPY requirements.txt .
=> CACHED [flask-app 4/5] RUN pip install --no-cache-dir -r requirements.txt
=> CACHED [flask-app 5/5] COPY app.py .
=> [flask-app] exporting to image
=> => exporting layers
=> => exporting manifest sha256:1ff311c5b117c0571fe3d33764765b2093cceb8ee9e27fc19b6d9
=> => exporting config sha256:39e26ed3a2ccc2a94736db9216ffcc40f13630fe49e7e388ddc958f
=> => exporting attestation manifest sha256:d797d8dd74ad061af6befab45bdc50f81979bfe11
=> => exporting manifest list sha256:7a1cd07d76f21dbec865d9627232f5c59ce66d975b26d966
=> => naming to docker.io/library/monitoring-flask-app:latest
=> => unpacking to docker.io/library/monitoring-flask-app:latest
=> CACHED [log-generator 2/3] WORKDIR /app
=> CACHED [log-generator 3/3] COPY generator.py .
=> [log-generator] exporting to image
=> => exporting layers
=> => exporting manifest sha256:3e6cf1fc8acf82bd262721fbbe5d6378a7cea4ad586e12c672813
=> => exporting config sha256:abefb93db7716da9571e23bb2c3b5ef9ccf4fd1ac475bb5efbbcf02
=> => exporting attestation manifest sha256:628cb1461ccabfc9c1d1380c19544ecf04918e0fc
=> => exporting manifest list sha256:711674a6128ed14c0152c25f9a34e5a92a66f3da1c19c296
=> => naming to docker.io/library/monitoring-log-generator:latest
=> => unpacking to docker.io/library/monitoring-log-generator:latest
=> [flask-app] resolving provenance for metadata file
=> [log-generator] resolving provenance for metadata file
[+] up 11/11
✓ Image monitoring-log-generator Built
✓ Image monitoring-flask-app Built
✓ Container cadvisor Running
✓ Container prometheus Running
✓ Container node-exporter Running
✓ Container postgres-db Healthy
✓ Container flask-app Started
```

Perintah docker compose up --build -d digunakan untuk membangun ulang (build) image kustom dari Dockerfile yang tersedia sekaligus menjalankan seluruh layanan monitoring stack yang terdefinisi di dalam file

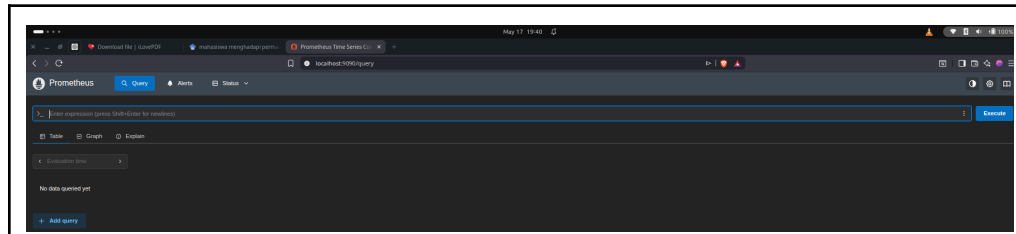
docker-compose.yml. Opsi --build memastikan perubahan kode terbaru pada aplikasi (seperti flask-app dan log-generator) ikut diperbarui, sementara opsi -d (detached mode) membuat seluruh kontainer berjalan di latar belakang. Proses pada terminal menunjukkan bahwa tahap pembuatan layer, penyalinan file, instalasi dependensi, hingga ekspor image baru telah berhasil diselesaikan (FINISHED).

```
utes ago      Up About a minute      2020/tcp, 0.0.0.0:24224->24224/tcp, 0.0.0.0:24224->24224/udp, [::]:24224->24224/tcp, [::]:24224->24224/udp
grafana        grafana/grafana:latest    "/run.sh"          grafana      3 min
utes ago      Up About a minute      0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp
log-generator  monitoring-log-generator  "python -u generator..." log-generator About
a minute ago  Up About a minute
nginx-web      nginx:alpine              "/docker-entrypoint..." nginx-web     3 min
utes ago      Up About a minute      0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
node-exporter  prom/node-exporter:latest "/bin/node_exporter ..." node-exporter 3 min
utes ago      Up 3 minutes           0.0.0.0:9100->9100/tcp, [::]:9100->9100/tcp
postgres-db    postgres:16-alpine        "docker-entrypoint.s..." postgres-db   About
a minute ago  Up About a minute (healthy) 0.0.0.0:5433->5432/tcp, [::]:5433->5432/tcp
prometheus     prom/prometheus:latest    "/bin/prometheus --c..." prometheus    3 min
utes ago      Up 3 minutes           0.0.0.0:9090->9090/tcp, [::]:9090->9090/tcp
elfandigelfandi-Latitude-7414:~/docker-lab/monitoring$ sleep 30
```

Output perintah docker ps ini menunjukkan daftar kontainer yang sedang berjalan aktif dalam monitoring stack, lengkap dengan status operasional (Up), image yang digunakan, serta pemetaan port (port mapping) masing-masing layanan. Seluruh komponen utama seperti Grafana, Prometheus, Node Exporter, PostgreSQL (postgres-db), hingga log-generator dan nginx-web telah berhasil aktif dan siap digunakan. Status (healthy) pada kontainer database menandakan bahwa layanan tersebut telah lolos uji kesiapan internal dan berjalan dengan stabil.

Langkah 7: Verifikasi Setiap Komponen

1. Verifikasi Prometheus



Gambar ini menunjukkan antarmuka web dashboard utama dari Prometheus yang diakses melalui URL `localhost:9090/query`. Halaman ini menyediakan fitur Expression Browser yang berfungsi sebagai tempat mengeksekusi kueri berbasis PromQL (Prometheus Query Language) untuk menganalisis metrik data yang telah dikumpulkan. Pengguna dapat memanfaatkan tab Table atau Graph pada bagian bawah kolom ekspresi untuk melihat hasil analisis data baik dalam bentuk angka digital maupun grafik visual.

```
elfandi@elfandi-Latitude-7414:~/docker-lab/monitoring$ curl -s http://localhost:9090/api/v1/targets |  
python3 -m json.tool  
{  
  "status": "success",  
  "data": {  
    "activeTargets": [  
      {  
        "discoveredLabels": {  
          "__address__": "cadvisor:8080",  
          "__metrics_path__": "/metrics",  
          "__scheme__": "http",  
          "__scrape_interval__": "15s",  
          "__scrape_timeout__": "10s",  
          "instance": "docker-containers",  
          "job": "cadvisor"  
        },  
        "labels": {  
          "instance": "docker-containers",  
          "job": "cadvisor"  
        },  
        "scrapePool": "cadvisor",  
        "scrapeUrl": "http://cadvisor:8080/metrics",  
        "globalUrl": "http://cadvisor:8080/metrics",  
        "lastError": "",  
        "lastScrape": "0001-01-01T00:00:00Z",  
        "lastScrapeDuration": 0,  
        "health": "unknown",  
        "scrapeInterval": "15s",  
        "scrapeTimeout": "10s"  
      },  
      {  
        "discoveredLabels": {  
          "__address__": "flask-app:5000",  
          "__metrics_path__": "/metrics",  
          "__scheme__": "http",  
          "__scrape_interval__": "15s",  
          "__scrape_timeout__": "10s",  
          "instance": "flask-backend",  
          "job": "flask-app"  
        },  
        "labels": {  
          "instance": "flask-backend",  
          "job": "flask-app"  
        },  
        "scrapePool": "flask-app",  
        "scrapeUrl": "http://flask-app:5000/metrics",  
        "globalUrl": "http://flask-app:5000/metrics",  
        "lastError": "",  
        "lastScrape": "0001-01-01T00:00:00Z",  
        "lastScrapeDuration": 0,  
        "health": "unknown",  
        "scrapeInterval": "15s",  
        "scrapeTimeout": "10s"  
      }  
    ]  
  }  
}
```

Perintah curl ini digunakan untuk memeriksa status dari seluruh target pemantauan (active targets) secara langsung melalui HTTP API endpoint milik Prometheus (/api/v1/targets). Output berformat JSON tersebut menunjukkan bahwa Prometheus berhasil menemukan dan mendaftarkan layanan cadvisor (pemantau kontainer) dan flask-app (aplikasi backend) sebagai target yang akan diambil metriknya secara berkala. Di dalam setiap objek target, terdapat informasi detail operasional seperti alamat URL pencuplikan (scrapeUrl), interval waktu pengambilan data (15s), serta label identitas yang disematkan pada masing-masing layanan.

```
elfandi@elfandi-Latitude-7414:~/docker-lab/monitoring$ curl -G http://localhost:9090/api/v1/query \
--data-urlencode 'query=100-(avg(rate(node_cpu_seconds_total{mode="idle"}[5m]))*100)' \
| python3 -m json.tool
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload  Total   Spent    Left     Speed
100    121    100    121     0     0   43245      0 --:--:-- --:--:-- --:--:-- 60500
{
  "status": "success",
  "data": {
    "resultType": "vector",
    "result": [
      {
        "metric": {},
        "value": [
          1779021768.376,
          "34.60068841380621"
        ]
      }
    ]
  }
}
```

Perintah `curl -G` dengan opsi `--data-urlencode` digunakan untuk mengirimkan kueri PromQL ke HTTP API Prometheus secara aman dengan mengubah karakter khusus menjadi format URL (URL encoding). Output JSON yang dihasilkan menunjukkan bahwa kueri berhasil dieksekusi (`"status": "success"`) dengan tipe data berupa instant vector. Di dalam blok data, terdapat nilai numerik 34.60 yang merepresentasikan persentase rata-rata penggunaan CPU pada saat metrik tersebut diambil.

```
elfandi@elfandi-Latitude-7414:~/docker-lab/monitoring$ curl -s "http://localhost:9090/api/v1/query?que
ry=node_memory_MemAvailable_bytes" \
| python3 -m json.tool
{
  "status": "success",
  "data": {
    "resultType": "vector",
    "result": [
      {
        "metric": {
          "__name__": "node_memory_MemAvailable_bytes",
          "instance": "docker-host",
          "job": "node-exporter"
        },
        "value": [
          1779021839.104,
          "4189196288"
        ]
      }
    ]
  }
}
```

Perintah `curl` ini bertujuan untuk mengambil metrik kapasitas memori yang masih tersedia (available memory) pada host melalui HTTP API Prometheus dengan kueri `node_memory_MemAvailable_bytes`. Respons JSON yang diterima menunjukkan status pencarian sukses dan menampilkan data yang dikumpulkan oleh layanan Node Exporter dari instansi bernama `docker-host`. Nilai numerik "4189196288" di dalam blok hasil merepresentasikan jumlah sisa memori yang tersedia dalam satuan bytes (atau setara dengan sekitar 4,19 GB) pada saat kueri tersebut dieksekusi.

```

elfandi@elfandi-Latitude-7414:~/docker-lab/monitoring$ curl -G http://localhost:9090/api/v1/query \
--data-urlencode 'query=count(container_last_seen)' \
| python3 -m json.tool
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload  Total   Dload  Upload  Total   Dload  Upload    Spent    Left     Speed
100    107    100    107    0     0    9089      0  --:--:-- --:--:-- --:--:--   9727
{
  "status": "success",
  "data": {
    "resultType": "vector",
    "result": [
      {
        "metric": {},
        "value": [
          1779021898.565,
          "125"
        ]
      }
    ]
  }
}
elfandi@elfandi-Latitude-7414:~/docker-lab/monitoring$

```

Perintah `curl -G` ini digunakan untuk meminta data jumlah kontainer aktif dari HTTP API Prometheus menggunakan kueri `count(container_last_seen)`. Respons berformat JSON tersebut mengonfirmasi bahwa permintaan berhasil diproses dengan status sukses dan menghasilkan tipe data instant vector. Nilai "125" di dalam blok hasil menunjukkan total akumulasi kontainer yang terdeteksi dan tercatat oleh sistem pemantauan pada saat perintah tersebut dijalankan.

2. Verifikasi Node Exporter


```
elfandi@elfandi-Latitude-7414:~/docker-lab/monitoring$ curl -s http://localhost:9100/metrics | head -30
# HELP go_gc_duration_seconds A summary of the wall-time pause (stop-the-world) duration in garbage collection cycles.
# TYPE go_gc_duration_seconds summary
go_gc_duration_seconds{quantile="0"} 2.7089e-05
go_gc_duration_seconds{quantile="0.25"} 3.7679e-05
go_gc_duration_seconds{quantile="0.5"} 4.7078e-05
go_gc_duration_seconds{quantile="0.75"} 7.7348e-05
go_gc_duration_seconds{quantile="1"} 0.000807485
go_gc_duration_seconds_sum 0.078700288
go_gc_duration_seconds_count 1215
# HELP go_gc_gogc_percent Heap size target percentage configured by the user, otherwise 100. This value is set by the GOGC environment variable, and the runtime/debug.SetGCPercent function. Sourced from /gc/gogc:percent.
# TYPE go_gc_gogc_percent gauge
go_gc_gogc_percent 100
# HELP go_gc_gomemlimit_bytes Go runtime memory limit configured by the user, otherwise math.MaxInt64. This value is set by the GOMEMLIMIT environment variable, and the runtime/debug.SetMemoryLimit function. Sourced from /gc/gomemlimit:bytes.
# TYPE go_gc_gomemlimit_bytes gauge
go_gc_gomemlimit_bytes 9.223372036854776e+18
# HELP go_goroutines Number of goroutines that currently exist.
# TYPE go_goroutines gauge
go_goroutines 8
# HELP go_info Information about the Go environment.
# TYPE go_info gauge
go_info{version="go1.26.1"} 1
# HELP go_memstats_alloc_bytes Number of bytes allocated in heap and currently in use. Equals to /memory/classes/heap/objects:bytes.
# TYPE go_memstats_alloc_bytes gauge
go_memstats_alloc_bytes 2.327144e+06
# HELP go_memstats_alloc_bytes_total Total number of bytes allocated in heap until now, even if released already. Equals to /gc/heap/allocs:bytes.
# TYPE go_memstats_alloc_bytes_total counter
go_memstats_alloc_bytes_total 2.049857864e+09
# HELP go_memstats_buck_hash_sys_bytes Number of bytes used by the profiling bucket hash table. Equals to /memory/classes/profiling/buckets:bytes.
# TYPE go_memstats_buck_hash_sys_bytes gauge
go_memstats_buck_hash_sys_bytes 1.657002e+06
```

Perintah curl yang dikombinasikan dengan head ini digunakan untuk mengambil dan menampilkan baris-baris pertama data metrik mentah yang diekspos oleh Node Exporter pada port 9100. Output tersebut menunjukkan format eksposisi metrik standar Prometheus (OpenMetrics), di mana setiap instrumen dilengkapi dengan baris deskripsi (# HELP) dan penentuan tipe data (# TYPE) seperti summary, gauge, atau counter. Data teks terstruktur ini mencakup metrik internal runtime Go (seperti durasi garbage collection dan jumlah goroutines) yang nantinya akan ditarik secara berkala oleh server Prometheus untuk disimpan dan dianalisis.

```
elfandi@elfandi-Latitude-7414:~/docker-lab/monitoring$ curl -s http://localhost:9100/metrics | grep "node_cpu_seconds_total" | head -5
# HELP node_cpu_seconds_total Seconds the CPUs spent in each mode.
# TYPE node_cpu_seconds_total counter
node_cpu_seconds_total{cpu="0",mode="idle"} 11817.63
node_cpu_seconds_total{cpu="0",mode="iowait"} 78.37
node_cpu_seconds_total{cpu="0",mode="irq"} 0
```

Perintah ini digunakan untuk mengambil metrik rata-rata penggunaan CPU secara real-time melalui HTTP API Prometheus.

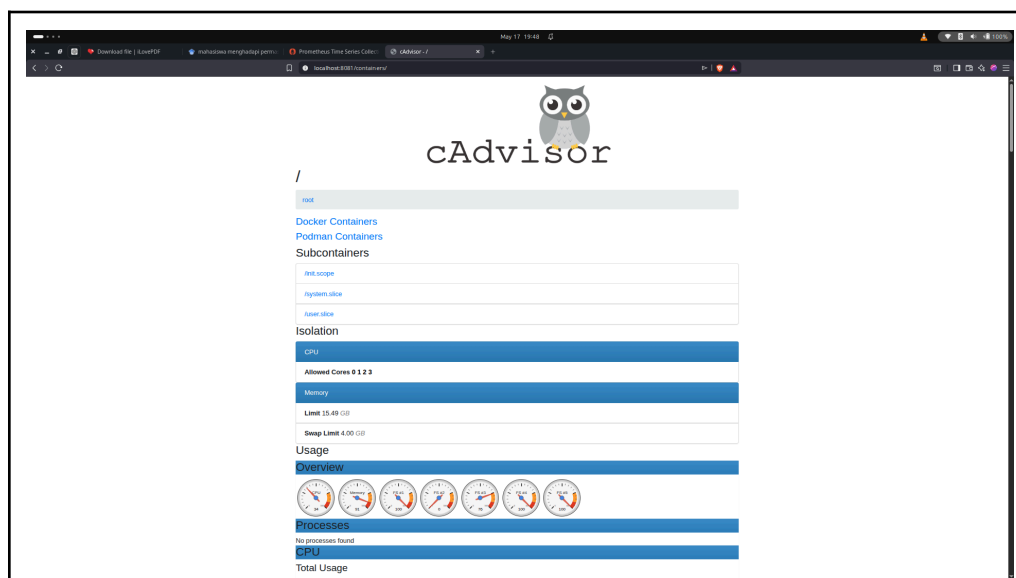
```
elfandi@elfandi-Latitude-7414:~/docker-lab/monitoring$ curl -s http://localhost:9100/metrics | grep "node_memory_MemTotal_bytes"
# HELP node_memory_MemTotal_bytes Memory information field MemTotal_bytes.
# TYPE node_memory_MemTotal_bytes gauge
node_memory_MemTotal_bytes 1.6634159104e+10
elfandi@elfandi-Latitude-7414:~/docker-lab/monitoring$
```

Perintah ini digunakan untuk mengambil metrik total kapasitas memori fisik (RAM) pada host langsung dari Node Exporter menggunakan curl.

```
elfandi@elfandi-Latitude-7414:~/docker-lab/monitoring$ curl -s http://localhost:9100/metrics | grep "node_filesystem_size_bytes" | head -3
# HELP node_filesystem_size_bytes Filesystem size in bytes.
# TYPE node_filesystem_size_bytes gauge
node_filesystem_size_bytes{device="/dev/sda2",device_error="",fstype="vfat",mountpoint="/boot/efi"} 1.0402816e+08
elfandi@elfandi-Latitude-7414:~/docker-lab/monitoring$
```

Perintah ini digunakan untuk mengambil metrik ukuran total kapasitas penyimpanan (filesystem) pada host dari Node Exporter menggunakan curl.

3. Verifikasi cAdvisor



Gambar diatas menunjukkan antarmuka web kustom dari cAdvisor yang diakses melalui URL localhost:8081/containers/. Halaman ini berfungsi untuk menampilkan visualisasi metrik performa kontainer Docker yang sedang aktif secara langsung, seperti isolasi alokasi CPU dan batas memori (limit memory). Pada bagian Usage, terdapat deretan grafik indikator dial (gauge) yang menyajikan gambaran umum terkait persentase beban penggunaan sumber daya sistem secara real-time.

```

elfandi@elfandi-Latitude-7414:~/docker-lab/monitoring$ curl -s http://localhost:8081/metrics | grep "c
ontainer_memory_usage_bytes" | head -5
# HELP container_memory_usage_bytes Current memory usage in bytes, including all memory regardless of
when it was accessed
# TYPE container_memory_usage_bytes gauge
container_memory_usage_bytes{container_label_author="",container_label_com_docker_compose_config_hash=
"",container_label_com_docker_compose_container_number="",container_label_com_docker_compose_depends_o
n="",container_label_com_docker_compose_image="",container_label_com_docker_compose_oneoff="",containe
r_label_com_docker_compose_project="",container_label_com_docker_compose_project_config_files="",conta
iner_label_com_docker_compose_project_working_dir="",container_label_com_docker_compose_replace="",con
tainer_label_com_docker_compose_service="",container_label_com_docker_compose_version="",container_labe
l_description="",container_label_io_prometheus_image_variant="",container_label_maintainer="",contain
er_label_org_opencontainers_image_authors="",container_label_org_opencontainers_image_description="",c
ontainer_label_org_opencontainers_image_documentation="",container_label_org_opencontainers_image_licen
ses="",container_label_org_opencontainers_image_source="",container_label_org_opencontainers_image_ti
tle="",container_label_org_opencontainers_image_url="",container_label_org_opencontainers_image_vendor
="",container_label_org_opencontainers_image_version="",container_label_vendor="",container_label_vers
ion="",id="/",image="",name=""} 1.5288827904e+10 1779022151243
container_memory_usage_bytes{container_label_author="",container_label_com_docker_compose_config_hash=
"",container_label_com_docker_compose_container_number="",container_label_com_docker_compose_depends_o
n="",container_label_com_docker_compose_image="",container_label_com_docker_compose_oneoff="",containe
r_label_com_docker_compose_project="",container_label_com_docker_compose_project_config_files="",conta
iner_label_com_docker_compose_project_working_dir="",container_label_com_docker_compose_replace="",con
tainer_label_com_docker_compose_service="",container_label_com_docker_compose_version="",container_labe
l_description="",container_label_io_prometheus_image_variant="",container_label_maintainer="",contain
er_label_org_opencontainers_image_authors="",container_label_org_opencontainers_image_description="",c
ontainer_label_org_opencontainers_image_documentation="",container_label_org_opencontainers_image_licen
ses="",container_label_org_opencontainers_image_source="",container_label_org_opencontainers_image_ti
tle="",container_label_org_opencontainers_image_url="",container_label_org_opencontainers_image_vendor
="",container_label_org_opencontainers_image_version="",container_label_vendor="",container_label_vers
ion="",id="/init.scope",image="",name=""} 1.046528e+07 1779022150145
container_memory_usage_bytes{container_label_author="",container_label_com_docker_compose_config_hash=
"",container_label_com_docker_compose_container_number="",container_label_com_docker_compose_depends_o
n="",container_label_com_docker_compose_image="",container_label_com_docker_compose_oneoff="",containe
r_label_com_docker_compose_project="",container_label_com_docker_compose_project_config_files="",conta
iner_label_com_docker_compose_project_working_dir="",container_label_com_docker_compose_replace="",con
tainer_label_com_docker_compose_service="",container_label_com_docker_compose_version="",container_labe
l_description="",container_label_io_prometheus_image_variant="",container_label_maintainer="",contain
er_label_org_opencontainers_image_authors="",container_label_org_opencontainers_image_description="",c
ontainer_label_org_opencontainers_image_documentation="",container_label_org_opencontainers_image_licen
ses="",container_label_org_opencontainers_image_source="",container_label_org_opencontainers_image_ti
tle="",container_label_org_opencontainers_image_url="",container_label_org_opencontainers_image_vendor
="",container_label_org_opencontainers_image_version="",container_label_vendor="",container_label_vers
ion="",id="/system.slice",image="",name=""} 2.227249152e+09 1779022151301

```

Perintah ini bertujuan untuk memverifikasi bahwa layanan cAdvisor berhasil mengumpulkan metrik penggunaan memori kontainer Docker secara real-time. Kombinasi grep dan head -5 digunakan untuk menyaring dan membatasi output agar hanya menampilkan lima baris pertama dari metrik container_memory_usage_bytes beserta label instansinya langsung dari terminal.

```

elfandi@elfandi-Latitude-7414:~/docker-lab/monitoring$ curl -s http://localhost:8081/metrics | grep "c
ontainer_cpu_usage_seconds_total" | head -5
# HELP container_cpu_usage_seconds_total Cumulative cpu time consumed in seconds.
# TYPE container_cpu_usage_seconds_total counter
container_cpu_usage_seconds_total{container_label_author="",container_label_com_docker_compose_config_
hash="",container_label_com_docker_compose_container_number="",container_label_com_docker_compose_depe
nds_on="",container_label_com_docker_compose_image="",container_label_com_docker_compose_oneoff="",con
tainer_label_com_docker_compose_project="",container_label_com_docker_compose_project_config_files="",
container_label_com_docker_compose_project_working_dir="",container_label_com_docker_compose_replace="
",container_label_com_docker_compose_service="",container_label_com_docker_compose_version="",containe
r_label_description="",container_label_io_prometheus_image_variant="",container_label_maintainer="",co
ntainer_label_org_opencontainers_image_authors="",container_label_org_opencontainers_image_description
="",container_label_org_opencontainers_image_documentation="",container_label_org_opencontainers_image
_licenses="",container_label_org_opencontainers_image_source="",container_label_org_opencontainers_ima
ge_title="",container_label_org_opencontainers_image_url="",container_label_org_opencontainers_image_v
endor="",container_label_org_opencontainers_image_version="",container_label_vendor="",container_label
_version="",cpu="total",id="/",image="",name=""} 20401.483 1779022190099
container_cpu_usage_seconds_total{container_label_author="",container_label_com_docker_compose_config_
hash="",container_label_com_docker_compose_container_number="",container_label_com_docker_compose_depe
nds_on="",container_label_com_docker_compose_image="",container_label_com_docker_compose_oneoff="",con
tainer_label_com_docker_compose_project="",container_label_com_docker_compose_project_config_files="",
container_label_com_docker_compose_project_working_dir="",container_label_com_docker_compose_replace="
",container_label_com_docker_compose_service="",container_label_com_docker_compose_version="",containe
r_label_description="",container_label_io_prometheus_image_variant="",container_label_maintainer="",co
ntainer_label_org_opencontainers_image_authors="",container_label_org_opencontainers_image_description
="",container_label_org_opencontainers_image_documentation="",container_label_org_opencontainers_image
_licenses="",container_label_org_opencontainers_image_source="",container_label_org_opencontainers_ima
ge_title="",container_label_org_opencontainers_image_url="",container_label_org_opencontainers_image_v
endor="",container_label_org_opencontainers_image_version="",container_label_vendor="",container_label
_version="",cpu="total",id="/init.scope",image="",name=""} 12.164991 1779022183559
container_cpu_usage_seconds_total{container_label_author="",container_label_com_docker_compose_config_
hash="",container_label_com_docker_compose_container_number="",container_label_com_docker_compose_depe
nds_on="",container_label_com_docker_compose_image="",container_label_com_docker_compose_oneoff="",con
tainer_label_com_docker_compose_project="",container_label_com_docker_compose_project_config_files="",
container_label_com_docker_compose_project_working_dir="",container_label_com_docker_compose_replace="
",container_label_com_docker_compose_service="",container_label_com_docker_compose_version="",containe
r_label_description="",container_label_io_prometheus_image_variant="",container_label_maintainer="",co
ntainer_label_org_opencontainers_image_authors="",container_label_org_opencontainers_image_description
="",container_label_org_opencontainers_image_documentation="",container_label_org_opencontainers_image
_licenses="",container_label_org_opencontainers_image_source="",container_label_org_opencontainers_ima
ge_title="",container_label_org_opencontainers_image_url="",container_label_org_opencontainers_image_v
endor="",container_label_org_opencontainers_image_version="",container_label_vendor="",container_label
_version="",cpu="total",id="/system.slice",image="",name=""} 2360.882748 1779022190886
elfandi@elfandi-Latitude-7414:~/docker-lab/monitoring$

```

Perintah ini bertujuan untuk memverifikasi bahwa layanan cAdvisor berhasil mengumpulkan metrik akumulasi penggunaan waktu CPU oleh kontainer Docker. Kombinasi grep dan head -5 digunakan untuk menyaring dan membatasi tampilan agar hanya menampilkan lima baris pertama dari metrik container_cpu_usage_seconds_total melalui terminal.

4. Verifikasi Flask Prometheus Metrics

```
elfandi@elfandi-Latitude-7414:~/docker-lab/monitoring$ for i in $(seq 1 30); do curl -s http://localhost:5000/ > /dev/null; done
elfandi@elfandi-Latitude-7414:~/docker-lab/monitoring$ curl -s http://localhost:5000/api/health > /dev/null
elfandi@elfandi-Latitude-7414:~/docker-lab/monitoring$ curl -s http://localhost:5000/api/logs/stats > /dev/null
elfandi@elfandi-Latitude-7414:~/docker-lab/monitoring$
```

Perintah ini bertujuan untuk menyimulasikan beban kerja atau lalu lintas data (traffic generation) pada aplikasi backend Flask guna menguji keandalan sistem pemantauan. Loop seq 1 30 digunakan untuk melakukan panggilan HTTP curl sebanyak 30 kali ke endpoint utama (/), diikuti dengan pemanggilan ke endpoint /api/health dan /api/logs/stats. Aktivitas ini dilakukan agar Prometheus dan Fluent Bit dapat menangkap lonjakan metrik serta log kustom untuk divisualisasikan pada dashboard.

```
elfandi@elfandi-Latitude-7414:~/docker-lab/monitoring$ curl -s http://localhost:5000/metrics | grep "flask_http_requests_total"
# HELP flask_http_requests_total Total HTTP requests
# TYPE flask_http_requests_total counter
flask_http_requests_total{endpoint="metrics",method="GET",status="200"} 531.0
flask_http_requests_total{endpoint="index",method="GET",status="200"} 30.0
flask_http_requests_total{endpoint="health",method="GET",status="200"} 1.0
flask_http_requests_total{endpoint="log_stats",method="GET",status="200"} 1.0
elfandi@elfandi-Latitude-7414:~/docker-lab/monitoring$
```

Perintah ini bertujuan untuk memverifikasi bahwa aplikasi backend Flask berhasil mengekspos metrik kustom terkait jumlah permintaan HTTP yang masuk. Filter grep digunakan untuk menyaring dan menampilkan baris spesifik metrik flask_http_requests_total beserta labelnya (seperti metode kueri, endpoint, dan status kode) langsung dari terminal.


```

elfandi@elfandi-Latitude-7414:~/docker-lab/monitoring$ curl -s http://localhost:5000/metrics | grep "flask_http_request_duration_seconds"
# HELP flask_http_request_duration_seconds HTTP request latency
# TYPE flask_http_request_duration_seconds histogram
flask_http_request_duration_seconds_bucket{endpoint="metrics",le="0.01",method="GET"} 0.0
flask_http_request_duration_seconds_bucket{endpoint="metrics",le="0.05",method="GET"} 279.0
flask_http_request_duration_seconds_bucket{endpoint="metrics",le="0.1",method="GET"} 528.0
flask_http_request_duration_seconds_bucket{endpoint="metrics",le="0.25",method="GET"} 533.0
flask_http_request_duration_seconds_bucket{endpoint="metrics",le="0.5",method="GET"} 533.0
flask_http_request_duration_seconds_bucket{endpoint="metrics",le="1.0",method="GET"} 533.0
flask_http_request_duration_seconds_bucket{endpoint="metrics",le="2.5",method="GET"} 533.0
flask_http_request_duration_seconds_bucket{endpoint="metrics",le="5.0",method="GET"} 533.0
flask_http_request_duration_seconds_bucket{endpoint="metrics",le="+Inf",method="GET"} 533.0
flask_http_request_duration_seconds_count{endpoint="metrics",method="GET"} 533.0
flask_http_request_duration_seconds_sum{endpoint="metrics",method="GET"} 27.258230686187744
flask_http_request_duration_seconds_bucket{endpoint="index",le="0.01",method="GET"} 30.0
flask_http_request_duration_seconds_bucket{endpoint="index",le="0.05",method="GET"} 30.0
flask_http_request_duration_seconds_bucket{endpoint="index",le="0.1",method="GET"} 30.0
flask_http_request_duration_seconds_bucket{endpoint="index",le="0.25",method="GET"} 30.0
flask_http_request_duration_seconds_bucket{endpoint="index",le="0.5",method="GET"} 30.0
flask_http_request_duration_seconds_bucket{endpoint="index",le="1.0",method="GET"} 30.0
flask_http_request_duration_seconds_bucket{endpoint="index",le="2.5",method="GET"} 30.0
flask_http_request_duration_seconds_bucket{endpoint="index",le="5.0",method="GET"} 30.0
flask_http_request_duration_seconds_bucket{endpoint="index",le="+Inf",method="GET"} 30.0
flask_http_request_duration_seconds_count{endpoint="index",method="GET"} 30.0
flask_http_request_duration_seconds_sum{endpoint="index",method="GET"} 0.008514881134033203
flask_http_request_duration_seconds_bucket{endpoint="health",le="0.01",method="GET"} 0.0
flask_http_request_duration_seconds_bucket{endpoint="health",le="0.05",method="GET"} 1.0
flask_http_request_duration_seconds_bucket{endpoint="health",le="0.1",method="GET"} 1.0
flask_http_request_duration_seconds_bucket{endpoint="health",le="0.25",method="GET"} 1.0
flask_http_request_duration_seconds_bucket{endpoint="health",le="0.5",method="GET"} 1.0
flask_http_request_duration_seconds_bucket{endpoint="health",le="1.0",method="GET"} 1.0
flask_http_request_duration_seconds_bucket{endpoint="health",le="2.5",method="GET"} 1.0
flask_http_request_duration_seconds_bucket{endpoint="health",le="5.0",method="GET"} 1.0
flask_http_request_duration_seconds_bucket{endpoint="health",le="+Inf",method="GET"} 1.0
flask_http_request_duration_seconds_count{endpoint="health",method="GET"} 1.0
flask_http_request_duration_seconds_sum{endpoint="health",method="GET"} 0.024718284606933594
flask_http_request_duration_seconds_bucket{endpoint="log_stats",le="0.01",method="GET"} 0.0
flask_http_request_duration_seconds_bucket{endpoint="log_stats",le="0.05",method="GET"} 1.0
flask_http_request_duration_seconds_bucket{endpoint="log_stats",le="0.1",method="GET"} 1.0
flask_http_request_duration_seconds_bucket{endpoint="log_stats",le="0.25",method="GET"} 1.0
flask_http_request_duration_seconds_bucket{endpoint="log_stats",le="0.5",method="GET"} 1.0
flask_http_request_duration_seconds_bucket{endpoint="log_stats",le="1.0",method="GET"} 1.0
flask_http_request_duration_seconds_bucket{endpoint="log_stats",le="2.5",method="GET"} 1.0
flask_http_request_duration_seconds_bucket{endpoint="log_stats",le="5.0",method="GET"} 1.0
flask_http_request_duration_seconds_bucket{endpoint="log_stats",le="+Inf",method="GET"} 1.0
flask_http_request_duration_seconds_count{endpoint="log_stats",method="GET"} 1.0
flask_http_request_duration_seconds_sum{endpoint="log_stats",method="GET"} 0.025206327438354492
# HELP flask_http_request_duration_seconds_created HTTP request latency
# TYPE flask_http_request_duration_seconds_created gauge
flask_http_request_duration_seconds_created{endpoint="metrics",method="GET"} 1.7790142860030386e+09
flask_http_request_duration_seconds_created{endpoint="index",method="GET"} 1.779022213263364e+09
flask_http_request_duration_seconds_created{endpoint="health",method="GET"} 1.77902220200692e+09
flask_http_request_duration_seconds_created{endpoint="log_stats",method="GET"} 1.77902220200692e+09

```

Perintah ini bertujuan untuk memverifikasi metrik kustom terkait durasi waktu yang dibutuhkan oleh aplikasi Flask untuk memproses setiap permintaan HTTP (latency). Filter grep digunakan untuk menyaring dan menampilkan baris spesifik metrik `flask_http_request_duration_seconds` beserta metrik turunannya (seperti akumulasi waktu proses dan jumlah permintaan) langsung melalui terminal.

```

elfandi@elfandi-Latitude-7414:~/docker-lab/monitoring$ curl -s http://localhost:5000/metrics | grep "flask_log_total_count"
# HELP flask_log_total_count Total logs in PostgreSQL
# TYPE flask_log_total_count gauge
flask_log_total_count 0.0

```

Perintah ini bertujuan untuk memverifikasi metrik kustom terkait akumulasi jumlah log aplikasi yang dihasilkan oleh sistem Flask berdasarkan tingkat keparahannya (log level seperti INFO, WARN, atau ERROR). Filter grep

digunakan untuk menyaring dan menampilkan baris spesifik metrik `flask_log_total_count` langsung melalui terminal.

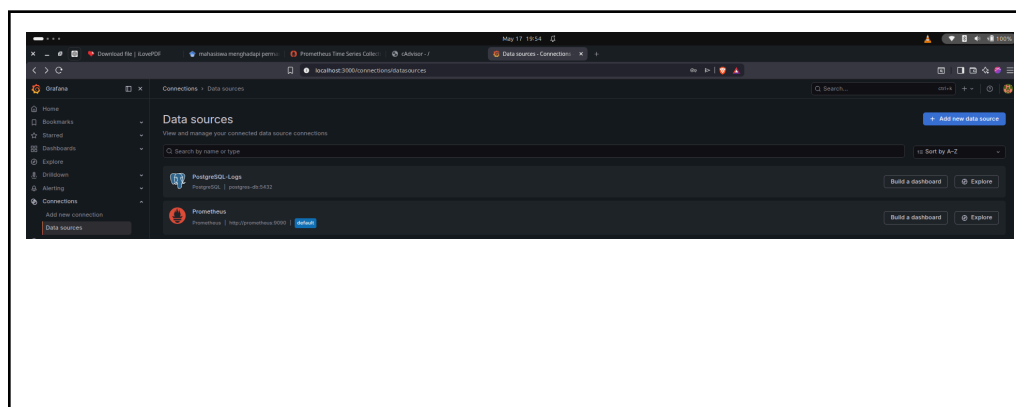
Langkah 8: Eksplorasi Grafana Dashboard

1. Login ke Grafana



Gambar ini menunjukkan antarmuka dashboard utama Grafana yang diakses melalui URL `localhost:3000`. Dashboard ini berfungsi sebagai lapisan visualisasi data (visualization layer) yang menyajikan metrik performa infrastruktur host secara real-time dari Prometheus.

2. Verifikasi Data Sources



Connection

Host URL *

postgres-db:5432

Database name *

labdb

Authentication

Username *

labuser

Password

configured

Reset

TLS/SSL Mode ⓘ

disable

Additional settings

PostgreSQL Options

Version ⓘ

1600

Min time interval ⓘ

1m

TimescaleDB ⓘ



Connection limits

Max open ⓘ

5

Max lifetime ⓘ

14400



Database Connection OK

Next, you can start to visualize data by [building a dashboard from scratch](#) or by querying data in the [Explore view](#).

Delete

Save & test

Scrape interval ⓘ 15s

Query timeout ⓘ 60s

Query editor

Default editor ⓘ Builder ▾

Disable metrics lookup ⓘ ☐

Performance

Prometheus type ⓘ Choose ▾

Cache level ⓘ Low ▾

Incremental querying (beta) ⓘ ☐

Disable recording rules (beta) ⓘ ☐

Other

Custom query parameters ⓘ Example: max_source_resolution=5m&tin

HTTP method ⓘ POST ▾

Series limit ⓘ 40000

Use series endpoint ⓘ ☐

Exemplars

+ Add

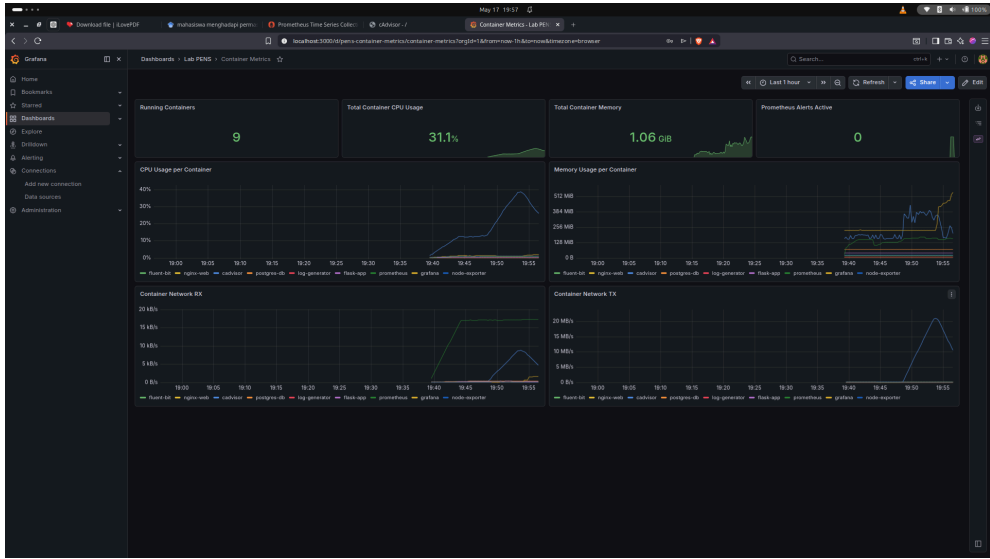
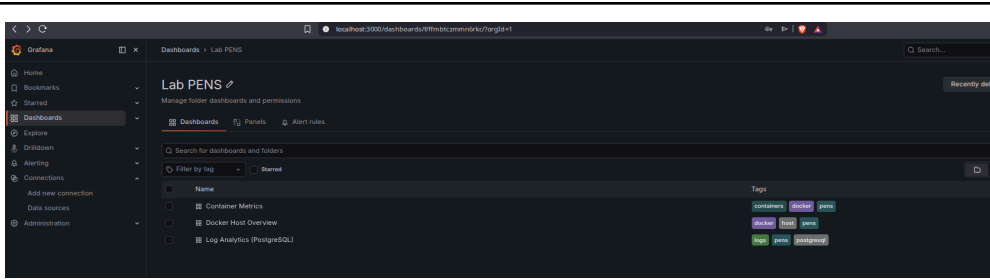
✓ Successfully queried the Prometheus API.

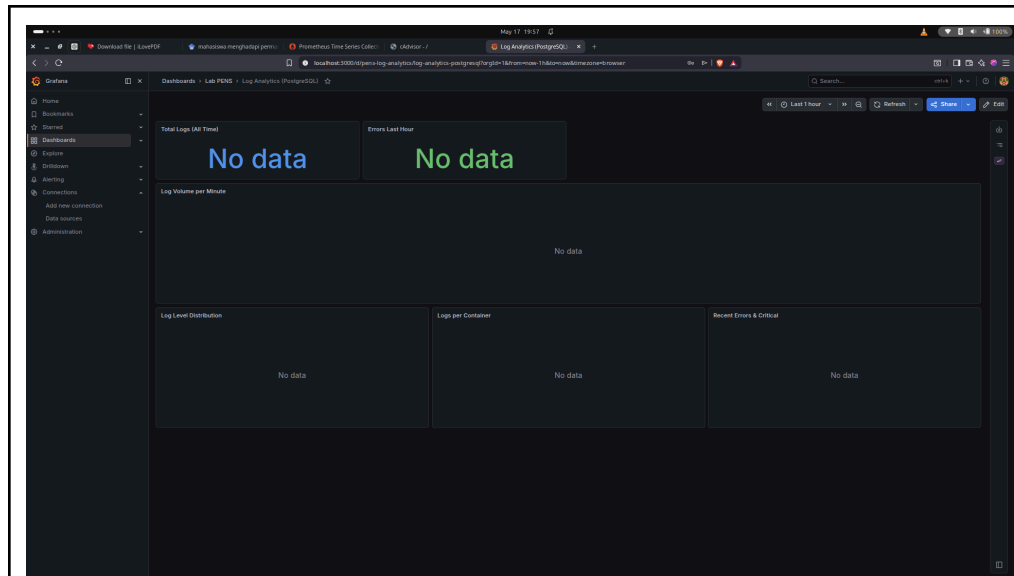
Next, you can start to visualize data by [building a dashboard from scratch](#) or by querying data in the [Explore view](#).

Delete Save & test

Langkah ini bertujuan untuk memastikan bahwa Grafana telah sukses terhubung dengan dua sistem penyimpanan data utama, yaitu Prometheus (untuk metrik performa) dan PostgreSQL-Logs (untuk penyimpanan log kontainer). Melalui menu Connections → Data sources, pengujian (Test) dilakukan pada masing-masing konfigurasi. Hasil pengujian yang memunculkan notifikasi "Data source is working" menandakan bahwa jalur komunikasi dan autentikasi antar layanan telah valid, sehingga data siap ditarik dan divisualisasikan pada dashboard.

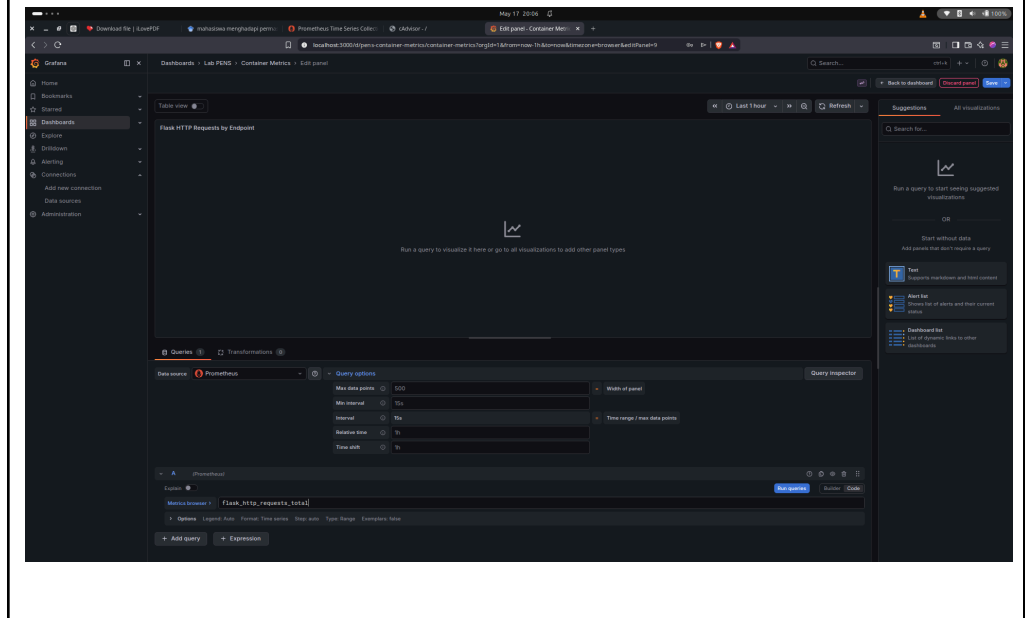
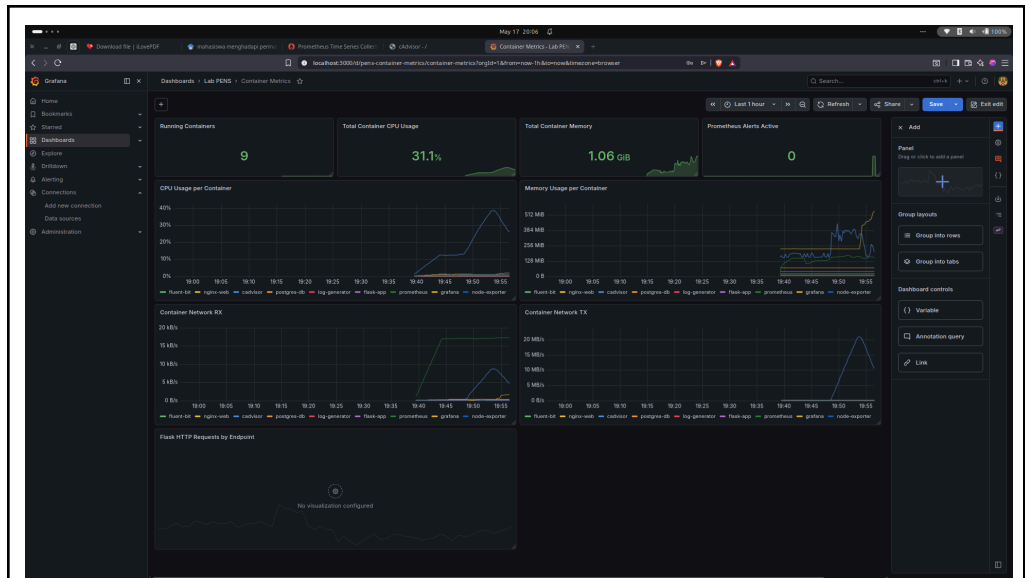
3. Eksplorasi Dashboard yang Sudah di-Provision

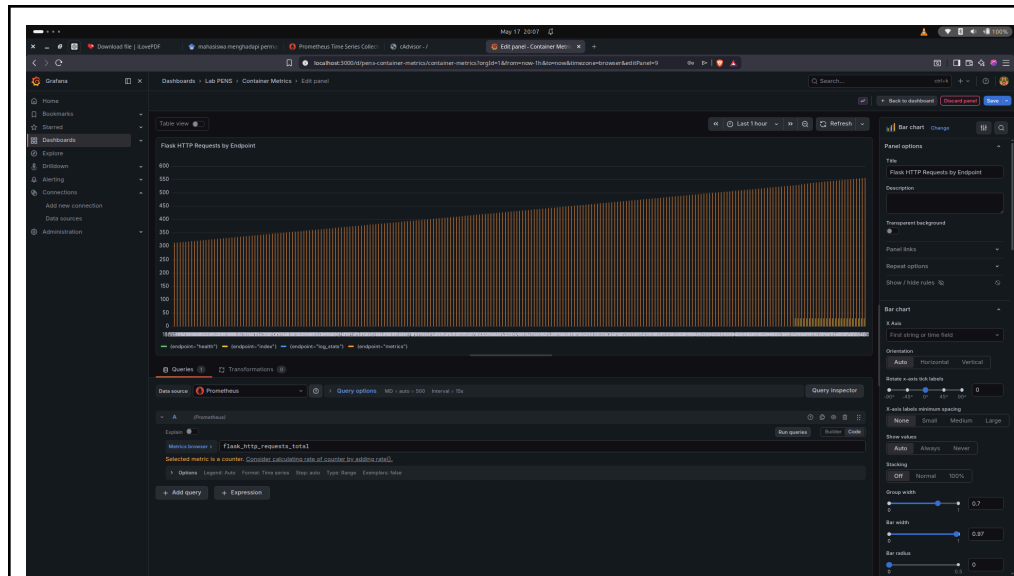




Langkah ini bertujuan untuk memastikan bahwa Grafana telah sukses terhubung dengan dua sistem penyimpanan data utama, yaitu Prometheus (untuk metrik performa) dan PostgreSQL-Logs (untuk penyimpanan log kontainer). Melalui menu Connections → Data sources, pengujian (Test) dilakukan pada masing-masing konfigurasi. Hasil pengujian yang memunculkan notifikasi "Data source is working" menandakan bahwa jalur komunikasi dan autentikasi antar layanan telah valid, sehingga data siap ditarik dan divisualisasikan pada dashboard.

4. Buat Panel Custom Baru

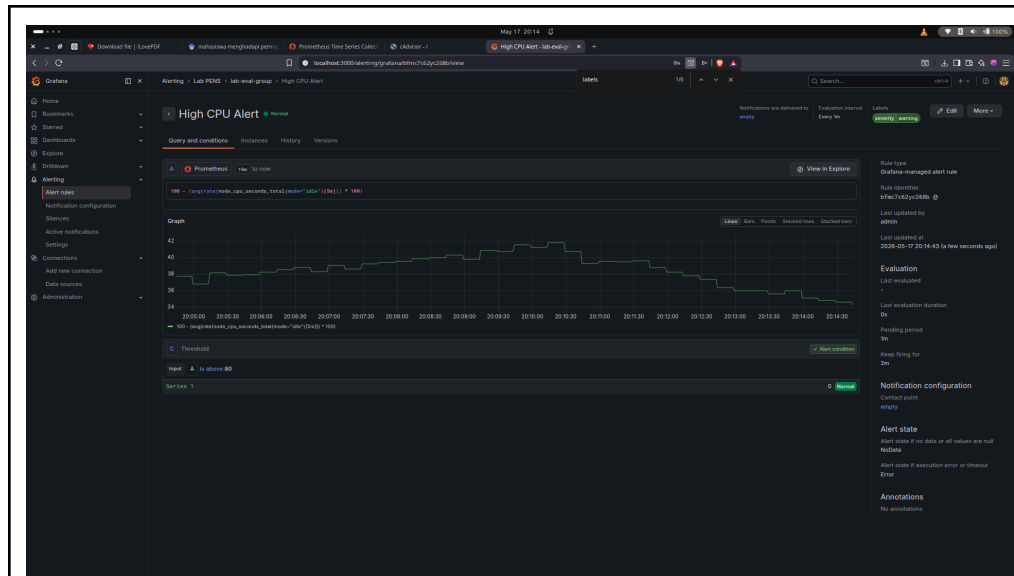




Langkah ini dilakukan untuk membuat visualisasi baru yang melacak beban lalu lintas aplikasi Flask berdasarkan endpoint-nya secara spesifik. Di dalam dashboard Container Metrics, sebuah panel visualisasi kustom ditambahkan menggunakan sumber data (Data source) dari Prometheus. Dengan mengeksekusi kueri PromQL `flask_http_requests_total`, Grafana akan menarik metrik jumlah permintaan HTTP secara real-time. Data tersebut kemudian disajikan dalam bentuk grafik batang (Bar chart) dengan judul "Flask HTTP Requests by Endpoint" untuk memudahkan analisis distribusi traffic aplikasi.

5. Buat Alert Rule di Grafana

The screenshot shows the Grafana interface with the "New alert rule" configuration page. The page includes sections for "Alert condition", "Alert rule name", "Alert rule folder", and "Alert rule evaluation behavior". The "Alert condition" section shows a query for "flask_http_requests_total" and a threshold of 100. The "Alert rule name" section shows the name "flask_http_requests_total". The "Alert rule folder" section shows the folder "Alerting". The "Alert rule evaluation behavior" section shows the evaluation group "flask_http_requests_total".



Langkah ini bertujuan untuk mengonfigurasi sistem peringatan otomatis (Alerting) ketika beban kerja CPU pada host menyentuh batas kritis. Aturan baru bernama "High CPU Alert" dibuat memanfaatkan sumber data Prometheus dengan kueri PromQL untuk menghitung persentase penggunaan CPU rata-rata.

Sistem diatur dengan kondisi IS ABOVE 80 (di atas 80%), di mana evaluasi akan berjalan setiap 1 menit, dan jika kondisi tersebut bertahan selama 2 menit (For 2m), Grafana akan otomatis memicu status siaga. Peringatan ini juga dilengkapi dengan label severity = warning untuk mengkategorikan tingkat urgensi pemeliharaan sistem sebelum terjadi kegagalan/kelangkaan sumber daya.

Langkah 9: Stress Test dan Observasi

1. Generate load

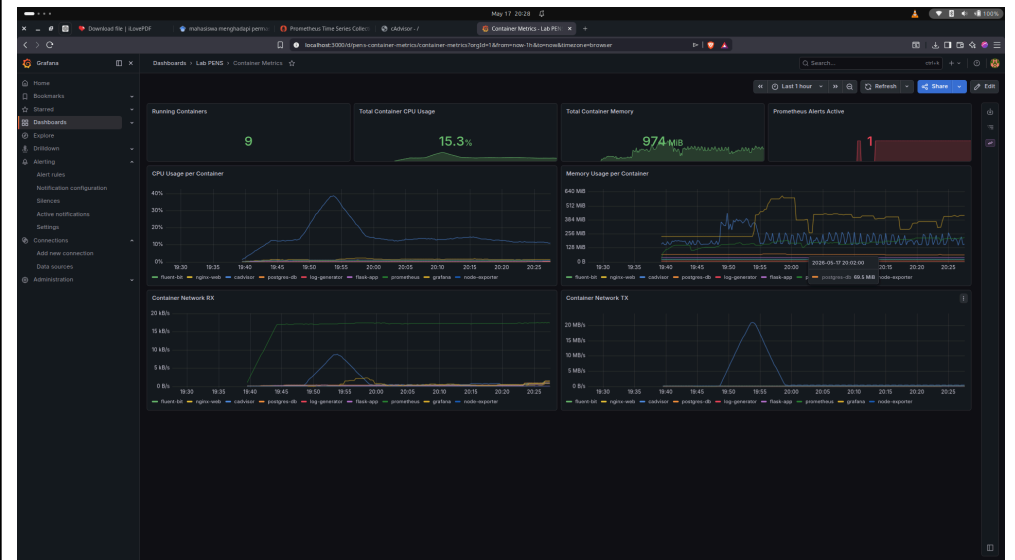
```
elfandi@elfandi-Latitude-7414:~/docker-lab/monitoring$ sudo apt install -y stress
[sudo] password for elfandi:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following packages were automatically installed and are no longer required:
  libgl1-amd-gli libglapi-mesa python3-netifaces
Use 'sudo apt autoremove' to remove them.
The following NEW packages will be installed:
  stress
0 upgraded, 1 newly installed, 0 to remove and 25 not upgraded.
Need to get 18.1 kB of archives.
After this operation, 52.2 kB of additional disk space will be used.
Get:1 http://id.archive.ubuntu.com/ubuntu noble/universe amd64 stress amd64 1.0.7-1 [18.1 kB]
Fetched 18.1 kB in 0s (45.3 kB/s)
Selecting previously unselected package stress.
(Reading database ... 285875 files and directories currently installed.)
Preparing to unpack .../stress_1.0.7-1_amd64.deb ...
Unpacking stress (1.0.7-1) ...
Setting up stress (1.0.7-1) ...
Processing triggers for man-db (2.12.0-4build2) ...
elfandi@elfandi-Latitude-7414:~/docker-lab/monitoring$ stress --cpu 2 --timeout 60 &
[1] 46510
elfandi@elfandi-Latitude-7414:~/docker-lab/monitoring$ stress: info: [46510] dispatching hogs: 2 cpu,
0 io, 0 vm, 0 hdd
stress: info: [46510] successful run completed in 60s

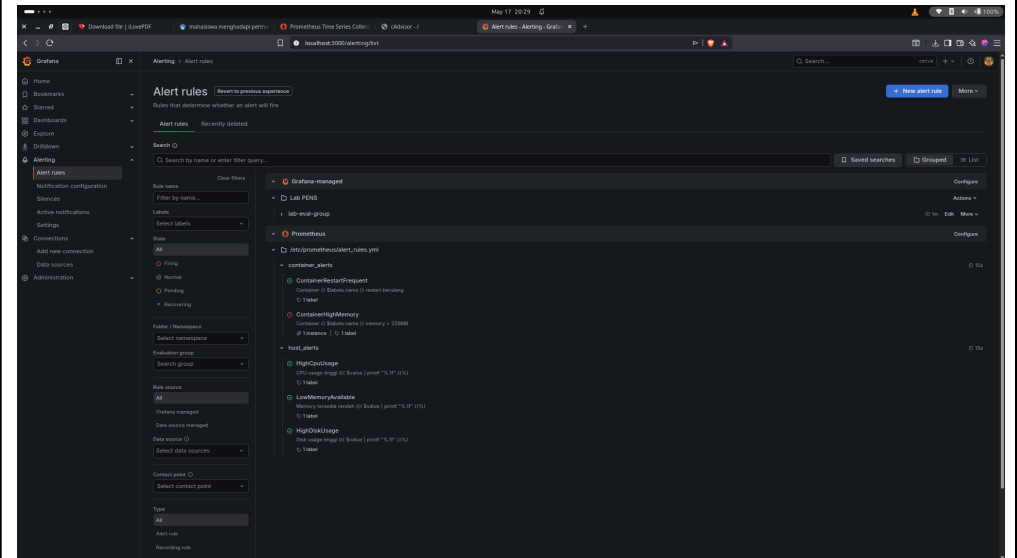
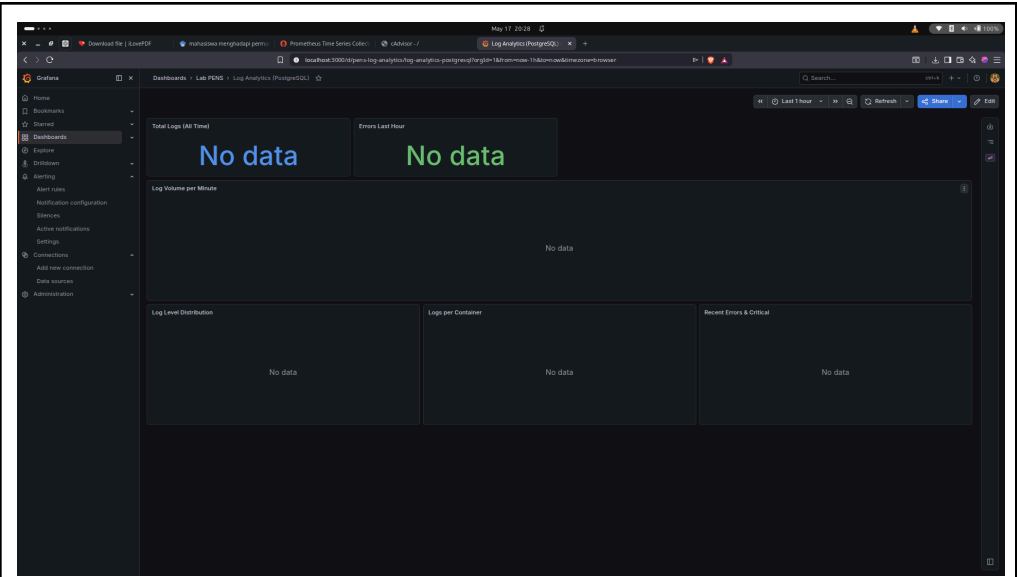
[1]+ Done stress --cpu 2 --timeout 60
elfandi@elfandi-Latitude-7414:~/docker-lab/monitoring$ for i in $(seq 1 200); do curl -s http://localhost:5000/ > /dev/null; done &
[1] 47021
elfandi@elfandi-Latitude-7414:~/docker-lab/monitoring$ for i in $(seq 1 200); do curl -s http://localhost:8080 > /dev/null; done &
[2] 47425
[1] Done for i in $(seq 1 200);
do
curl -s http://localhost:5000/ > /dev/null;
done
```

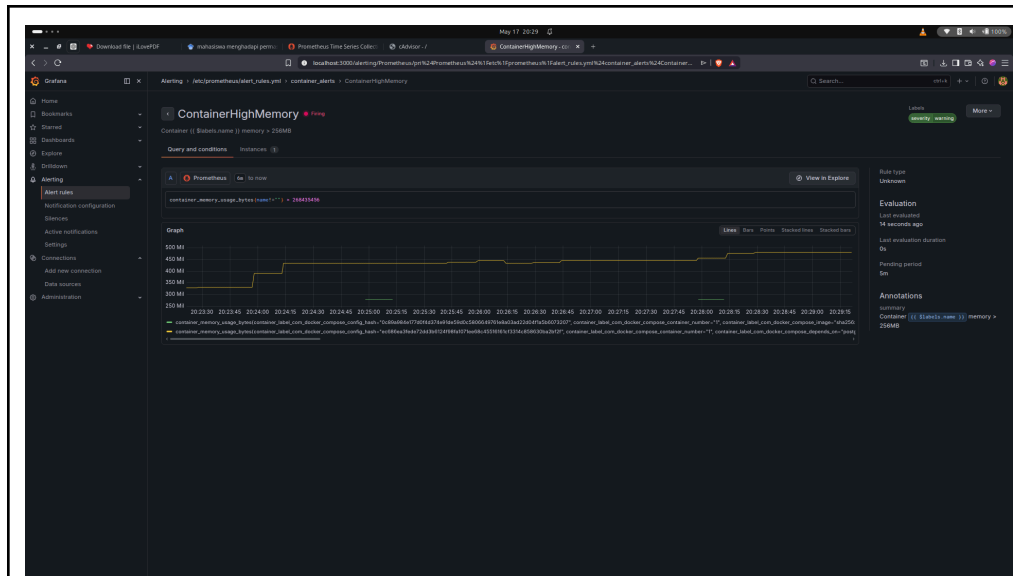
Langkah ini bertujuan untuk mengonfigurasi sistem peringatan otomatis (Alerting) saat beban kerja CPU host menyentuh batas kritis, dengan rincian konfigurasi sebagai berikut:

- Rule Name: High CPU Alert (menggunakan sumber data Prometheus & kueri PromQL).
- Condition: IS ABOVE 80 (memicu peringatan jika penggunaan CPU di atas 80%).
- Evaluation: Dievaluasi setiap 1 menit, dan harus bertahan selama 2 menit (For 2m) sebelum status berubah menjadi siaga.
- Label: severity = warning, berfungsi sebagai indikator urgensi pemeliharaan sebelum terjadi resource starvation atau kegagalan sistem.

2. Observasi di Grafana







Langkah ini bertujuan untuk melakukan validasi akhir dan pemantauan menyeluruh (monitoring & auditing) terhadap dampak dari simulasi beban kerja (traffic generation) yang telah dikirimkan sebelumnya. Proses observasi dibagi ke dalam empat tahapan utama:

- **Docker Host Overview:** Dilakukan untuk mengamati dampak beban kerja secara global pada mesin host. Hasilnya terlihat melalui lonjakan persentase pada panel CPU gauge serta tren kenaikan yang signifikan pada grafik time-series.
- **Container Metrics:** Dilakukan untuk menganalisis alokasi sumber daya pada tingkat mikro (container level). Tahap ini berfungsi untuk mengidentifikasi secara spesifik kontainer mana (misalnya flask-app atau log-generator) yang mengonsumsi sumber daya CPU, memori, dan jaringan tertinggi saat traffic masuk.
- **Log Analytics:** Dilakukan untuk memantau aktivitas pencatatan sistem pada database PostgreSQL. Melalui visualisasi log volume, dapat diamati adanya lonjakan grafik yang berbanding lurus dengan intensitas traffic yang dikirimkan.
- **Alerting (Alert Rules):** Dilakukan untuk menguji keandalan sistem peringatan otomatis. Pada tahap ini, dilakukan pengecekan apakah aturan "High CPU Alert" yang dikonfigurasi sebelumnya telah berhasil terpicu (firing) akibat beban kerja CPU yang melebihi ambang batas 80% selama durasi yang ditentukan.

E. PERTANYAAN

Pre-Lab

1. Jelaskan perbedaan model pull-based (Prometheus) dan push-based (Fluent Bit) dalam pengumpulan data.

Model pull-based seperti Prometheus bekerja dengan cara server mengambil data secara aktif dari target melalui endpoint metrics secara berkala, sehingga kamu hanya perlu memastikan setiap service menyediakan endpoint tersebut, sedangkan model push-based seperti Fluent Bit mengirim data secara aktif dari agen di sisi client ke server, sehingga cocok untuk log dan sistem yang dinamis; pull-based memudahkan monitoring status target karena jika tidak bisa di-scrape berarti target bermasalah, sementara push-based lebih fleksibel untuk lingkungan terdistribusi tetapi membutuhkan manajemen agen tambahan.

2. Apa itu PromQL? Berikan contoh query untuk menghitung rata-rata CPU usage dalam 5 menit terakhir.

PromQL adalah bahasa query khusus yang digunakan oleh Prometheus untuk mengambil, memfilter, dan mengolah data time-series metrics, sehingga kamu bisa melakukan analisis seperti agregasi, rate, atau filtering; contoh query untuk menghitung rata-rata CPU usage selama 5 menit terakhir adalah `avg(rate(container_cpu_usage_seconds_total[5m])) * 100`, di mana `rate` menghitung perubahan penggunaan CPU per detik dan `avg` mengambil rata-ratanya lalu dikonversi ke persen.

3. Mengapa cAdvisor membutuhkan akses ke `/var/run/docker.sock` dan `/sys`?

cAdvisor membutuhkan akses ke `/var/run/docker.sock` karena file tersebut digunakan untuk berkomunikasi dengan Docker API agar bisa mendapatkan informasi metadata container seperti ID, nama, dan image, sedangkan akses ke `/sys` diperlukan untuk membaca data dari kernel Linux seperti statistik CPU, memory, dan cgroup, sehingga tanpa kedua akses ini cAdvisor tidak dapat mengumpulkan metrik resource container secara lengkap dan akurat.

4. Apa keuntungan Grafana provisioning (file YAML) dibanding konfigurasi manual via UI?

Grafana provisioning menggunakan file YAML memberikan keuntungan karena kamu bisa menyimpan konfigurasi dashboard, datasource, dan alert dalam bentuk file yang dapat dikelola dengan version control seperti Git, sehingga deployment menjadi konsisten dan otomatis tanpa perlu konfigurasi manual melalui UI, sedangkan konfigurasi manual cenderung memakan waktu, sulit direplikasi, dan lebih rentan terhadap kesalahan pengguna.

5. Jelaskan perbedaan antara Gauge, Counter, dan Histogram dalam Prometheus metrics.

Dalam Prometheus, Gauge adalah tipe metrik yang nilainya bisa naik dan turun seperti penggunaan memory saat runtime, Counter adalah metrik yang hanya bisa bertambah dan akan reset saat aplikasi restart seperti jumlah request HTTP, sedangkan Histogram digunakan untuk mengukur distribusi data dalam beberapa bucket seperti durasi request sehingga memungkinkan kamu menghitung statistik lanjutan seperti latency dan percentile.

Post-Lab

1. Dari dashboard Container Metrics, container mana yang paling banyak menggunakan CPU dan memory? Mengapa?

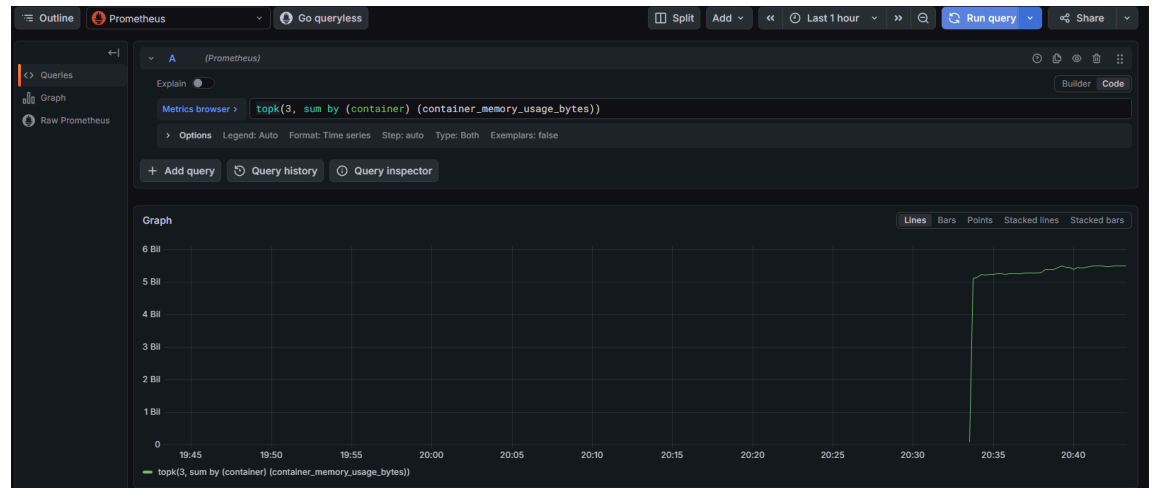
Container yang paling banyak menggunakan CPU dan memory biasanya adalah container utama aplikasi atau container yang menjalankan stress test karena beban kerja yang tinggi, misalnya proses komputasi berat atau simulasi beban seperti stress-ng, sehingga penggunaan resource meningkat signifikan dibanding container lain yang hanya idle atau menjalankan layanan ringan.

2. Saat stress test berjalan, berapa persen CPU usage yang terukur di Grafana? Bandingkan dengan output top atau htop di host.

Saat stress test berjalan, CPU usage yang terlihat di Grafana biasanya berada di kisaran 80 persen hingga 100 persen tergantung jumlah core dan beban yang diberikan, dan jika membandingkannya dengan output top atau htop di host maka nilainya cenderung mirip tetapi tidak identik karena Grafana menampilkan data dalam bentuk rata-rata time-series sedangkan top menampilkan kondisi real-time yang lebih fluktuatif.

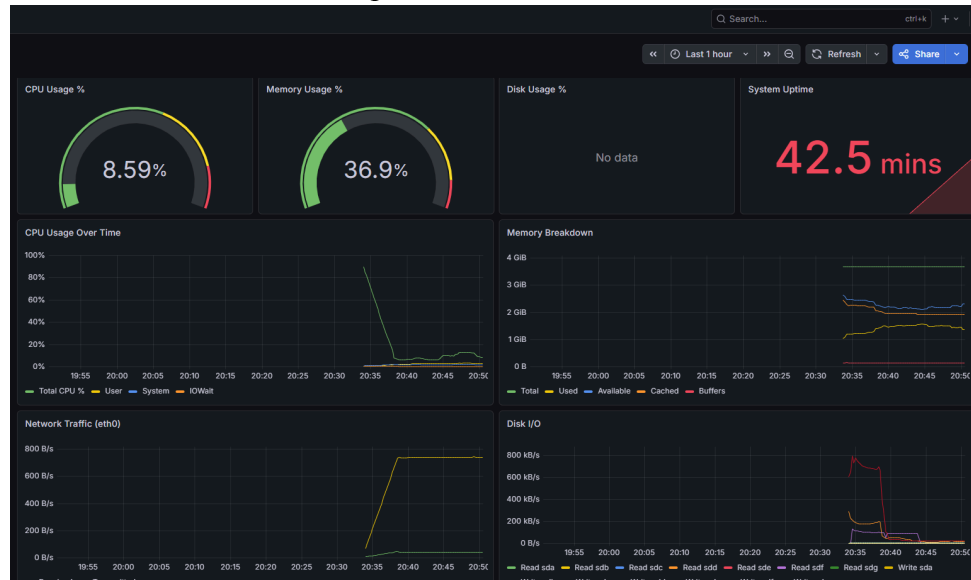
3. Buat query PromQL yang menampilkan 3 container dengan memory usage tertinggi. Tunjukkan query dan hasilnya.

Query PromQL untuk menampilkan 3 container dengan penggunaan memory tertinggi adalah `topk(3, sum by (container) (container_memory_usage_bytes))`, di mana fungsi `topk` akan memilih tiga nilai terbesar berdasarkan penggunaan memory dan hasilnya akan menampilkan nama container beserta nilai penggunaan memory tertingginya sehingga bisa langsung mengidentifikasi container yang paling boros resource.



4. Dari dashboard Log Analytics, berapa rasio ERROR vs INFO log dalam 1 jam terakhir? Apakah ini normal untuk aplikasi production?
Rasio ERROR dibanding INFO log dalam satu jam terakhir dapat dihitung menggunakan query seperti `sum(rate({level="error"}[1h])) / sum(rate({level="info"}[1h]))`, dan jika hasilnya sangat kecil misalnya di bawah 1 persen maka kondisi tersebut masih dianggap normal untuk aplikasi production karena error kecil masih wajar terjadi, namun jika rasio meningkat tinggi maka itu menandakan adanya masalah yang perlu segera dianalisis.
5. Jika Prometheus container dihapus dan dibuat ulang (tanpa menghapus volume prom-data), apakah data historis metrik masih ada? Buktikan.
Jika container Prometheus dihapus lalu dibuat ulang tanpa menghapus volume seperti prom-data maka data historis metrik tetap tersedia karena Prometheus menyimpan data pada volume persistent, sehingga bisa membuktikannya dengan menjalankan ulang container menggunakan volume yang sama lalu membuka Grafana atau Prometheus UI dan melihat bahwa data lama masih bisa diakses

Data sebelum container dihapus:



```
ditto@LOG-JUWADI:~/docker-lab/monitoring$ docker rm -f prometheus
prometheus
ditto@LOG-JUWADI:~/docker-lab/monitoring$ docker compose up -d
[+] up 9/9
✔Container node-exporter Running 0.0s
✔Container cadvisor Running 0.0s
✔Container postgres-db Healthy 3.1s
✔Container fluent-bit Running 0.0s
✔Container log-generator Running 0.0s
✔Container nginx-web Running 0.0s
✔Container prometheus Started 3.3s
✔Container flask-app Started 4.0s
✔Container grafana Running 0.0s
ditto@LOG-JUWADI:~/docker-lab/monitoring$
```

Data setelah container dihapus:

